Relatório do Repositório

Pasta raiz analisada: D:\AEROESPACIAL\Mestrado\Cadeiras\2026.2----EA-291\trabalho 2\PIPER-1-6

Data de geração: 24/10/2026 09:28:49

Este relatório foi gerado automaticamente a partir da pasta raiz do projeto. O relatório lista a estrutura de pastas e arquivos do repositório e, em seguida, inclui o conteúdo completo apenas dos arquivos com extensão .m e .md selecionados.

Total de arquivos listados: 104

Total de arquivos .m e .md com conteúdo incluído: 86

Todas as pastas foram consideradas para inclusao de conteudo .m e .md.

Arquivo PDF salvo em: D:\AEROESPACIAL\Mestrado\Cadeiras\2026.2----EA-291\trabalho 2\PIPER-1-6\docs\relatorio_repositorio.pdf

1. Estrutura de Pastas e Arquivos

```
PIPER-1-6/
├── docs
│   ├── aula7_c.pdf
│   ├── Dissertacao_Mestrado_Sato (1).pdf
│   ├── dissertação_Marcelo.pdf
│   └── Trabalho3_compartilhado.pdf
├── guiagem
│   ├── analise_sim.m
│   ├── find_trim.m
│   ├── gui_waypoints.m
│   ├── inicializar.m
│   ├── NL_guidance.slx
│   ├── plots_guidance.mlx
│   ├── readme.md
│   ├── scr_aux_wp.m
│   ├── scr_waypoints.m
│   └── trim_cost_fn.m
├── modelos
│   ├── Não Linear
│   │   ├── aerodynamics.m
│   │   ├── aerodynamics2.m
│   │   ├── decoupling.m
│   │   ├── dyn_rigidbody.m
│   │   ├── equilibrium.m
│   │   ├── executar.m
│   │   ├── gerar_log.m
│   │   ├── ISA.m
│   │   ├── lin.m
│   │   ├── modelo.slx
│   │   ├── obs_rigidbody.m
│   │   ├── Piper J3 1_4 VT0_20.mat
│   │   ├── Piper_bkp_AVL_1_6.mat
│   │   ├── plot_long.m
│   │   ├── propulsion.m
│   │   ├── readme.md
│   │   ├── Sato_longitudinal_Piper_1_6.mat
│   │   ├── sfunction_piper.m
│   │   └── trim_vector.mat
│   └── readme.md
├── navigation
│   ├── DBN
│   │   ├── DBN_solver.m
│   │   └── init_DBN.m
│   ├── FK
│   │   ├── EKF_DI_EM_solver.m
│   │   ├── EKF_DI_solver.m
│   │   ├── EKF_INDI_EM_solver.m
│   │   ├── EKF_INDI_solver.m
│   │   ├── init_EKF_DI.m
│   │   └── init_EKF_INDI.m
│   ├── sensors
│   │   ├── ICM20689
│   │   │   ├── ins_acc_errors_icm20689.m
│   │   │   ├── ins_gyro_errors_icm20689.m
│   │   │   └── ins_init_icm20689.m
│   │   ├── IST8310
│   │   │   ├── ins_init_ist8310.m
│   │   │   └── ins_mag_meas.m
│   │   ├── NEOM8
│   │   │   ├── ins_gps_measurements.m
│   │   │   └── ins_init_gps.m
│   │   ├── ins_init_sensors.m
│   │   ├── ins_telemetry.m
│   │   └── README.md
│   ├── ins_init_block.m
│   └── README.md
├── plots
│   ├── init_plots.m
│   ├── plot3d_voo.m
│   ├── plot3d_voo_DBN.m
│   ├── plot3d_voo_xplane.m
│   ├── plot_compare_all.m
│   ├── plot_compare_all2.m
│   ├── plot_compare_all3.m
│   ├── plot_compare_all4.m
│   └── plot_compare_all5.m
├── xplane
│   ├── Xplane_Interface
│   │   ├── interface
│   │   │   ├── ins_delta_posi_calculation.m
│   │   │   ├── ins_init_xplane.m
│   │   │   ├── ins_initial_state_xplane.m
│   │   │   ├── ins_read_xplane.m
```

```
│   │   │   ├── ins_send_xplane.m
│   │   │   └── ins_wyps_insertion.m
│   │   ├── tests
│   │   │   ├── ins_initial_state_xplane_test.m
│   │   │   ├── README.md
│   │   │   ├── tests_inicializar_xplane.m
│   │   │   └── xplane_tests.slx
│   │   └── README.md
│   └── XPlaneConnect-master
│       ├── MATLAB
│       │   ├── +XPlaneConnect
│       │   │   ├── closeUDP.m
│       │   │   ├── getCTRL.m
│       │   │   ├── getDREFs.m
│       │   │   ├── getPOSI.m
│       │   │   ├── openUDP.m
│       │   │   ├── pauseSim.m
│       │   │   ├── readDATA.m
│       │   │   ├── selectDATA.m
│       │   │   ├── sendCTRL.m
│       │   │   ├── sendDATA.m
│       │   │   ├── sendDREF.m
│       │   │   ├── sendDREFs.m
│       │   │   ├── sendPOSI.m
│       │   │   ├── sendTEXT.m
│       │   │   ├── sendVIEW.m
│       │   │   ├── sendWYPT.m
│       │   │   ├── setConn.m
│       │   │   └── XPlaneConnect.jar
│       │   ├── Example
│       │   │   └── Example.m
│       │   ├── MonitorExample
│       │   │   └── MonitorExample.m
│       │   └── PlaybackExample
│       │       ├── MyRecording.txt
│       │       ├── Playback.m
│       │       └── Record.m
│       ├── .gitignore
│       ├── license.pdf
│       └── README.md
├── .gitignore
├── .python-version
├── inicializar_UI.m
└── README.md
```

2. Lista Ordenada de Arquivos

A lista abaixo apresenta os arquivos encontrados no repositório, em ordem alfabética por caminho relativo.

```
.gitignore
.python-version
docs/aula7_c.pdf
docs/Dissertacao_Mestrado_Sato (1).pdf
docs/dissertação_Marcelo.pdf
docs/Trabalho3_compartilhado.pdf
guiagem/analise_sim.m
guiagem/find_trim.m
guiagem/gui_waypoints.m
guiagem/inicializar.m
guiagem/NL_guidance.slx
guiagem/plots_guidance.mlx
guiagem/readme.md
guiagem/scr_aux_wp.m
guiagem/scr_waypoints.m
guiagem/trim_cost_fn.m
inicializar_UI.m
modelos/Não Linear/aerodynamics.m
modelos/Não Linear/aerodynamics2.m
modelos/Não Linear/decoupling.m
modelos/Não Linear/dyn_rigidbody.m
modelos/Não Linear/equilibrium.m
modelos/Não Linear/executar.m
modelos/Não Linear/gerar_log.m
modelos/Não Linear/ISA.m
modelos/Não Linear/lin.m
modelos/Não Linear/modelo.slx
modelos/Não Linear/obs_rigidbody.m
modelos/Não Linear/Piper J3 1_4 VT0_20.mat
modelos/Não Linear/Piper_bkp_AVL_1_6.mat
modelos/Não Linear/plot_long.m
modelos/Não Linear/propulsion.m
modelos/Não Linear/readme.md
modelos/Não Linear/Sato_longitudinal_Piper_1_6.mat
modelos/Não Linear/sfunction_piper.m
modelos/Não Linear/trim_vector.mat
modelos/readme.md
navigation/DBN/DBN_solver.m
navigation/DBN/init_DBN.m
navigation/FK/EKF_DI_EM_solver.m
navigation/FK/EKF_DI_solver.m
navigation/FK/EKF_INDI_EM_solver.m
navigation/FK/EKF_INDI_solver.m
navigation/FK/init_EKF_DI.m
navigation/FK/init_EKF_INDI.m
navigation/ins_init_block.m
navigation/README.md
navigation/sensors/ICM20689/ins_acc_errors_icm20689.m
navigation/sensors/ICM20689/ins_gyro_errors_icm20689.m
navigation/sensors/ICM20689/ins_init_icm20689.m
navigation/sensors/ins_init_sensors.m
navigation/sensors/ins_telemetry.m
navigation/sensors/IST8310/ins_init_ist8310.m
navigation/sensors/IST8310/ins_mag_meas.m
navigation/sensors/NEOM8/ins_gps_measurements.m
navigation/sensors/NEOM8/ins_init_gps.m
navigation/sensors/README.md
plots/init_plots.m
plots/plot3d_voo.m
plots/plot3d_voo_DBN.m
plots/plot3d_voo_xplane.m
plots/plot_compare_all.m
plots/plot_compare_all2.m
plots/plot_compare_all3.m
plots/plot_compare_all4.m
plots/plot_compare_all5.m
README.md
xplane/Xplane_Interface/interface/ins_delta_posi_calculation.m
xplane/Xplane_Interface/interface/ins_init_xplane.m
xplane/Xplane_Interface/interface/ins_initial_state_xplane.m
xplane/Xplane_Interface/interface/ins_read_xplane.m
xplane/Xplane_Interface/interface/ins_send_xplane.m
xplane/Xplane_Interface/interface/ins_wyps_insertion.m
xplane/Xplane_Interface/README.md
xplane/Xplane_Interface/tests/ins_initial_state_xplane_test.m
xplane/Xplane_Interface/tests/README.md
xplane/Xplane_Interface/tests/tests_inicializar_xplane.m
xplane/Xplane_Interface/tests/xplane_tests.slx
xplane/XPlaneConnect-master/.gitignore
xplane/XPlaneConnect-master/license.pdf
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/closeUDP.m
```

xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getCTRL.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getDREFs.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getPOSI.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/openUDP.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/pauseSim.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/readDATA.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/selectDATA.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendCTRL.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDATA.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREF.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREFs.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendPOSI.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendTEXT.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendVIEW.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendWYPT.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/setConn.m
xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/XPlaneConnect.jar
xplane/XPlaneConnect-master/MATLAB/Example/Example.m
xplane/XPlaneConnect-master/MATLAB/MonitorExample/MonitorExample.m
xplane/XPlaneConnect-master/MATLAB/PlaybackExample/MyRecording.txt
xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Playback.m
xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Record.m
xplane/XPlaneConnect-master/README.md

3. Resumo por Extensão

A contagem abaixo considera apenas nomes e extensões dos arquivos, sem leitura de conteúdo.

```
.jar: 1
.m: 77
.mat: 4
.md: 9
.mlx: 1
.pdf: 5
.slx: 3
.txt: 1
[sem extensão]: 3
```

4. Conteúdo dos Arquivos .m e .md

Todas as pastas foram consideradas para inclusao de conteudo .m e .md.

4.1. guiagem/analise_sim.m

Arquivo: guiagem/analise_sim.m

Extensão: .m

Conteúdo:

```
%% analise_sim.m - Analise pos-simulacao do NL_guidance
% Rodar APOS a simulacao terminar para verificar o comportamento
% dos controladores e identificar problemas.
%
% Uso: >> analise_sim

fprintf('\n========== ANALISE POS-SIMULACAO ==========\n\n');

%% ========== Localizar variavel ALT ==========
% O modelo pode salvar ALT diretamente no workspace OU dentro de 'out'
% (quando "Single simulation output" esta ativado no Model Settings).
ALT_data = [];

if exist('ALT', 'var')
    ALT_data = ALT;
    fprintf('  ALT encontrada no workspace (class=%s)\n', class(ALT));
elseif exist('out', 'var')
    % Tentar extrair ALT de dentro do objeto out
    try
        ALT_data = out.ALT;
        fprintf('  ALT extraida de out.ALT (class=%s)\n', class(ALT_data));
    catch
        % Listar o que tem dentro de out
        fprintf('  Variavel out existe mas out.ALT nao encontrada.\n');
        try
            who_out = out.who;
            fprintf('  Conteudo de out: %s\n', strjoin(who_out, ', '));
        catch
            fprintf('  (nao foi possivel listar conteudo de out)\n');
        end
    end
elseif exist('simOut', 'var')
    try
        ALT_data = simOut.ALT;
        fprintf('  ALT extraida de simOut.ALT (class=%s)\n', class(ALT_data));
    catch
        fprintf('  simOut existe mas simOut.ALT nao encontrada.\n');
    end
end

%% ========== Processar e plotar altitude ==========
if ~isempty(ALT_data)
    % Extrair tempo e dados conforme o formato
    if isa(ALT_data, 'timeseries')
        t_alt = ALT_data.Time;
        vals  = ALT_data.Data;
    elseif isstruct(ALT_data) && isfield(ALT_data, 'signals')
        t_alt = ALT_data.time;
        vals  = ALT_data.signals.values;
    elseif isstruct(ALT_data) && isfield(ALT_data, 'time')
        t_alt = ALT_data.time;
        vals  = ALT_data.data;
    else
        fprintf('  AVISO: formato de ALT nao reconhecido (class=%s).\n', class(ALT_data));
        t_alt = []; vals = [];
    end

    if ~isempty(t_alt)
        if size(vals, 2) >= 2
            h_ref_log = vals(:,1);
            alt_log   = vals(:,2);
        else
            h_ref_log = [];
            alt_log   = vals(:,1);
        end

        figure('Name', 'Analise Altitude');
        if ~isempty(h_ref_log)
            subplot(2,1,1);
            plot(t_alt, alt_log, 'b', t_alt, h_ref_log, 'r--', 'LineWidth', 1.5);
            legend('Altitude', 'h_{ref}'); grid on;
            title('Altitude vs Referencia');
            ylabel('Altitude (m)');

            subplot(2,1,2);
            plot(t_alt, h_ref_log - alt_log, 'k', 'LineWidth', 1.5);
            title('Erro de Altitude (h_{ref} - alt)');
            ylabel('Erro (m)'); xlabel('Tempo (s)'); grid on;
```

```
        else
            plot(t_alt, alt_log, 'b', 'LineWidth', 1.5);
            title('Altitude'); ylabel('Altitude (m)'); xlabel('Tempo (s)'); grid on;
        end

        fprintf('  Altitude: %.1f -> %.1f m (delta = %.1f m)\n', ...
            alt_log(1), alt_log(end), alt_log(end) - alt_log(1));
        if ~isempty(h_ref_log)
            fprintf('  h_ref:    %.1f -> %.1f m\n', h_ref_log(1), h_ref_log(end));
        end
    end
else
    fprintf('  AVISO: Variavel ALT nao encontrada.\n');
    fprintf('         Verifique: Model Settings > Data Import/Export >\n');
    fprintf('         desmarque "Single simulation output" OU use out.ALT\n');
end

%% ========== Verificar logsout ou ScopeData ==========
scope_vars = {'ScopeData', 'ScopeData1', 'ScopeData2', 'ScopeData3'};
for i = 1:length(scope_vars)
    if evalin('base', sprintf('exist(''%s'',''var'')', scope_vars{i}))
        fprintf('  Scope: %s encontrado no workspace\n', scope_vars{i});
    end
end

%% ========== Verificar trim ==========
fprintf('\n--- Verificacao de Trim ---\n');
Xp0 = dyn_rigidbody(0, Xe, Ue, par_gen, par_aero, par_prop);
trim_quality = norm([Xp0(1:9); Xp0(12)]);
fprintf('  trim_quality = %.6e\n', trim_quality);
fprintf('  up=%.4e vp=%.4e wp=%.4e\n', Xp0(1), Xp0(2), Xp0(3));
fprintf('  pp=%.4e qp=%.4e rp=%.4e\n', Xp0(4), Xp0(5), Xp0(6));
fprintf('  xNp=%.4f xEp=%.4f xDp=%.4f\n', Xp0(10), Xp0(11), Xp0(12));

if abs(Xp0(12)) > 0.01
    fprintf('\n  ATENCAO: xDp = %.4f m/s no trim!\n', Xp0(12));
    fprintf('  Em 180s, a altitude desvia %+.1f m.\n', -Xp0(12)*180);
end

fprintf('\n========== FIM DA ANALISE ==========\n');
```

4.2. guiagem/find_trim.m

Arquivo: guiagem/find_trim.m

Extensão: .m

Conteúdo:

```
%% find_trim.m - Encontra o ponto de equilibrio (trim) para voo reto e nivelado
% Usa fminsearch para minimizar ||Xp|| variando [delta_T, delta_e, alpha].
%
% Condicoes de trim:
%   - Voo reto e nivelado (gamma = 0 -> theta = alpha)
%   - VT = 15 m/s
%   - beta = 0, phi = 0
%   - p = q = r = 0
%   - Altitude = 560m
%
% Uso: >> find_trim
%      (roda apos executar.m ter carregado par_aero, par_prop, par_gen)

fprintf('\n========== BUSCA DE TRIM ==========\n');

%% Verificar pre-requisitos
if ~exist('par_aero','var') || ~exist('par_prop','var') || ~exist('par_gen','var')
    error('Rode executar.m primeiro para carregar par_aero, par_prop, par_gen.');
end

%% Parametros fixos
VT_trim = 15;       % m/s
h_trim  = 560;      % m (altitude)

%% Chute inicial (valores do equilibrium.m)
alpha0  = 0.29129;
dT0     = 0.05;
dE0     = -1.17391;

x0 = [dT0, dE0, alpha0];

fprintf('  Chute inicial: dT=%.6f, dE=%.6f, alpha=%.6f rad (%.3f deg)\n', ...
    x0(1), x0(2), x0(3), rad2deg(x0(3)));

%% Funcao custo como funcao anonima
% Captura VT_trim, h_trim, par_gen, par_aero, par_prop por closure
trim_cost = @(xv) trim_cost_fn(xv, VT_trim, h_trim, par_gen, par_aero, par_prop);

%% Otimizacao
opts = optimset('TolFun', 1e-16, 'TolX', 1e-12, 'MaxFunEvals', 50000, ...
                'MaxIter', 10000, 'Display', 'off');

[x_opt, fval, exitflag] = fminsearch(trim_cost, x0, opts);

dT_trim_val = x_opt(1);
dE_trim_val = x_opt(2);
alpha_trim  = x_opt(3);
theta_trim  = alpha_trim;

fprintf('\n--- Resultado do Trim ---\n');
fprintf('  delta_T = %.8f  (era %.6f)\n', dT_trim_val, dT0);
fprintf('  delta_e = %.8f  (era %.6f)\n', dE_trim_val, dE0);
fprintf('  alpha   = %.8f rad = %.4f deg  (era %.6f rad = %.3f deg)\n', ...
    alpha_trim, rad2deg(alpha_trim), alpha0, rad2deg(alpha0));
fprintf('  exitflag = %d, custo residual = %.2e\n', exitflag, fval);

%% Montar Xe e Ue novos
u_trim = VT_trim * cos(alpha_trim);
v_trim = 0;
w_trim = VT_trim * sin(alpha_trim);

Xe_new = [u_trim; v_trim; w_trim; 0; 0; 0; 0; theta_trim; 0; 0; 0; -h_trim];
Ue_new = [dT_trim_val; dE_trim_val; 0; 0];

%% Verificar
Xp_old = dyn_rigidbody(0, Xe, Ue, par_gen, par_aero, par_prop);
Xp_new = dyn_rigidbody(0, Xe_new, Ue_new, par_gen, par_aero, par_prop);
fprintf('\n--- Verificacao ---\n');
fprintf('  ||Xp_old|| = %.6e  (trim antigo)\n', norm(Xp_old));
fprintf('  ||Xp_new|| = %.6e  (trim novo)\n', norm(Xp_new));
fprintf('  Derivadas: up=%.4e  vp=%.4e  wp=%.4e\n', Xp_new(1), Xp_new(2), Xp_new(3));
fprintf('  Rotacao:   pp=%.4e  qp=%.4e  rp=%.4e\n', Xp_new(4), Xp_new(5), Xp_new(6));
fprintf('  Cinemat:   phip=%.4e  thetap=%.4e  psip=%.4e\n', Xp_new(7), Xp_new(8), Xp_new(9));
fprintf('  Posicao:   xNp=%.4f  xEp=%.4f  xDp=%.6f\n', Xp_new(10), Xp_new(11), Xp_new(12));

%% Atualizar workspace
% Excluir xNp(10) e xEp(11) do criterio - sao velocidades de translacao
% esperadas em voo reto (xNp ~ VT). Verificar apenas derivadas do corpo
```

```
% (1-9) e taxa vertical xDp(12).
trim_quality = norm([Xp_new(1:9); Xp_new(12)]);
fprintf('  trim_quality (sem xNp,xEp) = %.6e\n', trim_quality);
if trim_quality < 1.0
    Xe = Xe_new;
    Ue = Ue_new;
    TrimInput = Ue';
    Xe_init = Xe;
    h_eq = -Xe(12);

    % Recalcular WPs com nova altitude
    WPs = [
         0,    0, h_eq,      15;
       300,    0, h_eq,      15;
       300,  200, h_eq,      15;
         0,  200, h_eq,      15;
         0,    0, h_eq,      15;
    ];

    fprintf('\n  >>> Xe, Ue, TrimInput, WPs ATUALIZADOS no workspace <<<\n');
    fprintf('  VT = %.2f m/s, alpha = %.4f deg, h = %.0f m\n', ...
        VT_trim, rad2deg(alpha_trim), h_eq);
    fprintf('  TrimInput = [%.6f, %.6f, %.6f, %.6f]\n', TrimInput);
    fprintf('\n  Pode simular diretamente (Ctrl+T).\n');
else
    fprintf('\n  AVISO: Trim ainda nao converge bem (trim_quality=%.2e).\n', trim_quality);
    fprintf('  Tente variar o chute inicial ou verificar o modelo aerodinamico.\n');
    fprintf('  Para aceitar mesmo assim:\n');
    fprintf('    Xe = Xe_new; Ue = Ue_new; TrimInput = Ue''; Xe_init = Xe;\n');
end

fprintf('\n========== FIM DO TRIM ==========\n');
```

4.3. guiagem/gui_waypoints.m

Arquivo: guiagem/gui_waypoints.m

Extensão: .m

Conteúdo:

```
function gui_waypoints()
%% gui_waypoints - Interface visual para inserção de waypoints
% Permite clicar no mapa para posicionar waypoints, definir altitude e
% velocidade, e simular automaticamente o modelo NL_guidance.slx.
%
% Uso:
%   >> gui_waypoints
%
% A GUI carrega automaticamente todos os parâmetros necessários.
% Não é preciso rodar inicializar.m antes.

    %% ========== Inicialização ==========
    rootDir = fileparts(fileparts(mfilename('fullpath')));  % raiz do projeto
    % comunicação xplane
    global GlobalSocket;
    import XPlaneConnect.*;
    % Rodar inicializar.m no base workspace (carrega TODOS os parâmetros)
    % oldDir = pwd;
    % cd(rootDir);
    % evalin('base', 'inicializar');
    % cd(oldDir);

    % Importar variáveis necessárias do base workspace
    par_aero  = evalin('base', 'par_aero');
    par_prop  = evalin('base', 'par_prop');
    par_gen   = evalin('base', 'par_gen');
    Xe        = evalin('base', 'Xe');
    Ue        = evalin('base', 'Ue');
    Xe_init   = evalin('base', 'Xe_init');
    C_alt     = evalin('base', 'C_alt');
    C_theta   = evalin('base', 'C_theta');
    C_vel     = evalin('base', 'C_vel');
    C_phi     = evalin('base', 'C_phi');
    Kq        = evalin('base', 'Kq');
    Kp        = evalin('base', 'Kp');
    Kr        = evalin('base', 'Kr');
    INPUTS    = evalin('base', 'INPUTS');
    TrimInput = evalin('base', 'TrimInput');
    Kp_sas    = evalin('base', 'Kp_sas');
    h_eq      = evalin('base', 'h_eq');

    %% ========== Dados dos Waypoints ==========
    % WP1 fixo na posição inicial (0, 0, h_eq, 15)
    wp_data = [0, 0, h_eq, 15];  % matriz Nx4

    %% ========== Criar Figura ==========
    fig = uifigure('Name', 'Waypoints - Piper J-3 Cub 1/6', ...
        'Position', [100 100 1000 650], ...
        'Color', [0.95 0.95 0.95]);

    %% ========== Mapa 2D (UIAxes) ==========
    ax = uiaxes(fig, 'Position', [20 20 580 600]);
    ax.XLabel.String = 'Leste (m)';
    ax.YLabel.String = 'Norte (m)';
    ax.Title.String  = 'Clique para adicionar waypoints';
    ax.XGrid = 'on';
    ax.YGrid = 'on';
    ax.Box = 'on';
    ax.FontSize = 11;
    hold(ax, 'on');

    % Callback de clique no mapa
    ax.ButtonDownFcn = @mapClick;

    %% ========== Painel Lateral ==========
    panelX = 620;
    panelW = 360;

    % Título
    uilabel(fig, 'Position', [panelX 600 panelW 30], ...
        'Text', 'Waypoints', ...
        'FontSize', 16, 'FontWeight', 'bold', ...
        'HorizontalAlignment', 'center');

    % Tabela de waypoints
    tbl = uitable(fig, 'Position', [panelX 360 panelW 235], ...
        'ColumnName', {'#', 'Norte (m)', 'Leste (m)', 'Alt (m)', 'Vel (m/s)'}, ...
        'ColumnWidth', {30, 75, 75, 70, 70}, ...
```

```
    'ColumnEditable', [false true true true true], ...
    'CellEditCallback', @tableEdited);

% --- Botões de controle ---
btnRemover = uibutton(fig, 'Position', [panelX 320 170 30], ...
    'Text', 'Remover Último', ...
    'ButtonPushedFcn', @removeLastWP);

btnLimpar = uibutton(fig, 'Position', [panelX+180 320 170 30], ...
    'Text', 'Limpar Tudo', ...
    'ButtonPushedFcn', @clearWPs);

% --- Campos de entrada ---
yPos = 270;

uilabel(fig, 'Position', [panelX yPos 130 22], ...
    'Text', 'Altitude do próximo WP:', 'FontSize', 11);
fldAlt = uieditfield(fig, 'numeric', ...
    'Position', [panelX+170 yPos 80 22], ...
    'Value', h_eq, ...
    'Limits', [0 1000]);
uilabel(fig, 'Position', [panelX+255 yPos 30 22], 'Text', 'm');

yPos = yPos - 35;
uilabel(fig, 'Position', [panelX yPos 160 22], ...
    'Text', 'Velocidade do próximo WP:', 'FontSize', 11);
fldVel = uieditfield(fig, 'numeric', ...
    'Position', [panelX+170 yPos 80 22], ...
    'Value', 15, ...
    'Limits', [5 50]);
uilabel(fig, 'Position', [panelX+255 yPos 30 22], 'Text', 'm/s');

yPos = yPos - 35;
uilabel(fig, 'Position', [panelX yPos 160 22], ...
    'Text', 'Raio de aceitação:', 'FontSize', 11);
fldRaccept = uieditfield(fig, 'numeric', ...
    'Position', [panelX+170 yPos 80 22], ...
    'Value', 80, ...
    'Limits', [10 500]);
uilabel(fig, 'Position', [panelX+255 yPos 30 22], 'Text', 'm');

yPos = yPos - 35;
uilabel(fig, 'Position', [panelX yPos 160 22], ...
    'Text', 'Tempo de simulação:', 'FontSize', 11);
fldStopTime = uieditfield(fig, 'numeric', ...
    'Position', [panelX+170 yPos 80 22], ...
    'Value', 60, ...
    'Limits', [10 5000]);
uilabel(fig, 'Position', [panelX+255 yPos 30 22], 'Text', 's');

% --- Botão SIMULAR ---
btnSimular = uibutton(fig, 'Position', [panelX 100 panelW 50], ...
    'Text', 'SIMULAR', ...
    'FontSize', 16, 'FontWeight', 'bold', ...
    'BackgroundColor', [0.2 0.6 0.2], ...
    'FontColor', 'white', ...
    'ButtonPushedFcn', @runSimulation);

% --- Label de status ---
lblStatus = uilabel(fig, 'Position', [panelX 70 panelW 40], ...
    'Text', 'Pronto. Clique no mapa para adicionar waypoints.', ...
    'FontSize', 10, 'WordWrap', 'on', ...
    'FontColor', [0.3 0.3 0.3]);

% --- Info ---
uilabel(fig, 'Position', [panelX 20 panelW 45], ...
    'Text', 'WP1 é fixo na origem (posição inicial). A guiagem inicia mirando WP2.', ...
    'FontSize', 9, 'WordWrap', 'on', ...
    'FontColor', [0.5 0.5 0.5]);

%% ========== Desenho inicial ==========
updateTable();
updateMap();

%% =================== CALLBACKS ====================

function mapClick(~, event)
    % Adicionar waypoint na posição clicada
    pt = event.IntersectionPoint;
    leste = pt(1);
    norte = pt(2);
    alt = fldAlt.Value;
    vel = fldVel.Value;

    wp_data(end+1, :) = [norte, leste, alt, vel];
    updateTable();
```

```
    updateMap();
    lblStatus.Text = sprintf('WP%d adicionado: N=%.0f  E=%.0f  Alt=%.0f  V=%.0f', ...
        size(wp_data, 1), norte, leste, alt, vel);
end

function removeLastWP(~, ~)
    if size(wp_data, 1) > 1
        wp_data(end, :) = [];
        updateTable();
        updateMap();
        lblStatus.Text = 'Último waypoint removido.';
    else
        lblStatus.Text = 'WP1 é fixo e não pode ser removido.';
    end
end

function clearWPs(~, ~)
    wp_data = [0, 0, h_eq, 15];
    updateTable();
    updateMap();
    lblStatus.Text = 'Waypoints resetados. Apenas WP1 (origem).';
end

function tableEdited(~, event)
    row = event.Indices(1);
    col = event.Indices(2);
    % Coluna 1 é #, colunas 2-5 são Norte, Leste, Alt, Vel
    if col >= 2 && col <= 5
        wp_data(row, col-1) = event.NewData;
        updateMap();
    end
end

function updateTable()
    n = size(wp_data, 1);
    nums = (1:n)';
    tbl.Data = [num2cell(nums), num2cell(wp_data)];
end

function updateMap()
    cla(ax);
    hold(ax, 'on');

    n = size(wp_data, 1);
    R = fldRaccept.Value;
    theta_circ = linspace(0, 2*pi, 100);

    % Linhas conectando waypoints
    if n > 1
        plot(ax, wp_data(:,2), wp_data(:,1), '--', ...
            'Color', [0.5 0.5 0.5], 'LineWidth', 1);
    end

    % Waypoints e círculos de aceitação
    for i = 1:n
        norte_i = wp_data(i, 1);
        leste_i = wp_data(i, 2);

        % Círculo de aceitação
        plot(ax, leste_i + R*cos(theta_circ), norte_i + R*sin(theta_circ), ...
            'r--', 'LineWidth', 0.5);

        % Marcador
        plot(ax, leste_i, norte_i, 'rs', 'MarkerSize', 10, ...
            'MarkerFaceColor', 'r');

        % Rótulo
        text(ax, leste_i + R*0.3, norte_i + R*0.3, ...
            sprintf('WP%d\n%.0fm', i, wp_data(i,3)), ...
            'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end

    % Marcar origem
    plot(ax, 0, 0, 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');

    % Ajustar eixos
    if n > 1
        margem = max(R * 2, 100);
        xlims = [min(wp_data(:,2)) - margem, max(wp_data(:,2)) + margem];
        ylims = [min(wp_data(:,1)) - margem, max(wp_data(:,1)) + margem];
        ax.XLim = xlims;
        ax.YLim = ylims;
    else
        ax.XLim = [-200 200];
        ax.YLim = [-200 200];
    end
```

```
    ax.XLabel.String = 'Leste (m)';
    ax.YLabel.String = 'Norte (m)';
    ax.Title.String  = sprintf('Mapa de Waypoints (%d pontos)', n);

    % Habilitar clique (precisa re-setar após cla)
    ax.ButtonDownFcn = @mapClick;
    hold(ax, 'off');
end

function runSimulation(~, ~)
    % Verificar mínimo de waypoints
    if size(wp_data, 1) < 2
        lblStatus.Text = 'Adicione pelo menos 2 waypoints para simular.';
        lblStatus.FontColor = [0.8 0 0];
        return;
    end

    lblStatus.Text = 'Simulando...';
    lblStatus.FontColor = [0 0 0.6];
    drawnow;

    try
        R_accept_val = fldRaccept.Value;

        % Atribuir todas as variáveis no workspace base
        assignin('base', 'par_aero', par_aero);
        assignin('base', 'par_prop', par_prop);
        assignin('base', 'par_gen',  par_gen);
        assignin('base', 'Xe',       Xe);
        assignin('base', 'Ue',       Ue);
        assignin('base', 'Xe_init',  Xe_init);

        % Ganhos PID
        assignin('base', 'C_alt',   C_alt);
        assignin('base', 'C_theta', C_theta);
        assignin('base', 'C_vel',   C_vel);
        assignin('base', 'C_phi',   C_phi);
        assignin('base', 'Kq',      Kq);
        assignin('base', 'Kp',      Kp);
        assignin('base', 'Kr',      Kr);

        % Compatibilidade
        assignin('base', 'INPUTS',    INPUTS);
        assignin('base', 'TrimInput', TrimInput);
        assignin('base', 'Kp_sas',   Kp_sas);

        % Interpolar waypoints com diferença de altitude grande
        max_dalt = 30;  % máxima variação de altitude por trecho (m)
        wp_interp = wp_data(1, :);
        for i = 2:size(wp_data, 1)
            dalt = abs(wp_data(i,3) - wp_data(i-1,3));
            if dalt > max_dalt
                % Número de subdivisões necessárias
                n_sub = ceil(dalt / max_dalt);
                for k = 1:n_sub-1
                    frac = k / n_sub;
                    wp_mid = wp_data(i-1,:) + frac * (wp_data(i,:) - wp_data(i-1,:));
                    wp_interp(end+1, :) = wp_mid;
                end
            end
            wp_interp(end+1, :) = wp_data(i, :);
        end

        % Waypoints e raio
        assignin('base', 'WPs',      wp_interp);
        assignin('base', 'R_accept', R_accept_val);

        lblStatus.Text = sprintf('Simulando... (%ds, %d WPs interpolados)', ...
            fldStopTime.Value, size(wp_interp,1));
        drawnow;

        %% Carregar modelo e configurar
        t_stop = fldStopTime.Value;
        modelPath = fullfile(rootDir, 'guiagem', 'NL_guidance.slx');
        load_system(modelPath);
        set_param('NL_guidance', 'StopTime', num2str(t_stop));

        out = sim('NL_guidance');
        assignin('base', 'out', out);
        assignin('base', 'WPs', wp_data);
        assignin('base', 'R_accept', R_accept_val);
        %% PAUSANDO SIMULAÇÃO NO XPLANE
        if isempty(GlobalSocket)
            try
                GlobalSocket = openUDP('127.0.0.1', 49009);
                pauseSim(1, GlobalSocket);
```

```
                    fprintf("Xplane: Simulação Pausada")
                catch ME
                    disp(['pausar xplane: falha ao conectar - ' ME.message]);
                    return
                end
            else
                pauseSim(1, GlobalSocket);
                fprintf("Xplane: Simulação Pausada")
            end
            pauseSim(1, GlobalSocket);
            %% Plotar resultados
            close all;
            % chamar o função de plot
            evalin('base', 'init_plots');
            lblStatus.Text = 'Simulação concluída com sucesso!';
            lblStatus.FontColor = [0 0.5 0];

        catch ME
            lblStatus.Text = sprintf('Erro: %s', ME.message);
            lblStatus.FontColor = [0.8 0 0];
            fprintf('Erro na simulação:\n%s\n', getReport(ME));
        end
    end

end
```

4.4. guiagem/inicializar.m

Arquivo: guiagem/inicializar.m

Extensão: .m

Conteúdo:

```
%% inicializar.m - Script principal de inicializacao
% Carrega parametros da aeronave, define ganhos dos controladores PID,
% waypoints e condicao de equilibrio.
%
% Este script inicializa o workspace para AMBOS os modelos:
%   - NL_guidance.slx (guiagem por waypoints)
%   - modeloNL1.slx   (controle em malha fechada)
%
% Uso para GUIAGEM:
%   1) >> inicializar
%   2) >> open('guiagem/NL_guidance.slx')
%   3) Simular (Ctrl+T)
%   4) >> plot3d_voo          % visualizar trajetoria 3D
%
% Uso para CONTROLE:
%   1) >> inicializar
%   2) >> open('controle/Nao Linear/modeloNL1.slx')
%   3) Simular (Ctrl+T)
%
% NOTA: O modelo usa sfunction_piper (21 outputs, DirFeedthrough=0).
%       Mesmo arquivo usado em ambos os modelos Simulink.

%clear; clc;
fprintf("\n -------------- init INICIALIZAR.m --------------\n");
%% ========== Paths ==========
addpath(fullfile(rootDir, 'modelos', 'Não Linear'));  % sfunction_piper, aerodynamics, dyn_rigidbody, etc.
%% XPlane Connection
addpath(fullfile(rootDir, 'xplane','Xplane_Interface','interface')); % função realizam a conexão
addpath(fullfile(rootDir, 'xplane','XPlaneConnect-master','MATLAB'));
%% Flight Dynamics - sfunction_piper, aerodynamics, dyn_rigidbody, etc.
addpath(fullfile(rootDir, 'modelos', 'Não Linear'));
%% navigation block
addpath(fullfile(rootDir, 'navigation'));                % navigation block
addpath(fullfile(rootDir, 'navigation'));
addpath(fullfile(rootDir, 'navigation', 'sensors'));
addpath(fullfile(rootDir, 'navigation', 'sensors','ICM20689'));
addpath(fullfile(rootDir, 'navigation', 'sensors','NEOM8'));
addpath(fullfile(rootDir, 'navigation', 'sensors','IST8310'));
addpath(fullfile(rootDir, 'navigation', 'FK'));
addpath(fullfile(rootDir, 'navigation', 'DBN'));
%% Plots
addpath(fullfile(rootDir, 'plots'));
%% ========== Parametros da Aeronave ==========
% Carrega par_aero, par_prop, par_gen
load('Sato_longitudinal_Piper_1_6.mat');
%% ========== Estado de Equilibrio ==========
equilibrium;  % Define Xe (12x1) e Ue (4x1)
%% ========== Verificacao do Trim ==========
% Checa se as derivadas no ponto de equilibrio sao ~zero
Xp0 = dyn_rigidbody(0, Xe, Ue, par_gen, par_aero, par_prop);
% Excluir xNp(10) e xEp(11) do residuo — sao velocidades de translacao
% esperadas em voo reto (xNp ~ VT * cos(alpha) ~ 15 m/s)
trim_residual = norm([Xp0(1:9); Xp0(12)]);
fprintf('  Residuo do trim (corpo+xDp): %.6e\n', trim_residual);
fprintf('  xNp=%.4f m/s, xEp=%.4f m/s (velocidades de translacao)\n', Xp0(10), Xp0(11));
% NOTA: O trim foi obtido via fmincon no Simulink (trimagem_piper.m).
% Quando avaliado diretamente por dyn_rigidbody, o residuo pode ser ~0.19
% devido a diferencas na implementacao (ex: feedthrough, integracao).
% Isso NAO indica erro — o trim e correto para o modelo Simulink.
if trim_residual > 0.5
    warning('Trim NAO esta em equilibrio! Residuo = %.4f. Verifique Xe/Ue.', trim_residual);
    fprintf('  Derivadas: %s\n', mat2str(Xp0', 4));
elseif trim_residual > 0.1
    fprintf('  (residuo > 0.1 — normal para trim obtido via Simulink/fmincon)\n');
end

%% ========== Ganhos do Piloto Automatico ==========
% Valores extraidos do modeloNL1.slx.
% Os blocos PID em ambos os modelos referenciam estas variaveis do workspace.

% --- Longitudinal ---

% Altitude Hold: h_ref -> theta_ref
% Saturacao: [-0.17, 0.26] rad (anti-windup no PID do Simulink)
% AJUSTADO: ganhos reduzidos para evitar saturacao com erros pequenos
% Com Kp=0.596, erro de 0.3m ja saturava o PID -> oscilacao de theta
C_alt.Kp = 0.08;
C_alt.Ki = 0.02;
```

```
C_alt.Kd = 0.0;
C_alt.N  = 20;

% Pitch (Atitude): theta_ref -> delta_e
% AJUSTADO: ganhos reduzidos para limites fisicos do Piper 1/6 (+/-25 deg)
% Valores anteriores (Projeto 1): Kp=20.31, Ki=22.60, Kd=1.77
% Calculados via pidtune com bw=2.0 rad/s, PM=60 deg, max elev ~4 deg
C_theta.Kp = 0.260;
C_theta.Ki = 0.143;
C_theta.Kd = 0.0;
C_theta.N  = 20;

% Velocidade: VT_ref -> delta_T
% NOTA: No modeloNL1.slx os ganhos sao negativos E o Sum usa |+-
% (VT_ref - VT). Ganhos negativos com essa convencao produzem um
% controlador INVERTIDO (reduz throttle quando esta lento = instavel).
% Aqui usamos ganhos POSITIVOS para obter o comportamento correto:
% VT < VT_ref -> erro > 0 -> PID > 0 -> mais throttle.
C_vel.Kp = 0.05;
C_vel.Ki = 0.02;
C_vel.Kd = 0.01;
C_vel.N  = 20;

% SAS Arfagem (amortecimento de q)
Kq = 0.1;

% --- Latero-direcional ---

% Roll (Bank Angle Hold): phi_ref -> delta_a
% AJUSTADO: ganhos reduzidos para limites fisicos do Piper 1/6 (+/-25 deg)
% Valores anteriores (Projeto 1): Kp=26.79, Ki=13.17, Kd=-0.088
% NOTA: a planta tem baixa efetividade de aileron (B_lat(2,1)=3.296),
% entao saturacao transitoria e esperada durante manobras grandes.
% Anti-windup (clamping) habilitado no bloco PID do Simulink.
C_phi.Kp = 10.0;
C_phi.Ki = 0.0;
C_phi.Kd = 0.0;
C_phi.N  = 20;
% Saturacao: [-0.43, 0.43] rad (hardcoded no modelo)

% SAS Rolamento (amortecimento de p)
Kp = 0.119;

% Amortecedor de Guinada (washout de r)
Kr = 0.15;

% Heading -> phi: ganho = 0.8 (hardcoded no modelo Latero)
% Heading PID:    P = 2.5, I = 0 (salvo no modelo)

%% ========== Waypoints ==========
% Formato: [Norte(m), Leste(m), Altitude(m), Velocidade(m/s)]
% WP1 deve corresponder a posicao inicial da aeronave (Xe)
%
% A guiagem inicia mirando WP2 (wp_idx = 2 no t=0).
% Altitude em NED: positiva para cima (convertida internamente).

% Altitude dos WPs deve ser consistente com Xe(12).
% Xe(12) = -100 -> altitude de equilibrio = 100m
h_eq = -Xe(12);  % altitude de equilibrio

% --- Selecionar missao ---
% 1 = voo reto (teste de controle puro, sem transicoes de WP)
% 2 = triangulo (mesma altitude)
% 3 = triangulo com variacao de altitude
missao = 3;

switch missao
    case 1
        % Voo reto nivelado — WP2 muito longe, heading ~0
        WPs = [
            0,     0, h_eq, 15;     % WP1: partida
            500, 0, h_eq, 15;     % WP2: Norte (nunca alcanca)
        ];
    case 2
        % Triangulo grande (altitude constante)
        WPs = [
             0,      0, h_eq,    15;   % WP1: partida (100m)
          1000,      0, h_eq,    15;   % WP2: Norte
           500,    800, h_eq,    15;   % WP3: Nordeste
             0,      0, h_eq,    15;   % WP4: volta ao inicio
        ];
    case 3
        % Triangulo com variacao de altitude
        WPs = [
             0,      0, h_eq,      15;   % WP1: partida (100m)
          1000,      0, h_eq + 50, 15;   % WP2: Norte, sobe para 150m
```

```
           500,    800, h_eq - 30, 15;   % WP3: Nordeste, desce para 70m
             0,      0, h_eq,      15;   % WP4: volta ao inicio (100m)
        ];
end

% Raio de aceitacao para troca de waypoint (m)
R_accept = 80;

% Estado inicial passado para Guidance_Star (validacao interna)
Xe_init = Xe;

%% ========== Compatibilidade com modeloNL1.slx ==========
% O modeloNL1 (controle) usa estas variaveis nos blocos Constant.
INPUTS    = Ue';   % [delta_T, delta_e, delta_a, delta_r]
TrimInput = Ue';   % Alias (alguns blocos usam TrimInput)
Kp_sas    = Kp;    % Alias do SAS de rolamento (Kp = 0.119)

%% ========== Inertial Navigation Block Initialization ===========
ins_init_block;
%% ========== Inicia e configura conexão com Xplane ===========
ins_init_xplane;
%% ========== Pronto ==========
disp('--- Workspace carregado (guiagem + controle) ---');
disp(['  Waypoints: ' num2str(size(WPs,1)) ' pontos']);
disp(['  R_accept:  ' num2str(R_accept) ' m']);
disp(['  VT_eq:     ' num2str(norm(Xe(1:3))) ' m/s']);
disp(['  Alt_eq:    ' num2str(-Xe(12)) ' m']);
fprintf('\n  Para GUIAGEM:  open(''guiagem/NL_guidance.slx''), simular, depois plot3d_voo\n');
fprintf('  Para CONTROLE: open(''controle/Nao Linear/modeloNL1.slx''), depois simular\n');
fprintf("\n -------------- end INICIALIZAR.m --------------\n")
```

4.5. guiagem/readme.md

Arquivo: guiagem/readme.md

Extensão: .md

Conteúdo:

````
# Guiagem - Piper J-3 Cub 1/6

Sistema de guiagem por waypoints integrado ao modelo não linear 6-DOF com piloto automático.

## Arquivos

| Arquivo | Descrição |
|---------|-----------|
| `NL_guidance.slx` | Modelo Simulink: guiagem LOS + controle + planta não linear |
| `gui_waypoints.m` | Interface visual para inserção de waypoints e simulação automática |
| `plot3d_voo.m` | Plota trajetória 3D pós-simulação |
| `plots_guidance.mlx` | Live Script de plotagem para validação (3 testes) |
| `find_trim.m` | Busca alternativa de trim via `fminsearch` (legado/teste) |
| `trim_cost_fn.m` | Função custo usada por `find_trim.m` |
| `analise_sim.m` | Análise pós-sim: estatísticas e checagens dos controladores |

## Como rodar

### Opção 1: Interface visual (recomendado)

```matlab
>> gui_waypoints
```

A GUI chama `inicializar.m` automaticamente para carregar todos os parâmetros do workspace. Basta clicar no mapa
para posicionar waypoints, ajustar altitude e velocidade, e clicar **SIMULAR**.

### Opção 2: Via linha de comando

1. Na raiz do repositório, rodar `inicializar` (carrega parâmetros, ganhos e waypoints).
2. Abrir `guiagem/NL_guidance.slx` no Simulink.
3. (Opcional) Alterar `missao` no `inicializar.m` para selecionar o perfil de voo.
4. Simular (Ctrl+T).
5. Rodar `plot3d_voo` para visualizar a trajetória 3D.

```matlab
>> inicializar
>> open('guiagem/NL_guidance.slx')
>> % Simular (Ctrl+T)
>> plot3d_voo
```

## Interface Visual (gui_waypoints)

Interface gráfica para definição interativa de waypoints. A GUI executa `inicializar.m` internamente para carregar
todos os parâmetros necessários.

### Funcionalidades

- **Mapa 2D clicável** (Norte x Leste): clique para adicionar waypoints na posição desejada
- **Tabela editável**: ajuste fino de coordenadas, altitude e velocidade de cada WP
- **Altitude e velocidade padrão**: define os valores aplicados ao próximo clique no mapa
- **Raio de aceitação**: configura o R_accept usado pela guiagem (padrão: 80m)
- **Tempo de simulação**: define o StopTime máximo da simulação (padrão: 200s)
- **Botão Simular**: executa automaticamente a inicialização, simulação e plotagem 3D
- **Remover Último / Limpar Tudo**: botões para gerenciar waypoints

### Interpolação automática de altitude

Quando a diferença de altitude entre dois waypoints consecutivos é maior que 30m, a GUI insere automaticamente
waypoints intermediários ao longo do trecho. Isso evita degraus bruscos de altitude que podem saturar o
controlador.

Exemplo: WP com altitude 100m → 200m gera ~3 waypoints intermediários (100 → 133 → 167 → 200m). Os waypoints
interpolados são usados na simulação, mas os gráficos mostram apenas os waypoints originais definidos pelo usuário.

### Dicas de uso

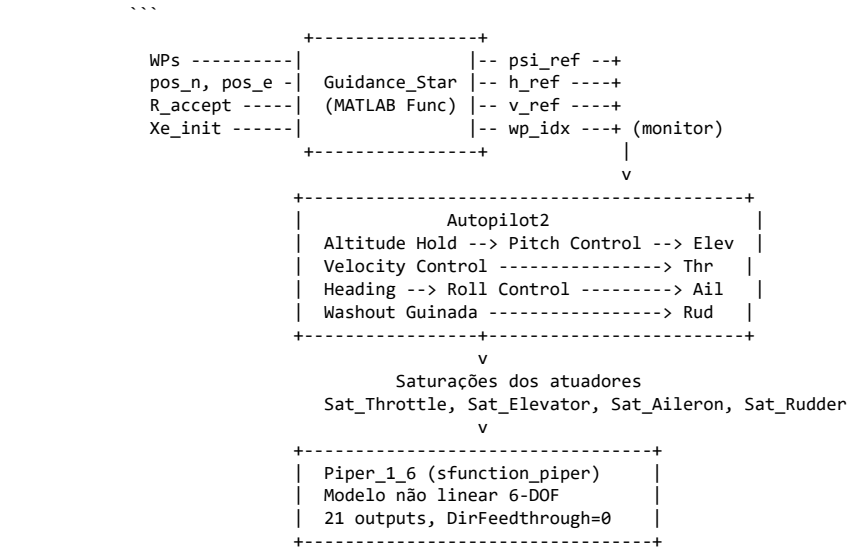
- **WP1 é fixo na origem** (0, 0, 100m) — posição inicial da aeronave
- **Espaçamento recomendado**: manter pelo menos 300-500m entre waypoints para uma navegação suave
- **R_accept**: aumentar para 150-200m se os waypoints estiverem próximos (~200m entre si)
- **Variação de altitude**: funciona bem até ±50m por trecho (a interpolação cuida do resto)
- Se a simulação terminar antes de atingir todos os WPs, aumente o tempo de simulação

## Missões disponíveis

Selecionar via variável `missao` no `inicializar.m`:
````

Missão	Descrição
1	Voo reto nivelado (teste de controle puro, sem transições de WP)
2	Triângulo com altitude constante (4 waypoints a 100m)
3	Triângulo com variação de altitude (100m → 150m → 70m → 100m)

Arquitetura



Algoritmo de Guiagem: Guidance_Star

Tipo: Line-of-Sight (LOS) puro com navegação por waypoints.

Entradas:
- `pos_n, pos_e` -- posição atual NED (m), feedback da planta
- `WPs` -- matriz de waypoints `[Norte, Leste, Altitude, Velocidade]`
- `R_accept` -- raio de aceitação para troca de waypoint (m)
- `Xe_init` -- vetor de estado inicial 12x1

Saídas:
- `psi_ref` -- heading desejado via `atan2(delta_e, delta_n)`
- `h_ref` -- altitude do waypoint alvo
- `v_ref` -- velocidade do waypoint alvo
- `wp_idx_monitor` -- índice do waypoint atual
- `dist_monitor` -- distância até o alvo

Lógica de troca de waypoint:
Quando `dist_error <= R_accept` e não é o último WP, avança para o próximo.

Função auxiliar: calc_erro_proa

Normaliza o erro de heading para `[-pi, +pi]` garantindo que a aeronave vire pelo lado mais curto:

```matlab
erro_psi = mod(diff + pi, 2*pi) - pi;
```

Ganhos do Piloto Automático

Os ganhos são carregados do workspace via `inicializar.m`. Os mesmos valores absolutos são usados no
`modeloNL1.slx` (controle). Os blocos PID no modelo referenciam diretamente as variáveis do workspace.

Longitudinal

Malha	Variável	Tipo	P	I	D	N	Saturação	Anti-Windup
Altitude Hold	`C_alt`	PID	0.08	0.02	0.0	20	[-0.17, 0.26]	clamping
Pitch (Atitude)	`C_theta`	PID	0.260	0.143	0.0	20	[-0.4363, 0.4363]	clamping
SAS Arfagem	`Kq`	Ganho	0.1	-	-	-	-	-
Velocidade	`C_vel`	PID	0.05	0.02	0.01	20	[-0.49, 0.51]	clamping

Látero-direcional

Malha	Variável	Tipo	P	I	D	N	Saturação	Anti-Windup
Roll (Bank Angle)	`C_phi`	PID	10.0	0.0	0.0	20	[-0.43, 0.43]	back-calculation
Heading	-	P	2.5	0	0	-	[-1.0, 1.0]	-
SAS Rolamento	`Kp`	Ganho	0.119	-	-	-	-	-
Heading → phi	-	Ganho	0.3	-	-	-	-	-
Amortecedor Guinada	`Kr`	Ganho + Washout	0.15	-	-	-	filtro s/(s+1)	-

Saturações dos atuadores

Atuador	Limites	Unidade
Sat_Throttle	[0, 1]	adimensional
Sat_Elevator	[-0.4363, 0.4363]	rad (±25°)
Sat_Aileron	[-0.4363, 0.4363]	rad (±25°)
Sat_Rudder	[-0.4363, 0.4363]	rad (±25°)

Dependências

Todas carregadas por `inicializar.m`:
- `par_aero`, `par_prop`, `par_gen` (structs de parâmetros da aeronave, via .mat)
- `Xe` (vetor de estado de equilíbrio 12x1, via `equilibrium.m`)
- `TrimInput` (vetor de comandos de equilíbrio `[thr, elev, ail, rud]`)
- `WPs` (matriz de waypoints `[Norte, Leste, Alt, Vel]`)
- `R_accept` (raio de aceitação em metros)
- `C_alt`, `C_theta`, `C_vel`, `C_phi` (structs com ganhos PID)
- `Kq`, `Kp`, `Kr` (ganhos dos SAS e amortecedores)
- `Xe_init` (estado inicial passado ao Guidance_Star)

Nota: O modelo usa `sfunction_piper` (21 outputs, DirFeedthrough=0), o mesmo arquivo usado pelo modeloNL1.slx.

Testes de Validação (plots_guidance.mlx)

O Live Script gera 3 figuras:

1. **Teste 1 -- Longitudinal:** Rastreamento de altitude + desacoplamento lateral + atuadores
2. **Teste 2 -- Lateral (Dogleg):** Rastreamento de heading + aileron + manutenção de altitude em curva
3. **Teste 3 -- Acoplado:** Altitude + heading + superfícies de controle simultâneos

4.6. guiagem/scr_aux_wp.m

Arquivo: guiagem/scr_aux_wp.m

Extensão: .m

Conteúdo:

```
function scr_aux_wp(WPs_user, R_accept, t_stop, h_eq)
%% scr_aux_wp - Rotina auxiliar de preparacao e execucao da simulacao
% Chamada por scr_waypoints.m. Centraliza toda a logica que gui_waypoints.m
% executa em runSimulation: rodar inicializar.m, importar parametros,
% montar WPs (com WP1 fixo na origem e interpolacao de altitude),
% atribuir variaveis no base workspace, executar o Simulink e plotar.
%
% Argumentos:
%   WPs_user : matriz Nx4 [N, E, h, vel] com os waypoints WP2, WP3, ...
%              (WP1 na origem e inserido automaticamente).
%   R_accept : raio de aceitacao dos waypoints (m).
%   t_stop   : tempo de parada da simulacao (s).
%   h_eq     : altitude de equilibrio / altitude de WP1 (m).

    %% ========== Inicializacao ==========
    % Descobrir raiz do projeto (pai de guiagem/)
    rootDir = fileparts(fileparts(mfilename('fullpath')));

    % Rodar inicializar.m no base workspace
    oldDir = pwd;
    cd(rootDir);
    evalin('base', 'inicializar');
    cd(oldDir);

    % Importar variaveis do base workspace (carregadas por inicializar.m)
    par_aero  = evalin('base', 'par_aero');
    par_prop  = evalin('base', 'par_prop');
    par_gen   = evalin('base', 'par_gen');
    Xe        = evalin('base', 'Xe');
    Ue        = evalin('base', 'Ue');
    Xe_init   = evalin('base', 'Xe_init');
    C_alt     = evalin('base', 'C_alt');
    C_theta   = evalin('base', 'C_theta');
    C_vel     = evalin('base', 'C_vel');
    C_phi     = evalin('base', 'C_phi');
    Kq        = evalin('base', 'Kq');
    Kp        = evalin('base', 'Kp');
    Kr        = evalin('base', 'Kr');
    INPUTS    = evalin('base', 'INPUTS');
    TrimInput = evalin('base', 'TrimInput');
    Kp_sas    = evalin('base', 'Kp_sas');

    % Valor padrao de velocidade para WP1 (espelha gui_waypoints.m)
    v_wp1 = 15;

    %% ========== Montar matriz de waypoints ==========
    % WP1 fixo na origem
    wp_data = [0, 0, h_eq, v_wp1];

    % Adicionar WPs do usuario (WP2 em diante)
    if ~isempty(WPs_user)
        wp_data = [wp_data; WPs_user];
    end

    if size(wp_data, 1) < 2
        error('scr_aux_wp:WPinsuficientes', ...
              'Defina pelo menos 1 waypoint em WPs (alem do WP1 automatico).');
    end

    %% ========== Interpolar waypoints com gradiente de altitude ==========
    % Espelha a logica de gui_waypoints.m para evitar variacoes bruscas.
    max_dalt = 30;  % maxima variacao de altitude por trecho (m)
    wp_interp = wp_data(1, :);
    for i = 2:size(wp_data, 1)
        dalt = abs(wp_data(i,3) - wp_data(i-1,3));
        if dalt > max_dalt
            n_sub = ceil(dalt / max_dalt);
            for k = 1:n_sub-1
                frac = k / n_sub;
                wp_mid = wp_data(i-1,:) + frac * (wp_data(i,:) - wp_data(i-1,:));
                wp_interp(end+1, :) = wp_mid;
            end
        end
        wp_interp(end+1, :) = wp_data(i, :);
    end

    %% ========== Atribuir parametros no base workspace ==========
    % Parametros do modelo
```

```
    assignin('base', 'par_aero', par_aero);
    assignin('base', 'par_prop', par_prop);
    assignin('base', 'par_gen',  par_gen);
    assignin('base', 'Xe',       Xe);
    assignin('base', 'Ue',       Ue);
    assignin('base', 'Xe_init',  Xe_init);

    % Ganhos PID
    assignin('base', 'C_alt',   C_alt);
    assignin('base', 'C_theta', C_theta);
    assignin('base', 'C_vel',   C_vel);
    assignin('base', 'C_phi',   C_phi);
    assignin('base', 'Kq',      Kq);
    assignin('base', 'Kp',      Kp);
    assignin('base', 'Kr',      Kr);

    % Compatibilidade
    assignin('base', 'INPUTS',    INPUTS);
    assignin('base', 'TrimInput', TrimInput);
    assignin('base', 'Kp_sas',   Kp_sas);

    % Waypoints (interpolados) e raio de aceitacao
    assignin('base', 'WPs',      wp_interp);
    assignin('base', 'R_accept', R_accept);

    fprintf('scr_aux_wp: %d WPs originais, %d apos interpolacao (max_dalt=%d m).\n', ...
        size(wp_data, 1), size(wp_interp, 1), max_dalt);
    fprintf('scr_aux_wp: Simulando por %.0f s com R_accept=%.0f m.\n', ...
        t_stop, R_accept);

    %% ========== Executar Simulink ==========
    modelName  = 'NL_guidance';
    modelPath  = fullfile(rootDir, 'guiagem', [modelName '.slx']);
    load_system(modelPath);
    set_param(modelName, 'StopTime', num2str(t_stop));

    out = sim(modelName);
    assignin('base', 'out', out);

    % Salvar tambem a versao NAO interpolada para referencia nos plots
    assignin('base', 'WPs',      wp_data);
    assignin('base', 'R_accept', R_accept);

    fprintf('scr_aux_wp: Simulacao concluida.\n');

    %% ========== Plotar resultados ==========
    close all;
    evalin('base', 'plot3d_voo');
    evalin('base', 'plot3d_voo_xplane');
end
```

4.7. guiagem/scr_waypoints.m

Arquivo: guiagem/scr_waypoints.m

Extensão: .m

Conteúdo:

```
%% scr_waypoints.m - Configuracao de waypoints para simulacao
% Define waypoints via tuplas (N, E, h, vel) e parametros auxiliares,
% depois chama scr_aux_wp para executar a simulacao.
%
% Uso:
%   >> scr_waypoints
%
% Edite apenas este arquivo para configurar a missao. Nao e necessario
% rodar inicializar.m antes - ele e executado internamente por scr_aux_wp.
%
% Convencoes:
%   - Coordenadas NED (Norte, Leste, Down). Altitude h e positiva para cima.
%   - WP1 e fixo na origem (0, 0, h_eq, 15 m/s) e e inserido automaticamente.
%   - Os waypoints definidos em WPs sao usados como WP2 em diante.
%   - Se |Delta h| entre dois WPs consecutivos exceder max_dalt (30 m),
%     pontos intermediarios sao interpolados automaticamente.

%% ========== WAYPOINTS ==========
% Formato: [Norte (m), Leste (m), Altitude (m), Velocidade (m/s)]
% Defina apenas WP2, WP3, ... (WP1 na origem e adicionado automaticamente).
WPs = [
    500,    0,   80,  15;   % WP2
    500,  500,  100,  18;   % WP3
      0,  500,   80,  15;   % WP4
];

%% ========== PARAMETROS AUXILIARES ==========
R_accept = 80;        % Raio de aceitacao dos waypoints (m)
t_stop   = 200;       % Tempo de simulacao (s)
h_eq     = 100;       % Altitude de equilibrio / WP1 (m)

%% ========== EXECUTAR ==========
% Adiciona o diretorio guiagem ao path para garantir que scr_aux_wp
% seja encontrado, e chama a rotina auxiliar.
addpath(fileparts(mfilename('fullpath')));

scr_aux_wp(WPs, R_accept, t_stop, h_eq);
```

4.8. guiagem/trim_cost_fn.m

Arquivo: guiagem/trim_cost_fn.m

Extensão: .m

Conteúdo:

```
function cost = trim_cost_fn(x_opt, VT, h, par_gen, par_aero, par_prop)
%TRIM_COST_FN Funcao custo para busca de trim via fminsearch.
%   cost = trim_cost_fn(x_opt, VT, h, par_gen, par_aero, par_prop)
%
%   x_opt = [delta_T, delta_e, alpha]

    dT    = x_opt(1);
    dE    = x_opt(2);
    alpha = x_opt(3);
    theta = alpha;   % gamma = 0 (voo nivelado)

    % Velocidades no corpo
    u = VT * cos(alpha);
    v = 0;
    w = VT * sin(alpha);

    % Estado
    X = [u; v; w; 0; 0; 0; 0; theta; 0; 0; 0; -h];

    % Controle
    U = [dT; dE; 0; 0];

    % Derivadas
    Xp = dyn_rigidbody(0, X, U, par_gen, par_aero, par_prop);

    % Custo: queremos up=0, wp=0, qp=0
    % Peso maior em qp (momento de pitch) pois eh o mais sensivel
    cost = Xp(1)^2 + Xp(3)^2 + 10*Xp(5)^2;
end
```

4.9. inicializar_UI.m

Arquivo: inicializar_UI.m

Extensão: .m

Conteúdo:

```
%% Reset all variables, functions, figures and parameters
close all; clear all;
restoredefaultpath; rehash toolboxcache; close all hidden;
clear classes; clear functions; clearvars -global; clc;

%% Adding essencial paths
rootDir = fileparts(mfilename('fullpath'));
addpath(fullfile(rootDir, 'guiagem'));


%% Setting up
%% range_time_without_gps: if [0 0], GPS considered during all trajectory
% [ti tf]: during ti and tf (ti <= t <= tf) GPS and yaw will not be used
% for correction
% use [a b; c d; e f] for multiples time ranges
range_time_without_correction = [0 0];
assignin('base','range_time_without_correction',range_time_without_correction);
%% Telemetry
% activate_telemetry:
%   true  -> imprime telemetria no Command Window
%   false -> desativa telemetria
%
% consider_FK_in_telemetry:
%   true  -> imprime também velocidade/euler do filtro/estimador
%   false -> imprime apenas dados do XPlane/comandos
%
% ins_telemetry_time_interval:
%   0   -> imprime sempre que a função for chamada
%   > 0 -> imprime a cada X segundos de simulação

activate_telemetry = true;
consider_FK_in_telemetry = false;
ins_telemetry_time_interval = 3.0;

assignin('base','activate_telemetry', activate_telemetry);
assignin('base','consider_FK_in_telemetry', consider_FK_in_telemetry);
assignin('base','ins_telemetry_time_interval', ins_telemetry_time_interval);
%% Initiating simulation
inicializar; % Start parameters, models, functions and XPlane connection
gui_waypoints; % initiate UI to choose waypoints.
%% PONTOS UTILIZADOS NO TESTE
% XN   | YE
%   0  | 0
% 300  | 0
% 500  | 100
% 500  | 400
% 300  | 500
%   0  | 500
% altitude constante a 100 m
% velocidade constante a 15 m/s
%% plots
% CONFIGURE O QUE PLOTAR EM plots/init_plots.m
```

4.10. modelos/Não Linear/aerodynamics.m

Arquivo: modelos/Não Linear/aerodynamics.m

Extensão: .m

Conteúdo:

```
% aerodynamic model
% INPUTS:
% aero_var: structure with following fields
%   -> rho: air density
%   -> V: TAS
%   -> alpha: angle of attack
%   -> beta: sideslip angle
%   -> p: roll rate
%   -> q: pitch rate
%   -> r: yaw rate
%   -> ca: aileron deflection
%   -> ce: elevator deflection
%   -> cr: rudder deflection
% par_aero: parameters of aerodynamic model (generated by DataA300)
% par_gen: general parameters (generated by DataA300)
function [D,Y,L,LA,MA,NA]=aerodynamics(aero_var,par_aero,par_gen)

% lift coefficient:
CL=par_aero.CL.null+...
    par_aero.CL.alpha*aero_var.alpha+...
    par_aero.CL.q*(aero_var.q*par_gen.c/par_aero.Vref);
     %par_aero.CL.ce*aero_var.ce;

% drag coefficient:
CD=par_aero.CD.null+...
    CL^2/(pi*0.8*(par_gen.b/par_gen.c));

% pitch moment coefficient:
Cm=par_aero.Cm.null+...
    par_aero.Cm.alpha*aero_var.alpha+...
    par_aero.Cm.q*(aero_var.q*par_gen.c/par_aero.Vref)+...
    par_aero.Cm.ce*aero_var.ce;

% lateral force coefficient:
CY=par_aero.CY.beta*aero_var.beta+...
    par_aero.CY.p*(aero_var.p*par_gen.b/par_aero.Vref)+...
    par_aero.CY.r*(aero_var.r*par_gen.b/par_aero.Vref)+...
    par_aero.CY.ca*aero_var.ca+...
    par_aero.CY.cr*aero_var.cr;

% roll moment coefficient:
Cl=par_aero.Cl.beta*aero_var.beta+...
    par_aero.Cl.p*(aero_var.p*par_gen.b/par_aero.Vref)+...
    par_aero.Cl.r*(aero_var.r*par_gen.b/par_aero.Vref)+...
    par_aero.Cl.ca*aero_var.ca+...
    par_aero.Cl.cr*aero_var.cr;

% yaw moment coefficient:
Cn=par_aero.Cn.beta*aero_var.beta+...
    par_aero.Cn.p*(aero_var.p*par_gen.b/par_aero.Vref)+...
    par_aero.Cn.r*(aero_var.r*par_gen.b/par_aero.Vref)+...
    par_aero.Cn.ca*aero_var.ca+...
    par_aero.Cn.cr*aero_var.cr;

% longitudinal aerodynamics
L=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CL; % lift
D=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CD; % drag
MA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.c*Cm; % pitch moment
% lateral-directional aerodynamics
Y=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CY; % lateral force
LA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.b*Cl; % roll moment
NA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.b*Cn; % roll moment
```

4.11. modelos/Não Linear/aerodynamics2.m

Arquivo: modelos/Não Linear/aerodynamics2.m

Extensão: .m

Conteúdo:

```
% aerodynamic model
% INPUTS:
% aero_var: structure with following fields
%   -> rho: air density
%   -> V: TAS
%   -> alpha: angle of attack
%   -> beta: sideslip angle
%   -> p: roll rate
%   -> q: pitch rate
%   -> r: yaw rate
%   -> ca: aileron deflection
%   -> ce: elevator deflection
%   -> cr: rudder deflection
% par_aero: parameters of aerodynamic model (generated by DataA300)
% par_gen: general parameters (generated by DataA300)
function [D,Y,L,LA,MA,NA]=Copy_of_aerodynamics(aero_var,par_aero,par_gen)

% lift coefficient:
CL=par_aero.CL.null+...
    par_aero.CL.alpha*aero_var.alpha+...
    par_aero.CL.q*(aero_var.q*par_gen.c/(2*par_aero.Vref));
     %par_aero.CL.ce*aero_var.ce;

% drag coefficient:
CD=par_aero.CD.null+...
    CL^2/(pi*0.8*(par_gen.b/par_gen.c));

% pitch moment coefficient:
Cm=par_aero.Cm.null+...
    par_aero.Cm.alpha*aero_var.alpha+...
    par_aero.Cm.q*(aero_var.q*par_gen.c/(2*par_aero.Vref))+...
    par_aero.Cm.ce*aero_var.ce;

% lateral force coefficient:
CY=par_aero.CY.beta*aero_var.beta+...
    par_aero.CY.p*(aero_var.p*par_gen.b/(2*par_aero.Vref))+...
    par_aero.CY.r*(aero_var.r*par_gen.b/(2*par_aero.Vref))+...
    par_aero.CY.ca*aero_var.ca+...
    par_aero.CY.cr*aero_var.cr;

% roll moment coefficient:
Cl=par_aero.Cl.beta*aero_var.beta+...
    par_aero.Cl.p*(aero_var.p*par_gen.b/(2*par_aero.Vref))+...
    par_aero.Cl.r*(aero_var.r*par_gen.b/(2*par_aero.Vref))+...
    par_aero.Cl.ca*aero_var.ca+...
    par_aero.Cl.cr*aero_var.cr;

% yaw moment coefficient:
Cn=par_aero.Cn.beta*aero_var.beta+...
    par_aero.Cn.p*(aero_var.p*par_gen.b/(2*par_aero.Vref))+...
    par_aero.Cn.r*(aero_var.r*par_gen.b/(2*par_aero.Vref))+...
    par_aero.Cn.ca*aero_var.ca+...
    par_aero.Cn.cr*aero_var.cr;

% longitudinal aerodynamics
L=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CL; % lift
D=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CD; % drag
MA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.c*Cm; % pitch moment
% lateral-directional aerodynamics
Y=0.5*aero_var.rho*aero_var.V^2*par_gen.S*CY; % lateral force
LA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.b*Cl; % roll moment
NA=0.5*aero_var.rho*aero_var.V^2*par_gen.S*par_gen.b*Cn; % roll moment
```

4.12. modelos/Não Linear/decoupling.m

Arquivo: modelos/Não Linear/decoupling.m

Extensão: .m

Conteúdo:

```
% ------------------------------------------------------------------------
% Decoupling of longitudinal and lateral-directional dynamics
% ------------------------------------------------------------------------
% Author:
% Marcelo Henrique dos Santos, ITA, São José dos Campos, SP
%
% Parameters:
% * mode: [1] - Longitudinal
%         [2] - Latero-Directional
%
% * Returns de state space matrices of the UAV
%   [A,B,C,D] = decoupling(mode) returns the state space matrices A, B,
%               C and D of the UAV.
%%

function [A,B,C,D] = decoupling( mode, par_gen, par_aero, par_prop )

    %% Indexes
    %States
    idx = [1 3 5 8 12;
           2 4 6 7 9];
    %Inputs
    idu = [1 2;
           3 4];
    %Outputs
    idy = [1 2 5 8 12;
           3 4 6 7 9];

    %% Initialization of Matrices
    A = zeros(length(idx));
    B = zeros(length(idx), length(idu));
    C = zeros(length(idy));
    D = zeros(length(idx), length(idu));

    %% Linearization of the nonlinear model
    % Load inputs and states of equilibrium;
    equilibrium;

    % Linearization
    [Ax, Bx, Cx, Dx] = lin(Xe, Ue, par_gen, par_aero, par_prop);

    %% Desacoplamento

    % Longitudinal
    if mode == 1
        %x = [u w q theta h]'
        for i=1:length(idx)
            for j=1:length(idx)
                A(i,j) = Ax(idx(mode, i), idx(mode,j));
            end
        end

        %u = [delta_th delta_e]'
        for i=1:length(idu)
            for j=1:length(idx)
                B(j,i) = Bx(idx(mode, j), idu(mode,i));
            end
        end

        %y = [Vk alpha q theta h]'
        for i=1:length(idy)
            for j=1:length(idy)
                C(i,j) = Cx(idy(mode, i), idx(mode,j));
            end
        end

    % Latero-Direcional
    elseif mode == 2
        %x = [v p r phi psi]'
        for i=1:length(idx)
            for j=1:length(idx)
                A(i,j) = Ax(idx(mode, i), idx(mode,j));
            end
        end

        %u = [delta_a delta_r]'
        for i=1:length(idu)
            for j=1:length(idx)
```

```
                B(j,i) = Bx(idx(mode, j), idu(mode,i));
            end
        end

        %y = [beta p r phi psi]'
        for i=1:length(idy)
            for j=1:length(idy)
                C(i,j) = Cx(idy(mode, i), idx(mode,j));
            end
        end
    else
        disp('Erro na escolha da dinâmica do UAV.')
    end
    clear Ax Bx Cx idx idy idu;
end
```

4.13. modelos/Não Linear/dyn_rigidbody.m

Arquivo: modelos/Não Linear/dyn_rigidbody.m

Extensão: .m

Conteúdo:

```
% ------------------------------------------------
% Function - Derivatives of rigid body dynamics
% ------------------------------------------------
% Inputs:
%           1- t: Time
%           2- X: State Variables
%               X(1)  = u
%               X(2)  = v
%               X(3)  = w
%               X(4)  = p
%               X(5)  = q
%               X(6)  = r
%               X(7)  = phi
%               X(8)  = theta
%               X(9)  = psi
%               X(10) = xN
%               X(11) = xE
%               X(12) = xD
%           3- U: Commands
%               U(1)  = dt
%               U(2)  = de
%               U(3)  = da
%               U(4)  = dr
% Output:
%           1- Xp: State Derivatives
%               Xp(1) = up
%               Xp(2) = vp
%               Xp(3) = wp
%               Xp(4) = pp
%               Xp(5) = qp
%               Xp(6) = rp
%               Xp(7) = phip
%               Xp(8) = thetap
%               Xp(9) = psip
%               Xp(10) = xNp
%               Xp(11) = xEp
%               Xp(12) = xDp
function Xp = dyn_rigidbody(t,X,U,par_gen,par_aero,par_prop)
    %% Input Vector Designation
    u           = X(1);
    v           = X(2);
    w           = X(3);
    aero_var.p = X(4);
    aero_var.q = X(5);
    aero_var.r = X(6);
    phi         = X(7);
    theta       = X(8);
    psi         = X(9);
    xN          = X(10);
    xE          = X(11);
    xD          = X(12);

    aero_var.ct = U(1);
    aero_var.ce = U(2);
    aero_var.ca = U(3);
    aero_var.cr = U(4);

    %% Wind
    wx = 0;
    wy = 0;
    wz = 0;

    %% Atmosphere Model
    [aero_var.rho, ~, ~] = ISA(-xD);
     %T    = 288.15*(1-6.5e-3*(-xD)/288.15);
     %aero_var.rho  = 1013.25e2*(1-6.5e-3*(-xD)/288.15)^(5.2561)/(287.3*T);


    %% Gravity Model
    gD = 9.80665;

    % Transformation Matrix
    Rbn = [cos(theta)*cos(psi)                              cos(theta)*sin(psi)
-sin(theta)
           sin(phi)*sin(theta)*cos(psi)-cos(phi)*sin(psi) sin(phi)*sin(theta)*sin(psi)+cos(phi)*cos(psi)
sin(phi)*cos(theta)
           cos(phi)*sin(theta)*cos(psi)+sin(phi)*sin(psi) cos(phi)*sin(theta)*sin(psi)-sin(phi)*cos(psi)
cos(phi)*cos(theta)];
```

```
    Rnb = Rbn';

    %% Aerodynamic Model

    % Aerodynamics Data
    aero_var.V     = sqrt((u-wx)^2 + (v-wy)^2 + (w-wz)^2);
    aero_var.alpha = atan((w-wz)/(u-wx));
    aero_var.beta  = asin((v-wy)/aero_var.V);

    % Transformation Matrix
    Rwb = [cos(aero_var.alpha)*cos(aero_var.beta)   sin(aero_var.beta)   sin(aero_var.alpha)*cos(aero_var.beta)
          -cos(aero_var.alpha)*sin(aero_var.beta)   cos(aero_var.beta)  -sin(aero_var.alpha)*sin(aero_var.beta)
          -sin(aero_var.alpha)                              0           cos(aero_var.alpha)];
    Rbw = Rwb';

    [D, Y, L, LA, MA, NA] = aerodynamics(aero_var, par_aero, par_gen);

    %% Propulsion Model

     % Transformation Matrix
    Rpb = [cos(par_prop.aF)*cos(par_prop.bF)   sin(par_prop.bF)   sin(par_prop.aF)*cos(par_prop.bF)
          -cos(par_prop.aF)*sin(par_prop.bF)   cos(par_prop.bF)  -sin(par_prop.aF)*sin(par_prop.bF)
          -sin(par_prop.aF)                             0           cos(par_prop.aF)];
    Rbp = Rpb';

    T = propulsion(aero_var, par_prop, par_aero);

    %% Inertia Model
    Ixx = par_gen.Ixx;
    Iyy = par_gen.Iyy;
    Izz = par_gen.Izz;
    Ixz = -par_gen.Ixz;
    % Inertia Tensor Matrix
    Ti = [Ixx    0     Ixz
           0     Iyy    0
          Ixz    0     Izz];

    %% Equations of Motion

    % Force Equations
    A = Rbw*[-D Y -L]';             % F_Aerodynamic
    W = Rbn*[0 0 gD]';              % F_Weight
    T = Rbp*T;                      % F_Thrust
    Fext = ((A + T)/par_gen.m)+ W; % F_External= F_Aerodynamic + F_Thrust + F_Weight

    up = Fext(1) - aero_var.q*w + aero_var.r*v;
    vp = Fext(2) - aero_var.r*u + aero_var.p*w;
    wp = Fext(3) - aero_var.p*v + aero_var.q*u;

    %Rotation Equations
    ppqprp = inv(Ti)*[(LA + (Iyy-Izz)*aero_var.q*aero_var.r + Ixz*aero_var.p*aero_var.q)
                      (MA + (Izz-Ixx)*aero_var.p*aero_var.r + Ixz*(aero_var.r^2-aero_var.p^2))
                      (NA + (Ixx-Iyy)*aero_var.p*aero_var.q - Ixz*aero_var.q*aero_var.r)];

    % Kinematic Equations
    phip   = aero_var.p + aero_var.q*sin(phi)*tan(theta) + aero_var.r*cos(phi)*tan(theta);
    thetap = aero_var.q*cos(phi) - aero_var.r*sin(phi);
    psip   = aero_var.q*sin(phi)/cos(theta) + aero_var.r*cos(phi)/cos(theta);

    % Navigation Equations
    xNpxEpxDp = Rnb*[u v w]';

    %% State Derivatives Vector Output
    Xp = [up          % Xp(1)
          vp          % Xp(2)
          wp          % Xp(3)
          ppqprp      % Xp(4,5,6)
          phip        % Xp(7)
          thetap      % Xp(8)
          psip        % Xp(9)
          xNpxEpxDp]; % Xp(10,11,12)
end
```

4.14. modelos/Não Linear/equilibrium.m

Arquivo: modelos/Não Linear/equilibrium.m

Extensão: .m

Conteúdo:

```
%% Cálculo para a função equilibrium.m
%
% Valores de trim obtidos via trimagem_piper.m (fmincon, interior-point).
% Resultado: fval = 1.95e-11, derivadas ~zero.
%
% Condição de voo: VT=15 m/s, h=100m, voo reto nivelado.
%
% Valores ANTIGOS (do AVL, NAO eram equilibrio real — ||Xp0|| = 201):
%   Alpha_old = 0.29129 rad (16.7 deg)
%   dT_old    = 0.05
%   dE_old    = -1.17391
%
% Valores do trim_vector.mat (trimagem_piper.m com fmincon):
%   Theta = -0.121684 rad (-6.97 deg)
%   dT    = 0.491387
%   dE    = 0.015456

VT = 15;                      % m/s (velocidade de trim)
Theta = -0.121684;            % rad (obtido via trimagem_piper.m / fmincon)
Beta = 0;
Alpha = Theta;                % gamma = 0 (voo nivelado)

U = VT;                       % aproximacao u ~ VT (consistente com trimagem_piper)
V = 0;
W = VT * tan(Theta);          % w = VT * tan(theta), conforme pipertrimcostfunction

%x = [u v w p q r phi theta psi pN pE h]'
%% valor de equilíbrio para voo reto nivelado a 100m
Xe = [U;V;W;0;0;0;0;Theta;0;0;0;-100];

%% valor de equilíbrio para o caso do Xplane
% Xe = [U;V;W;0;0;0;0;Theta;0;0;0;-1682.46594];

% x = [delta_T delta_e delta_a delta_r]'
Ue = [0.491387; 0.015456; 0; 0];
```

4.15. modelos/Não Linear/executar.m

Arquivo: modelos/Não Linear/executar.m

Extensão: .m

Conteúdo:

```
clear vars;clc;
% Modelo Latero-Direcional presente no:
% load('Piper_bkp_AVL_1_6.mat');

% Modelo Longitudinal presente no:
  load('Sato_longitudinal_Piper_1_6.mat');

% Para modelo Latero-Direcional:
% [A, B, C, D] = decoupling(2, par_gen, par_aero, par_prop);
% states = {'\beta' 'p' 'r' '\phi' '\psi'};
% inputs = {'\delta_a' '\delta_r'};
% outputs = {'\beta' 'p' 'r' '\phi' '\psi'};
% aeronave = ss(A, B, C , D, 'statename',states,...
%                            'inputname',inputs,...
%                            'outputname',outputs);
% clearvars A B C D;

% Carrega entradas e estados de equilibrio;
equilibrium;

disp('executado');
```

4.16. modelos/Não Linear/gerar_log.m

Arquivo: modelos/Não Linear/gerar_log.m

Extensão: .m

Conteúdo:

```
%% Salvar xE, xN, xU após simulação e log para cada caso:

 t_m    =   out.Y.time;
 da_m   =   out.U.signals.values(:,3) ;
 dr_m   =   out.U.signals.values(:,4) ;
 dt_m   =   out.U.signals.values(:,1) ;
 VT_m   =   out.Y.signals.values(:,1) ;
 beta_m =   out.Y.signals.values(:,3) ;
 phi_m  =   out.Y.signals.values(:,7) ;
 p_m    =   out.Y.signals.values(:,4) ;
 r_m    =   out.Y.signals.values(:,6) ;
 xN     =   out.Y.signals.values(:,10);
 xE     =   out.Y.signals.values(:,11);
 xU     =   out.Y.signals.values(:,12); %%valor de -xD

 pdot_m =   out.Y.signals.values(:,13);
 rdot_m =   out.Y.signals.values(:,14);
 ay_m   =   out.Y.signals.values(:,15);



%% Para gerar log para teste no OEM

 OEM_Data_lat_Medido_comp = [t_m da_m dr_m beta_m ...
    phi_m p_m r_m pdot_m rdot_m ay_m];
save('log_Simulado_manobra5_AVL.mat', 'OEM_Data_lat_Medido_comp');

disp('log salvo');

%% salvar variaveis direto na planilha
% Modelo
% save('log_Simulado_manobra2_Modelo1.mat', 't_m', 'da_m', 'dr_m', 'beta_m', 'phi_m',...
%  'p_m', 'r_m', 'pdot_m', 'rdot_m', 'ay_m', 'xE', 'xN', 'xU');


%
% Caso T1
% save('neu_T1.mat', 'xE', 'xN', 'xU');
%
% % Caso T2
% save('neu_T2.mat', 'xE', 'xN', 'xU');

% % Caso T3
% save('neu_T3.mat', 'xE', 'xN', 'xU');

% % Caso T4
% save('neu_T4.mat', 'xE', 'xN', 'xU');

% Caso Original
% save('neu_Original.mat', 'xE', 'xN', 'xU');




figure;
plot3(xE, xN, xU, 'b-', 'LineWidth', 1.5);
grid on;
xlabel('Leste (xE) [m]');
ylabel('Norte (xN) [m]');
zlabel('Altitude [m]');
title('Trajetória 3D da Aeronave em Coordenadas Locais');


figure;

subplot(3,2,1);
plot(t_m, da_m, 'k', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\delta_a [rad]');
title('Manobra no Aileron'); grid on;

subplot(3,2,2);
plot(t_m, dr_m, 'k', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\delta_r [rad]');
title('Manobra no Leme'); grid on;

subplot(3,2,3);
plot(t_m, rad2deg(p_m), 'r', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('p [°/s]');
```

```
title('Taxa de Rolagem'); grid on;

subplot(3,2,4);
plot(t_m, rad2deg(r_m), 'm', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('r [°/s]');
title('Taxa de Guinada'); grid on;

subplot(3,2,5);
plot(t_m, rad2deg(phi_m), 'g', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\phi [°]');
title('Ângulo de Rolamento'); grid on;

subplot(3,2,6);
plot(t_m, rad2deg(beta_m), 'b', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\beta [°]');
title('Ângulo de Deslizamento Lateral'); grid on;



%-------------------------------------------------------
%% manobra e ângulos no mesmo gráfico:
% % Supondo que você já tenha carregado ou executado gerar_log.m
% % e que as variáveis a seguir estejam disponíveis:
% % t_m, da_m, dr_m, phi_m, beta_m
%
% % ==== GRÁFICO 1: Manobras Laterais ====
% figure('Name','Manobras Laterais');
%
% plot(t_m, da_m, 'k-', 'LineWidth', 1.5); hold on;
% plot(t_m, dr_m, 'b--', 'LineWidth', 1.5);
% xlabel('Tempo [s]');
% ylabel('Deflexão [rad]');
% title('Manobras Laterais: Aileron e Leme');
% legend({'\delta_a (aileron)', '\delta_r (leme)'}, 'Location', 'best');
% grid on;
%
% % ==== GRÁFICO 2: Ângulos Laterais ====
% figure('Name','Ângulos Laterais');
%
% plot(t_m, rad2deg(phi_m), 'r-', 'LineWidth', 1.5); hold on;
% plot(t_m, rad2deg(beta_m), 'g--', 'LineWidth', 1.5);
% xlabel('Tempo [s]');
% ylabel('Ângulo [°]');
% title('Ângulos Laterais: Rolamento e Deslizamento');
% legend({'\phi (rolamento)', '\beta (deslizamento)'}, 'Location', 'best');
% grid on;

%% com o ângulo e a manobra na mesma figura:
% figure('Name', 'Manobra e Ângulos');
%
% % ==== SUBPLOT 1: Manobras (δa e δr) ====
% subplot(2,1,1);
% plot(t_m, da_m, 'k-', 'LineWidth', 1.5); hold on;
% plot(t_m, dr_m, 'b--', 'LineWidth', 1.5);
% xlabel('Tempo [s]');
% ylabel('Deflexão [rad]');
% title('Manobra');
% legend({'\delta_a (aileron)', '\delta_r (leme)'}, 'Location', 'best');
% grid on;
%
% % ==== SUBPLOT 2: Ângulos (φ e β) ====
% subplot(2,1,2);
% plot(t_m, rad2deg(phi_m), 'r-', 'LineWidth', 1.5); hold on;
% plot(t_m, rad2deg(beta_m), 'g--', 'LineWidth', 1.5);
% xlabel('Tempo [s]');
% ylabel('Ângulo [°]');
% title('Ângulos: Rolamento e Deslizamento');
% legend({'\phi (rolamento)', '\beta (deslizamento)'}, 'Location', 'best');
% grid on;
%
% saveas(gcf, 'manobra_validar.png');
```

4.17. modelos/Não Linear/ISA.m

Arquivo: modelos/Não Linear/ISA.m

Extensão: .m

Conteúdo:

```
%US Standard Atmosphere in SI units
%Input : h altitute (m)
%Output : T temperature (K), p pressure (N/m^2), rho density (kg/m^3)

function [rho,T,p]=ISA(h)
h=h/1000; % m -> km
h1=11; h2=20; h3=32; a0=-6.5e-3; a2=1e-3; g=9.80665; mol=28.9644;
R0=8.31432; R=R0/mol*1e3;
T0=288.15; p0=1.01325e5; rho0=1.2250; T1=T0+a0*h1*1e3;
p1=p0*(T1/T0)^(-g/a0/R); rho1=rho0*(T1/T0)^(-g/a0/R-1); T2=T1;
p2=p1*exp(-g/R/T2*(h2-h1)*1e3); rho2=rho1*exp(-g/R/T2*(h2-h1)*1e3);
if h <= h1
%disp('troposphere');
T=T0+a0*h*1e3;
p=p0*(T/T0)^(-g/a0/R);
rho=rho0*(T/T0)^(-g/a0/R-1);
elseif h <= h2
%disp('low stratosphere');
T=T1;
p=p1*exp(-g/R/T*(h-h1)*1e3);
rho=rho1*exp(-g/R/T*(h-h1)*1e3);
elseif h <= h3
%disp('stratosphere');
T=T2+a2*(h-h2)*1e3;
p=p2*(T/T2)^(-g/a2/R);
rho=rho2*(T/T2)^(-g/a2/R-1);
end
```

4.18. modelos/Não Linear/lin.m

Arquivo: modelos/Não Linear/lin.m

Extensão: .m

Conteúdo:

```
% ------------------------------------------------
% Function - Aircraft Linearization
% ------------------------------------------------
% Inputs:
%           1- Xe: Equilibrium States
%           2- Ue: Equilibrium Inputs
% Output:
%           1- A: State Space A Matrix
%           2- B: State Space B Matrix
%           3- C: State Space C Matrix
%           4- D: State Space D Matrix

function [A,B,C,D] = lin(Xe, Ue, par_gen, par_aero, par_prop)
    % Matrix Sizes
    n = size(Xe,1);
    m = size(Ue,1);

    % A Matrix
    for j = 1:n
        dxj    = zeros(n,1);
        dxj(j) = Xe(j)*0.025 + eps; % eps= Computer epsilon
        Xpup   = dyn_rigidbody(0, Xe+dxj, Ue, par_gen, par_aero, par_prop);
        Xpdw   = dyn_rigidbody(0, Xe-dxj, Ue, par_gen, par_aero, par_prop);
        A(:,j) = (Xpup-Xpdw)/(2*dxj(j));
    end

    % B Matrix
    for j = 1:m
        duj    = zeros(m,1);
        duj(j) = Ue(j)*0.025 + eps; % eps= Computer epsilon
        Xpup   = dyn_rigidbody(0, Xe, Ue+duj, par_gen, par_aero, par_prop);
        Xpdw   = dyn_rigidbody(0, Xe, Ue-duj, par_gen, par_aero, par_prop);
        B(:,j) = (Xpup-Xpdw)/(2*duj(j));
    end

    % C Matrix
    for j = 1:n
        dxj    = zeros(n,1);
        dxj(j) = Xe(j)*0.025 + eps; % eps= Computer epsilon
        Yup   = obs_rigidbody(0, Xe+dxj, Ue);
        Ydw   = obs_rigidbody(0, Xe-dxj, Ue);
        C(:,j) = (Yup-Ydw)/(2*dxj(j));
    end

    % D Matrix
    for j = 1:m
        duj    = zeros(m,1);
        duj(j) = Ue(j)*0.025 + eps; % eps= Computer epsilon
        Yup   = obs_rigidbody(0, Xe, Ue+duj);
        Ydw   = obs_rigidbody(0, Xe, Ue-duj);
        D(:,j) = (Yup-Ydw)/(2*duj(j));
    end
end
```

4.19. modelos/Não Linear/obs_rigidbody.m

Arquivo: modelos/Não Linear/obs_rigidbody.m

Extensão: .m

Conteúdo:

```
% ---------------------------------------------------------
% Function - Observation of Rigid Body Dynamics Outputs
% ---------------------------------------------------------
% Inputs:
%           1- t: Time
%           2- X: State Variables
%               X(1)  = u
%               X(2)  = v
%               X(3)  = w
%               X(4)  = p
%               X(5)  = q
%               X(6)  = r
%               X(7)  = phi
%               X(8)  = theta
%               X(9)  = psi
%               X(10) = xN
%               X(11) = xE
%               X(12) = xD
%           3- U: Commands
%               U(1)  = dt
%               U(2)  = de
%               U(3)  = da
%               U(4)  = dr
% Output:
%           1- Y: Output Vector
%               Y(1)  = VT
%               Y(2)  = alpha
%               Y(3)  = beta
%               Y(4)  = p
%               Y(5)  = q
%               Y(6)  = r
%               X(7)  = phi
%               X(8)  = theta
%               X(9)  = psi
%               X(10) = xN
%               X(11) = xE
%               X(12) = -xD
function Y = obs_rigidbody(t,X,U)
    %% Input Vector Designation
    u     = X(1);
    v     = X(2);
    w     = X(3);
    p     = X(4);
    q     = X(5);
    r     = X(6);
    phi   = X(7);
    theta = X(8);
    psi   = (X(9));
    xN    = X(10);
    xE    = X(11);
    xD    = X(12);

    %% Atmosphere, true airspeed, incidences
    % Aerodynamic velocity
    VT    = sqrt(u^2 + v^2 + w^2);
    % Aerodynamic angles
    alpha = atan(w/u);
    beta  = asin(v/VT);

    %% Output Vector Output
    Y = [VT         % Y(1)
         alpha      % Y(2)
         beta       % Y(3)
         p          % Y(4)
         q          % Y(5)
         r          % Y(6)
         phi        % Y(7)
         theta      % Y(8)
         psi        % Y(9)
         xN         % Y(10)
         xE         % Y(11)
        -xD];       % Y(12)
end
```

4.20. modelos/Não Linear/plot_long.m

Arquivo: modelos/Não Linear/plot_long.m

Extensão: .m

Conteúdo:

```
%% Salvar após simulação para cada caso:

 t_m    =   out.Y.time;
 dt_m   =   out.U.signals.values(:,1) ;
 de_m   =   out.U.signals.values(:,2) ;

 VT_m   =   out.Y.signals.values(:,1) ;
 beta_m =   out.Y.signals.values(:,3) ;

 theta_m  =   out.Y.signals.values(:,8) ;
 q_m    =   out.Y.signals.values(:,5) ;
 xN     =   out.Y.signals.values(:,10);
 xE     =   out.Y.signals.values(:,11);
 xU     =   out.Y.signals.values(:,12); %%valor de -xD

 ay_m   =   out.Y.signals.values(:,15);

figure(1);
plot3(xE, xN, xU, 'b-', 'LineWidth', 1.5);
grid on;
xlabel('Leste (xE) [m]');
ylabel('Norte (xN) [m]');
zlabel('Altitude [m]');
title('Trajetória 3D da Aeronave em Coordenadas Locais');


figure(2);
subplot(2,2,1);
plot(t_m, dt_m, 'b', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\delta_t [rad]');
title('Manobra na Manete'); grid on;

subplot(2,2,2);
plot(t_m, de_m, 'b', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\delta_e [rad]');
title('Manobra no Profundor'); grid on;

subplot(2,2,3);
plot(t_m, rad2deg(q_m), 'r', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('q [°/s]');
title('Taxa de Arfagem'); grid on;

subplot(2,2,4);
plot(t_m, rad2deg(theta_m), 'g', 'LineWidth', 1.2);
xlabel('Tempo [s]'); ylabel('\theta [°]');
title('Ângulo de Arfagem'); grid on;
```

4.21. modelos/Não Linear/propulsion.m

Arquivo: modelos/Não Linear/propulsion.m

Extensão: .m

Conteúdo:

```
% propulsion model
% INPUTS:
% aero_var: structure with following fields
%   -> rho: air density
%   -> V: TAS
%   -> alpha: angle of attack
%   -> beta: sideslip angle
%   -> p: roll rate
%   -> q: pitch rate
%   -> r: yaw rate
%   -> ct: throttle deflection
%   -> ca: aileron deflection
%   -> ce: elevator deflection
%   -> cr: rudder deflection
% par_prop: parameters of propulsion model (generated by DataA300)
% par_aero: parameters of aerodynamic model (generated by DataA300)
function [T]=propulsion(aero_var, par_prop, par_aero)

T = zeros(3,1);

% F_thrust= F_max*d_thrust*(VT_0/VT)^nv*(rho/rho_0)^np
T(1) = par_prop.Fmax*aero_var.ct*(par_aero.Vref/aero_var.V)*(aero_var.rho/1.225)^0.8;

%% please, wait a [moment]
% MT ? [ 0 zf*FT*cos(af)-xf*sin(af) 0]';
```

4.22. modelos/Não Linear/readme.md

Arquivo: modelos/Não Linear/readme.md

Extensão: .md

Conteúdo:

````
# Modelo Não Linear - Piper J-3 Cub 1/6

Modelo dinâmico não linear (6DOF) da aeronave Piper J-3 Cub em escala 1/6, integrado ao Simulink via S-Function.

## Arquivos

| Arquivo | Descrição |
|---------|-----------|
| `sfunction_piper.m` | S-Function Level-1 do Simulink (inicialização, derivadas e saídas) |
| `dyn_rigidbody.m` | Derivadas dos 12 estados (equações de corpo rígido) |
| `obs_rigidbody.m` | Mapeamento estado -> saídas observáveis |
| `aerodynamics.m` / `aerodynamics2.m` | Forças e momentos aerodinâmicos |
| `propulsion.m` | Modelo de propulsão |
| `equilibrium.m` | Condição de equilíbrio (Xe, Ue) |
| `decoupling.m` | Desacoplamento longitudinal / látero-direcional e geração de (A,B,C,D) |
| `lin.m` | Linearização em torno do ponto de equilíbrio |
| `ISA.m` | Atmosfera padrão internacional |
| `modelo.slx` | Modelo Simulink não linear (planta aberta, sem controlador) |
| `gerar_log.m` | Extrai variáveis látero-direcionais de `out.Y` e gera log para OEM |
| `plot_long.m` | Extrai e plota variáveis longitudinais de `out.Y` |
| `executar.m` | Script de inicialização básica (carrega .mat e chama `equilibrium`) |

## S-Function: sfunction_piper

**Arquivo único** usado por todos os modelos Simulink do repositório (modeloNL1.slx, NL_guidance.slx,
q2_trim_modelo24a.slx).

- **12 estados contínuos:** u, v, w, p, q, r, phi, theta, psi, xN, xE, xD
- **4 entradas:** delta_T (throttle), delta_e (profundor), delta_a (aileron), delta_r (leme)
- **21 saídas:**

| Índice | Saída | Descrição |
|--------|-------|-----------|
| 1 | VT | Velocidade total |
| 2 | alpha | Ângulo de ataque |
| 3 | beta | Ângulo de derrapagem |
| 4-6 | p, q, r | Taxas angulares |
| 7-9 | phi, theta, psi | Ângulos de Euler |
| 10-11 | xN, xE | Posição NED (Norte, Leste) |
| 12 | -xD | Altitude (positiva para cima) |
| 13-15 | u, v, w | Velocidades no corpo |
| 16-21 | up, vp, wp, pp, qp, rp | Derivadas (monitoramento) |

- **DirFeedthrough = 0:** Elimina loops algébricos. Os outputs 16-21 (derivadas) usam `u` do passo anterior, o que
é aceitável para monitoramento.

## Ponto de equilíbrio

Definido em `equilibrium.m` (valores obtidos via `trimagem_piper.m` com fmincon):

```
Xe = [15; 0; -1.8343; 0; 0; 0; 0; -0.1217; 0; 0; 0; -100]
Ue = [0.4914; 0.0155; 0; 0]
```

Condição: VT = 15 m/s, theta = -0.1217 rad, altitude = 100 m, voo reto nivelado.

## Passo a passo

1. Na raiz do repositório, rodar `inicializar` (carrega parâmetros, ganhos e equilíbrio).
2. Abrir o modelo Simulink desejado.
3. Rodar a simulação. Resultados ficam em `out`.
4. Para visualização:
   - `gerar_log.m` -- variáveis látero-direcionais e trajetória 3D
   - `plot_long.m` -- variáveis longitudinais
````

4.23. modelos/Não Linear/sfunction_piper.m

Arquivo: modelos/Não Linear/sfunction_piper.m

Extensão: .m

Conteúdo:

```
% ------------------------------------------------
% Function - Piper J Cub 1/6 Equations of Motion
% ------------------------------------------------
% S-Function Level-1 para modelo nao linear 6DOF.
% Usada tanto no modeloNL1.slx (controle) quanto no NL_guidance.slx (guiagem).
%
% Inputs:
%           1- t,x,u,flag,Xe: Model Initilization of s-function with Aircraft States at Equilibrium
%               Xe(1)  = u_e
%               Xe(2)  = v_e
%               Xe(3)  = w_e
%               Xe(4)  = p_e
%               Xe(5)  = q_e
%               Xe(6)  = r_e
%               Xe(7)  = phi_e
%               Xe(8)  = theta_e
%               Xe(9)  = psi_e
%               Xe(10) = xN_e
%               Xe(11) = xE_e
%               Xe(12) = xD_e
% Output:
%           case 0 (Initialization):
%               1- sys,x0,str,ts,simStateCompliance: Model Initialization with Aircraft States at Equilibrium
%           case 1 (Derivatives):
%               1- sys: State Derivatives
%                   sys(1) = up
%                   sys(2) = vp
%                   sys(3) = wp
%                   sys(4) = pp
%                   sys(5) = qp
%                   sys(6) = rp
%                   sys(7) = phip
%                   sys(8) = thetap
%                   sys(9) = psip
%                   sys(10) = xNp
%                   sys(11) = xEp
%                   sys(12) = xDp
%           case 3 (Output):
%               1- sys: Output Vector (21x1)
%                   sys(1)  = VT
%                   sys(2)  = alpha
%                   sys(3)  = beta
%                   sys(4)  = p
%                   sys(5)  = q
%                   sys(6)  = r
%                   sys(7)  = phi
%                   sys(8)  = theta
%                   sys(9)  = psi
%                   sys(10) = xN
%                   sys(11) = xE
%                   sys(12) = -xD (altitude positiva para cima)
%                   sys(13) = u (body velocity)
%                   sys(14) = v (body velocity)
%                   sys(15) = w (body velocity)
%                   sys(16) = up (derivada)
%                   sys(17) = vp (derivada)
%                   sys(18) = wp (derivada)
%                   sys(19) = pp (derivada)
%                   sys(20) = qp (derivada)
%                   sys(21) = rp (derivada)

function [sys,x0,str,ts,simStateCompliance]=sfunction_piper(t,x,u,flag,par_gen,par_aero,par_prop,Xe) % Do not
delete or change t,x,u,flag and sys,x0,str,ts
    switch flag
        %%%%%%%%%%%%%%%%%%
        % Initialization %
        %%%%%%%%%%%%%%%%%%
        case 0
            [sys,x0,str,ts,simStateCompliance] = mdlInitializeSizes(Xe);

        %%%%%%%%%%%%%%%
        % Derivatives %
        %%%%%%%%%%%%%%%
        case 1
            sys = dyn_rigidbody(t,x,u,par_gen,par_aero,par_prop);

        %%%%%%%%%%%%%%%%%%%%%%%
        % Update and Terminate %
```

```
        %%%%%%%%%%%%%%%%%%%%%%%%
        case {2,9}
            sys = []; % do nothing

        %%%%%%%%%%
        % Output %
        %%%%%%%%%%
        case 3
             % Calcular outputs com protecao contra NaN/Inf/complex
             ub = x(1); vb = x(2); wb = x(3);
             VT = sqrt(ub^2 + vb^2 + wb^2);
             if VT < 1e-6, VT = 1e-6; end  % protege divisao por zero
             alpha = atan2(wb, ub);          % atan2 nunca da Inf
             beta  = asin(max(-1, min(1, vb/VT)));  % clamp para [-1,1]

             % Saidas observaveis (12x1)
             y = [VT; alpha; beta; x(4); x(5); x(6); x(7); x(8); x(9); x(10); x(11); -x(12)];

             % Body velocities (3x1)
             bv = [ub; vb; wb];

             % Derivadas para monitoramento (6x1)
             % NOTA: Com DirFeedthrough=0, Simulink passa u do passo anterior.
             % Isso e aceitavel para monitoramento; nao afeta a dinamica.
             Xp = dyn_rigidbody(t,x,u,par_gen,par_aero,par_prop);
             dv = Xp(1:6); dv = dv(:);

             sys = real([y; bv; dv]);   % 21x1

             % Proteger contra NaN/Inf
             if any(~isfinite(sys))
                 %fprintf('SFUNC WARN t=%.6f: NaN/Inf detectado! x=%s u=%s\n', t, mat2str(x',4), mat2str(u',4));
                 sys(~isfinite(sys)) = 0;
             end

        otherwise
            DAStudio.error('Simulink:blocks:unhandledFlag', num2str(flag));
    end
end



% ------------------------------------------------------------------------------------
% Function - Return the sizes, initial conditions, and sample times for the S-function.
% ------------------------------------------------------------------------------------
function [sys,x0,str,ts,simStateCompliance] = mdlInitializeSizes(Xe)
    sizes = simsizes;
    sizes.NumContStates  = 12;
    sizes.NumDiscStates  = 0;
    sizes.NumOutputs     = 21;
    sizes.NumInputs      = 4;
    sizes.DirFeedthrough = 0;  % Outputs 1-15 dependem apenas dos estados (x).
                               % Outputs 16-21 (derivadas) usam u do passo
                               % anterior — aceitavel para monitoramento.
    sizes.NumSampleTimes = 1;

    sys = simsizes(sizes);
    str = [];
    x0  = Xe;
    ts  = [0 0];   % sample time: [period, offset]

    % specify that the simState for this s-function is same as the default
    simStateCompliance = 'DefaultSimState';

end % mdlInitializeSizes
```

4.24. modelos/readme.md

Arquivo: modelos/readme.md

Extensão: .md

Conteúdo:

```
# Modelos - Piper J-3 Cub 1/6

Modelos matemáticos da aeronave utilizados para simulação e projeto de controladores.

## Subdiretórios

| Pasta | Descrição |
|-------|-----------|
| `Linear/` | Matrizes de espaço de estados (A, B, C, D) obtidas por linearização |
| `Não Linear/` | Modelo 6-DOF completo via S-Function do Simulink (21 saídas) |

## Funções compartilhadas

As funções do modelo não linear (`sfunction_piper.m`, `dyn_rigidbody.m`, `aerodynamics.m`, `propulsion.m`,
`equilibrium.m`, etc.) ficam em `Não Linear/` e são referenciadas por outros módulos (trim, controle, guiagem) via
`addpath`.

O script `inicializar.m` (na raiz do repositório) adiciona automaticamente `modelos/Não Linear/` ao path do MATLAB.
```

4.25. navigation/DBN/DBN_solver.m

Arquivo: navigation/DBN/DBN_solver.m

Extensão: .m

Conteúdo:

```
function [euler_out, vel_out, pos_out, acc_n_out, qb_out] = DBN_solver( ...
    acc_b_in, gyro_b_in, reset, sample_valid, t_now, DBN_params)

persistent qb_state
persistent vel_state
persistent pos_state
persistent acc_n_prev
persistent gyro_prev
persistent t_prev
persistent initialized
persistent last_reset_token

if isempty(initialized)
    initialized = false;
end

if ~isfield(DBN_params, 'initialized') || ~DBN_params.initialized
    euler_out = zeros(3,1);
    vel_out   = zeros(3,1);
    pos_out   = zeros(3,1);
    acc_n_out = zeros(3,1);
    qb_out    = [0;0;0;1];
    return;
end

if isempty(last_reset_token)
    last_reset_token = -1;
end

new_params_loaded = false;

if isfield(DBN_params, 'reset_token')
    if DBN_params.reset_token ~= last_reset_token
        new_params_loaded = true;
        last_reset_token = DBN_params.reset_token;
    end
end

g_ned = [0; 0; DBN_params.g0];

%% Inicialização

if reset || ~initialized || new_params_loaded

    euler0 = DBN_params.euler0;
    pos0   = DBN_params.pos0_ned;
    vel0   = DBN_params.vel0_ned;

    phi0   = euler0(1);
    theta0 = euler0(2);
    psi0   = euler0(3);

    Rn2b0 = Rn2b_from_euler(phi0, theta0, psi0);

    b4 = 0.5 * sqrt(1 + Rn2b0(1,1) + Rn2b0(2,2) + Rn2b0(3,3));

    qb_state = zeros(4,1);

    qb_state(1) = (Rn2b0(3,2) - Rn2b0(2,3)) / (4*b4);
    qb_state(2) = (Rn2b0(1,3) - Rn2b0(3,1)) / (4*b4);
    qb_state(3) = (Rn2b0(2,1) - Rn2b0(1,2)) / (4*b4);
    qb_state(4) = b4;

    qb_state = qb_state / norm(qb_state);

    vel_state = vel0;
    pos_state = pos0;

    Rn2b_q = quatb_to_Rn2b(qb_state);
    Rb2n_q = Rn2b_q.';

    % acc_b_in vem do X-Plane já como aceleração translacional,
    % expressa no corpo. Portanto, apenas transforma corpo -> NED.
    % tratar quando usar com sensor real
    acc_n_prev = Rb2n_q * acc_b_in;

    gyro_prev = gyro_b_in;
    t_prev = t_now;
```

```
    initialized = true;

    euler_out = quatb_to_euler_body_to_ned(qb_state);
    vel_out = vel_state;
    pos_out = pos_state;
    acc_n_out = acc_n_prev;
    qb_out = qb_state;

    return;
end

%% Se não houver amostra válida, mantém estados

if sample_valid == 0

    euler_out = quatb_to_euler_body_to_ned(qb_state);
    vel_out = vel_state;
    pos_out = pos_state;
    acc_n_out = acc_n_prev;
    qb_out = qb_state;

    return;
end

%% Tempo

dt = t_now - t_prev;

if dt <= 0
    dt = 0;
end

%% Propagação do quaternion Farrel
% troca de sinal necessária para funcionar no xplane.
if isfield(DBN_params, 'gyro_sign')
    gyro_sign = DBN_params.gyro_sign;
else
    gyro_sign = -1;
end

wx = gyro_sign * gyro_prev(1);
wy = gyro_sign * gyro_prev(2);
wz = gyro_sign * gyro_prev(3);

Omega = [   0,   wz,  -wy,  wx;
          -wz,    0,   wx,  wy;
           wy,  -wx,    0,  wz;
          -wx,  -wy,  -wz,   0 ];

qbdot = 0.5 * Omega * qb_state;

qb_state = qb_state + qbdot * dt;

qb_state = qb_state / norm(qb_state);

%% Integração

vel_old = vel_state;

vel_state = vel_state + acc_n_prev * dt;

pos_state = pos_state + vel_old * dt;

%% Nova aceleração em NED

Rn2b_q = quatb_to_Rn2b(qb_state);
Rb2n_q = Rn2b_q.';

% acc_b_in já é aceleração translacional no corpo.
% Não subtrair gravidade novamente. tratar quando usar com sensor real
acc_n_current = Rb2n_q * acc_b_in;

%% Atualização das memórias

acc_n_prev = acc_n_current;
gyro_prev = gyro_b_in;
t_prev = t_now;

%% Saídas

euler_out = quatb_to_euler_body_to_ned(qb_state);
vel_out = vel_state;
pos_out = pos_state;
acc_n_out = acc_n_current;
qb_out = qb_state;
```

```
end

%% ========================================================================
% Euler -> Rn2b
% ========================================================================

function R = Rn2b_from_euler(phi, theta, psi)

R = [ ...
    cos(psi)*cos(theta), ...
    sin(psi)*cos(theta), ...
   -sin(theta);

    cos(psi)*sin(theta)*sin(phi) - sin(psi)*cos(phi), ...
    sin(psi)*sin(theta)*sin(phi) + cos(psi)*cos(phi), ...
    cos(theta)*sin(phi);

    cos(psi)*sin(theta)*cos(phi) + sin(psi)*sin(phi), ...
    sin(psi)*sin(theta)*cos(phi) - cos(psi)*sin(phi), ...
    cos(theta)*cos(phi) ...
];

end

%% ========================================================================
% Matriz usada na transformação da aceleração
% ========================================================================

function Rn2b = quatb_to_Rn2b(b)

b1 = b(1);
b2 = b(2);
b3 = b(3);
b4 = b(4);

Rn2b = [ ...
    b1^2 + b4^2 - b2^2 - b3^2, ...
    2*(b1*b2 - b3*b4), ...
    2*(b1*b3 + b2*b4);

    2*(b1*b2 + b3*b4), ...
    b2^2 + b4^2 - b1^2 - b3^2, ...
    2*(b2*b3 - b1*b4);

    2*(b1*b3 - b2*b4), ...
    2*(b2*b3 + b1*b4), ...
    b3^2 + b4^2 - b1^2 - b2^2 ...
];

end

%% ========================================================================
% Quaternion Farrel -> Euler corpo em relação ao NED
% ========================================================================

function eul = quatb_to_euler_body_to_ned(b)

x = b(1);
y = b(2);
z = b(3);
w = b(4);

norm2 = x^2 + y^2 + z^2 + w^2;

b_inv = [-x; -y; -z; w] / norm2;

b1 = b_inv(1);
b2 = b_inv(2);
b3 = b_inv(3);
b4 = b_inv(4);

val = -2*(b2*b4 + b1*b3);

if val > 1
    val = 1;
elseif val < -1
    val = -1;
end

theta = asin(val);

phi = atan2(2*(b2*b3 - b1*b4), ...
            1 - 2*(b1^2 + b2^2));

psi = atan2(2*(b1*b2 - b3*b4), ...
            1 - 2*(b2^2 + b3^2));
```

```
eul = [phi; theta; psi];

end
```

4.26. navigation/DBN/init_DBN.m

Arquivo: navigation/DBN/init_DBN.m

Extensão: .m

Conteúdo:

```
function DBN_params = init_DBN(xplane_init)
% init_DBN
%
% Inicializa os parâmetros do DBN a partir da condição inicial efetivamente
% aplicada no X-Plane.
% Sinal dos giros para propagação do quaternion Farrel que parametriza Rn2b.
% Para os sinais p,q,r vindos do X-Plane, a propagação de Rn2b requer sinal
% trocado
% Entrada:
%   xplane_init.euler0    = [phi0; theta0; psi0] [rad]
%   xplane_init.pos0_ned  = [N0; E0; D0] [m]
%   xplane_init.vel0_ned  = [vN0; vE0; vD0] [m/s]
%
% Saída:
%   DBN_params no workspace base.

DBN_params = struct();

DBN_params.initialized = true;

DBN_params.euler0 = xplane_init.euler0(:);
DBN_params.pos0_ned = xplane_init.pos0_ned(:);
DBN_params.vel0_ned = xplane_init.vel0_ned(:);

DBN_params.g0 = 9.81;

% Altitude física positiva para cima.
% Em NED: D = -altitude.
DBN_params.altitude0 = -DBN_params.pos0_ned(3);

% Notação usada no DBN:
% Farrel: b = [b1; b2; b3; b4], com b4 escalar.
DBN_params.quat_notation = 'farrel_b';

% Convenção validada nos testes:
% acc_n = R_acc * acc_b - g_ned
DBN_params.subtract_gravity = true;

% Token usado para identificar nova inicialização.
% Útil caso múltiplas simulações sejam executadas na mesma sessão.
DBN_params.reset_token = now;

assignin('base', 'DBN_params', DBN_params);

% Limpa persistentes do solver entre simulações.
clear DBN_solver;

disp('--- DBN_params inicializado a partir do estado inicial do X-Plane ---');
fprintf('Euler0 [deg] = [%.3f %.3f %.3f]\n', rad2deg(DBN_params.euler0));
fprintf('Pos0 NED [m] = [%.3f %.3f %.3f]\n', DBN_params.pos0_ned);
fprintf('Vel0 NED [m/s] = [%.3f %.3f %.3f]\n', DBN_params.vel0_ned);
fprintf('Altitude0 [m] = %.3f\n', DBN_params.altitude0);

end
```

4.27. navigation/FK/EKF_DI_EM_solver.m

Arquivo: navigation/FK/EKF_DI_EM_solver.m

Extensão: .m

Conteúdo:

```
function [euler_out, vel_out, pos_out, acc_n_out, xhat_out] = EKF_DI_EM_solver( ...
    acc_b_in, gyro_b_in, pos_meas_in, vel_meas_in, eul_meas_in, reset, sample_valid, t_now, EKF_DI_params)

persistent x_hat_DEM
persistent P_DEM
persistent t_prev_DEM
persistent initialized_DEM
persistent last_reset_token_DEM
persistent next_gps_time_DEM
persistent next_mag_time_DEM
persistent h0_meas_DEM

nx = 21;
I3 = eye(3);
Z3 = zeros(3);

if isempty(initialized_DEM)
    initialized_DEM = false;
end

if ~isfield(EKF_DI_params, 'initialized') || ~EKF_DI_params.initialized
    euler_out = zeros(3,1);
    vel_out   = zeros(3,1);
    pos_out   = zeros(3,1);
    acc_n_out = zeros(3,1);
    xhat_out  = zeros(nx,1);
    return;
end

if isempty(last_reset_token_DEM)
    last_reset_token_DEM = -1;
end

new_params_loaded = false;

if isfield(EKF_DI_params, 'reset_token')
    if EKF_DI_params.reset_token ~= last_reset_token_DEM
        new_params_loaded = true;
        last_reset_token_DEM = EKF_DI_params.reset_token;
    end
end

%% Inicialização

if reset || ~initialized_DEM || new_params_loaded

    x_hat_DEM = EKF_DI_params.x0;
    P_DEM = EKF_DI_params.P0;

    t_prev_DEM = t_now;
    initialized_DEM = true;

    next_gps_time_DEM = t_now + EKF_DI_params.gps_period;

    if isfield(EKF_DI_params, 'mag_period')
        next_mag_time_DEM = t_now + EKF_DI_params.mag_period;
    else
        next_mag_time_DEM = t_now + 0.02;
    end

    % pos_meas_in = [N; E; h]
    h0_meas_DEM = pos_meas_in(3);

    euler_out = x_hat_DEM(7:9);
    vel_out   = x_hat_DEM(4:6);
    pos_out = ekf_pos_output(x_hat_DEM(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_DEM;

    return;
end

%% Se não houver amostra válida, mantém estados

if sample_valid == 0

    euler_out = x_hat_DEM(7:9);
    vel_out   = x_hat_DEM(4:6);
```

```
    pos_out = ekf_pos_output(x_hat_DEM(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_DEM;

    return;
end

%% Tempo

dt = t_now - t_prev_DEM;

if dt <= 0
    dt = 0;
end

%% Parâmetros

Qw = EKF_DI_params.Qw;
R_pos = EKF_DI_params.R_pos;
R_vel = EKF_DI_params.R_vel;
lambda_y = EKF_DI_params.lambda_y;

if isfield(EKF_DI_params, 'R_yaw')
    R_yaw = EKF_DI_params.R_yaw;
else
    R_yaw = deg2rad(2.0)^2;
end

beta_y = exp(-lambda_y*dt);

%% Separar estados

p_hat     = x_hat_DEM(1:3);
v_hat     = x_hat_DEM(4:6);
eta_hat   = x_hat_DEM(7:9);
ba_hat    = x_hat_DEM(10:12);
bg_hat    = x_hat_DEM(13:15);
by_hat    = x_hat_DEM(16:18);
alpha_hat = x_hat_DEM(19:21);

%% Corrigir IMU

den = 1 - alpha_hat;

for k = 1:3
    if abs(den(k)) < 1e-6
        den(k) = sign(den(k))*1e-6;
        if den(k) == 0
            den(k) = 1e-6;
        end
    end
end

M_alpha = diag(1./den);

acc_corr_b = M_alpha*(acc_b_in + ba_hat);
gyro_corr_b = gyro_b_in + bg_hat;

%% Propagação nominal

Rbn = Rb2n_321(eta_hat);
Teta = T_321(eta_hat);

% O acc_b_in atual é aceleração translacional expressa no corpo.
a_hat_n = Rbn * acc_corr_b;

x_pred = x_hat_DEM;

x_pred(1:3)   = p_hat + v_hat*dt + 0.5*a_hat_n*dt^2;
x_pred(4:6)   = v_hat + a_hat_n*dt;
x_pred(7:9)   = eta_hat + Teta*gyro_corr_b*dt;
x_pred(10:12) = ba_hat;
x_pred(13:15) = bg_hat;
x_pred(16:18) = beta_y*by_hat;
x_pred(19:21) = alpha_hat;

x_pred(9) = atan2(sin(x_pred(9)), cos(x_pred(9)));

%% Linearização

F = zeros(nx,nx);

F(1:3,4:6) = I3;

[Rphi,Rtheta,Rpsi] = dRb2n_321(eta_hat);
Fv_eta = [Rphi*acc_corr_b, Rtheta*acc_corr_b, Rpsi*acc_corr_b];
```

```
Fv_ba = Rbn*M_alpha;

Fv_alpha = Rbn*diag((acc_b_in + ba_hat)./((1 - alpha_hat).^2));

[Tphi,Ttheta,Tpsi] = dT_321(eta_hat);
Feta_eta = [Tphi*gyro_corr_b, Ttheta*gyro_corr_b, Tpsi*gyro_corr_b];
Feta_bg = Teta;

F(4:6,7:9)     = Fv_eta;
F(4:6,10:12)   = Fv_ba;
F(4:6,19:21)   = Fv_alpha;

F(7:9,7:9)     = Feta_eta;
F(7:9,13:15)   = Feta_bg;

F(16:18,16:18) = -lambda_y*I3;

G = zeros(nx,15);

G(4:6,1:3)     = Rbn*M_alpha;
G(7:9,4:6)     = Teta;
G(10:12,7:9)   = I3;
G(13:15,10:12) = I3;
G(16:18,13:15) = I3;

Phi = eye(nx) + F*dt;
Qd = G*Qw*G'*dt;

P_DEM = Phi*P_DEM*Phi' + Qd;
P_DEM = 0.5*(P_DEM + P_DEM');

x_hat_DEM = x_pred;

%% Consideração ou não de medida de correção
correction_allowed = true;

range_no_corr = EKF_DI_params.range_time_without_correction;

for kk = 1:size(range_no_corr,1)
    ti = range_no_corr(kk,1);
    tf = range_no_corr(kk,2);

    if ~(ti == 0 && tf == 0)
        if t_now >= ti && t_now < tf
            correction_allowed = false;
        end
    end
end

%% Atualização GPS/Pseudo-GPS a 1 Hz

do_gps_update = false;

if t_now >= next_gps_time_DEM
    do_gps_update = true;

    while next_gps_time_DEM <= t_now
        next_gps_time_DEM = next_gps_time_DEM + EKF_DI_params.gps_period;
    end
end

if do_gps_update && correction_allowed

    %% Medição de posição
    % pos_meas_in = [N; E; h]
    % Converter h para D relativo ao D0 do filtro.
    N_meas = pos_meas_in(1);
    E_meas = pos_meas_in(2);
    h_meas = pos_meas_in(3);

    D0 = EKF_DI_params.pos0_ned(3);
    D_meas = D0 - (h_meas - h0_meas_DEM);

    z_pos = [N_meas; E_meas; D_meas];

    H_pos = [I3 Z3 Z3 Z3 Z3 I3 Z3];

    y_hat_pos = x_hat_DEM(1:3) + x_hat_DEM(16:18);

    innov_pos = z_pos - y_hat_pos;

    S_pos = H_pos*P_DEM*H_pos' + R_pos;
    K_pos = P_DEM*H_pos'/S_pos;

    x_hat_DEM = x_hat_DEM + K_pos*innov_pos;
```

```
    P_DEM = (eye(nx) - K_pos*H_pos)*P_DEM*(eye(nx) - K_pos*H_pos)' + K_pos*R_pos*K_pos';
    P_DEM = 0.5*(P_DEM + P_DEM');

    %% Medição de velocidade
    z_vel = vel_meas_in;

    H_vel = [Z3 I3 Z3 Z3 Z3 Z3 Z3];

    innov_vel = z_vel - x_hat_DEM(4:6);

    S_vel = H_vel*P_DEM*H_vel' + R_vel;
    K_vel = P_DEM*H_vel'/S_vel;

    x_hat_DEM = x_hat_DEM + K_vel*innov_vel;

    P_DEM = (eye(nx) - K_vel*H_vel)*P_DEM*(eye(nx) - K_vel*H_vel)' + K_vel*R_vel*K_vel';
    P_DEM = 0.5*(P_DEM + P_DEM');

end

%% Atualização de yaw pelo magnetômetro

do_mag_update = false;

if t_now >= next_mag_time_DEM
    do_mag_update = true;

    if isfield(EKF_DI_params, 'mag_period')
        mag_period = EKF_DI_params.mag_period;
    else
        mag_period = 0.02;
    end

    while next_mag_time_DEM <= t_now
        next_mag_time_DEM = next_mag_time_DEM + mag_period;
    end
end

if do_mag_update

    % eul_meas_in = [roll; pitch; yaw]
    % Apenas yaw é usado na correção.
    yaw_meas = eul_meas_in(3);

    if isfinite(yaw_meas)

        H_yaw = zeros(1,nx);
        H_yaw(9) = 1;

        yaw_hat = x_hat_DEM(9);

        innov_yaw = wrapToPi_local(yaw_meas - yaw_hat);

        S_yaw = H_yaw*P_DEM*H_yaw' + R_yaw;
        K_yaw = P_DEM*H_yaw'/S_yaw;

        x_hat_DEM = x_hat_DEM + K_yaw*innov_yaw;

        x_hat_DEM(7:9) = wrapToPi_local(x_hat_DEM(7:9));

        P_DEM = (eye(nx) - K_yaw*H_yaw)*P_DEM*(eye(nx) - K_yaw*H_yaw)' + K_yaw*R_yaw*K_yaw';
        P_DEM = 0.5*(P_DEM + P_DEM');

    end
end

%% Atualização do tempo

t_prev_DEM = t_now;

%% Saídas

euler_out = x_hat_DEM(7:9);
vel_out   = x_hat_DEM(4:6);
pos_out   = ekf_pos_output(x_hat_DEM(1:3));
acc_n_out = a_hat_n;
xhat_out  = x_hat_DEM;

end

%-=-=-=
```

```
function R = Rb2n_321(eta)

phi = eta(1);
theta = eta(2);
psi = eta(3);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);
sth = sin(theta);
cpsi = cos(psi);
spsi = sin(psi);

R = [ ...
    cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
    cth*spsi,  sphi*sth*spsi + cphi*cpsi,  cphi*sth*spsi - sphi*cpsi;
   -sth,       sphi*cth,                    cphi*cth ...
];

end

function T = T_321(eta)

phi = eta(1);
theta = eta(2);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    cth = sign(cth)*1e-6;
    if cth == 0
        cth = 1e-6;
    end
end

T = [ ...
    1, sphi*tan(theta), cphi*tan(theta);
    0, cphi,           -sphi;
    0, sphi/cth,        cphi/cth ...
];

end

function [Rphi,Rtheta,Rpsi] = dRb2n_321(eta)

phi = eta(1);
theta = eta(2);
psi = eta(3);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);
sth = sin(theta);
cpsi = cos(psi);
spsi = sin(psi);

Rphi = [ ...
    0,  cphi*sth*cpsi + sphi*spsi, -sphi*sth*cpsi + cphi*spsi;
    0,  cphi*sth*spsi - sphi*cpsi, -sphi*sth*spsi - cphi*cpsi;
    0,  cphi*cth,                  -sphi*cth ...
];

Rtheta = [ ...
   -sth*cpsi,  sphi*cth*cpsi,  cphi*cth*cpsi;
   -sth*spsi,  sphi*cth*spsi,  cphi*cth*spsi;
   -cth,      -sphi*sth,      -cphi*sth ...
];

Rpsi = [ ...
   -cth*spsi, -sphi*sth*spsi - cphi*cpsi, -cphi*sth*spsi + sphi*cpsi;
    cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
    0,         0,                          0 ...
];

end

function [Tphi,Ttheta,Tpsi] = dT_321(eta)

phi = eta(1);
theta = eta(2);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);
```

```
if abs(cth) < 1e-6
    cth = sign(cth)*1e-6;
    if cth == 0
        cth = 1e-6;
    end
end

sec_th = 1/cth;
tan_th = tan(theta);

Tphi = [ ...
    0,  cphi*tan_th,        -sphi*tan_th;
    0, -sphi,               -cphi;
    0,  cphi*sec_th,        -sphi*sec_th ...
];

Ttheta = [ ...
    0, sphi*(sec_th^2),          cphi*(sec_th^2);
    0, 0,                        0;
    0, sphi*sec_th*tan_th,       cphi*sec_th*tan_th ...
];

Tpsi = zeros(3);

end

function pos_out = ekf_pos_output(pos_ned)

N = pos_ned(1);
E = pos_ned(2);
D = pos_ned(3);

altitude = -D;

pos_out = [N; E; altitude];

end

function ang = wrapToPi_local(ang)

ang = mod(ang + pi, 2*pi) - pi;

end
```

4.28. navigation/FK/EKF_DI_solver.m

Arquivo: navigation/FK/EKF_DI_solver.m

Extensão: .m

Conteúdo:

```
function [euler_out, vel_out, pos_out, acc_n_out, xhat_out] = EKF_DI_solver( ...
    acc_b_in, gyro_b_in, pos_meas_in, vel_meas_in, eul_meas_in, ...
    reset, sample_valid, t_now, EKF_DI_params)

persistent x_hat_D
persistent P_D
persistent t_prev_D
persistent initialized_D
persistent last_reset_token_D
persistent next_gps_time_D
persistent next_mag_time_D
persistent h0_meas_D

nx = 21;
I3 = eye(3);
Z3 = zeros(3);

if isempty(initialized_D)
    initialized_D = false;
end

if ~isfield(EKF_DI_params, 'initialized') || ~EKF_DI_params.initialized
    euler_out = zeros(3,1);
    vel_out   = zeros(3,1);
    pos_out   = zeros(3,1);
    acc_n_out = zeros(3,1);
    xhat_out  = zeros(nx,1);
    return;
end

if isempty(last_reset_token_D)
    last_reset_token_D = -1;
end

new_params_loaded = false;

if isfield(EKF_DI_params, 'reset_token')
    if EKF_DI_params.reset_token ~= last_reset_token_D
        new_params_loaded = true;
        last_reset_token_D = EKF_DI_params.reset_token;
    end
end

%% Inicialização

if reset || ~initialized_D || new_params_loaded

    x_hat_D = EKF_DI_params.x0;
    P_D = EKF_DI_params.P0;

    t_prev_D = t_now;
    initialized_D = true;

    next_gps_time_D = t_now + EKF_DI_params.gps_period;
    % magnetometro - yaw
    if isfield(EKF_DI_params, 'mag_period')
        next_mag_time_D = t_now + EKF_DI_params.mag_period;
    else
        next_mag_time_D = t_now + 0.02;
    end

    % pos_meas_in = [N; E; h]
    h0_meas_D = pos_meas_in(3);

    euler_out = x_hat_D(7:9);
    vel_out   = x_hat_D(4:6);
    pos_out = ekf_pos_output(x_hat_D(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_D;

    return;
end

%% Se não houver amostra válida, mantém estados

if sample_valid == 0

    euler_out = x_hat_D(7:9);
```

```
    vel_out   = x_hat_D(4:6);
    pos_out = ekf_pos_output(x_hat_D(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_D;

    return;
end

%% Tempo

dt = t_now - t_prev_D;

if dt <= 0
    dt = 0;
end

%% Parâmetros

Qw = EKF_DI_params.Qw;
R_pos = EKF_DI_params.R_pos;
R_vel = EKF_DI_params.R_vel;
lambda_y = EKF_DI_params.lambda_y;
% magnetometro - yaw
if isfield(EKF_DI_params, 'R_yaw')
    R_yaw = EKF_DI_params.R_yaw;
else
    R_yaw = deg2rad(2.0)^2;
end
beta_y = exp(-lambda_y*dt);

%% Separar estados

p_hat     = x_hat_D(1:3);
v_hat     = x_hat_D(4:6);
eta_hat   = x_hat_D(7:9);
ba_hat    = x_hat_D(10:12);
bg_hat    = x_hat_D(13:15);
by_hat    = x_hat_D(16:18);
alpha_hat = x_hat_D(19:21);

%% Corrigir IMU

den = 1 - alpha_hat;

for k = 1:3
    if abs(den(k)) < 1e-6
        den(k) = sign(den(k))*1e-6;
        if den(k) == 0
            den(k) = 1e-6;
        end
    end
end

M_alpha = diag(1./den);

acc_corr_b = M_alpha*(acc_b_in + ba_hat);
gyro_corr_b = gyro_b_in + bg_hat;

%% Propagação nominal

Rbn = Rb2n_321(eta_hat);
Teta = T_321(eta_hat);

% O acc_b_in atual é aceleração translacional expressa no corpo.
a_hat_n = Rbn * acc_corr_b;

x_pred = x_hat_D;

x_pred(1:3)   = p_hat + v_hat*dt + 0.5*a_hat_n*dt^2;
x_pred(4:6)   = v_hat + a_hat_n*dt;
x_pred(7:9)   = eta_hat + Teta*gyro_corr_b*dt;
x_pred(10:12) = ba_hat;
x_pred(13:15) = bg_hat;
x_pred(16:18) = beta_y*by_hat;
x_pred(19:21) = alpha_hat;

x_pred(9) = atan2(sin(x_pred(9)), cos(x_pred(9)));

%% Linearização

F = zeros(nx,nx);

F(1:3,4:6) = I3;

[Rphi,Rtheta,Rpsi] = dRb2n_321(eta_hat);
Fv_eta = [Rphi*acc_corr_b, Rtheta*acc_corr_b, Rpsi*acc_corr_b];
```

```
Fv_ba = Rbn*M_alpha;

Fv_alpha = Rbn*diag((acc_b_in + ba_hat)./((1 - alpha_hat).^2));

[Tphi,Ttheta,Tpsi] = dT_321(eta_hat);
Feta_eta = [Tphi*gyro_corr_b, Ttheta*gyro_corr_b, Tpsi*gyro_corr_b];
Feta_bg = Teta;

F(4:6,7:9)     = Fv_eta;
F(4:6,10:12)   = Fv_ba;
F(4:6,19:21)   = Fv_alpha;

F(7:9,7:9)     = Feta_eta;
F(7:9,13:15)   = Feta_bg;

F(16:18,16:18) = -lambda_y*I3;

G = zeros(nx,15);

G(4:6,1:3)     = Rbn*M_alpha;
G(7:9,4:6)     = Teta;
G(10:12,7:9)   = I3;
G(13:15,10:12) = I3;
G(16:18,13:15) = I3;

Phi = eye(nx) + F*dt;
Qd = G*Qw*G'*dt;

P_D = Phi*P_D*Phi' + Qd;
P_D = 0.5*(P_D + P_D');

x_hat_D = x_pred;

%% Consideração ou não de medida de correção
correction_allowed = true;

range_no_corr = EKF_DI_params.range_time_without_correction;

for kk = 1:size(range_no_corr,1)
    ti = range_no_corr(kk,1);
    tf = range_no_corr(kk,2);

    if ~(ti == 0 && tf == 0)
        if t_now >= ti && t_now < tf
            correction_allowed = false;
        end
    end
end

%% Atualização GPS/Pseudo-GPS a 1 Hz

do_gps_update = false;

if t_now >= next_gps_time_D
    do_gps_update = true;

    while next_gps_time_D <= t_now
        next_gps_time_D = next_gps_time_D + EKF_DI_params.gps_period;
    end
end

if do_gps_update && correction_allowed

    %% Medição de posição
    % pos_meas_in = [N; E; h]
    % Converter h para D relativo ao D0 do filtro.
    N_meas = pos_meas_in(1);
    E_meas = pos_meas_in(2);
    h_meas = pos_meas_in(3);

    D0 = EKF_DI_params.pos0_ned(3);
    D_meas = D0 - (h_meas - h0_meas_D);

    z_pos = [N_meas; E_meas; D_meas];

    H_pos = [I3 Z3 Z3 Z3 Z3 I3 Z3];

    y_hat_pos = x_hat_D(1:3) + x_hat_D(16:18);

    innov_pos = z_pos - y_hat_pos;

    S_pos = H_pos*P_D*H_pos' + R_pos;
    K_pos = P_D*H_pos'/S_pos;

    x_hat_D = x_hat_D + K_pos*innov_pos;
```

```
    P_D = (eye(nx) - K_pos*H_pos)*P_D*(eye(nx) - K_pos*H_pos)' + K_pos*R_pos*K_pos';
    P_D = 0.5*(P_D + P_D');

    %% Medição de velocidade
    z_vel = vel_meas_in;

    H_vel = [Z3 I3 Z3 Z3 Z3 Z3 Z3];

    innov_vel = z_vel - x_hat_D(4:6);

    S_vel = H_vel*P_D*H_vel' + R_vel;
    K_vel = P_D*H_vel'/S_vel;

    x_hat_D = x_hat_D + K_vel*innov_vel;

    P_D = (eye(nx) - K_vel*H_vel)*P_D*(eye(nx) - K_vel*H_vel)' + K_vel*R_vel*K_vel';
    P_D = 0.5*(P_D + P_D');

end

%% Atualização de yaw pelo magnetômetro

do_mag_update = false;

if t_now >= next_mag_time_D
    do_mag_update = true;

    if isfield(EKF_DI_params, 'mag_period')
        mag_period = EKF_DI_params.mag_period;
    else
        mag_period = 0.02;
    end

    while next_mag_time_D <= t_now
        next_mag_time_D = next_mag_time_D + mag_period;
    end
end

if do_mag_update

    yaw_meas = eul_meas_in(3);

    if isfinite(yaw_meas)

        H_yaw = zeros(1,nx);
        H_yaw(9) = 1;

        yaw_hat = x_hat_D(9);

        innov_yaw = wrapToPi_local(yaw_meas - yaw_hat);

        S_yaw = H_yaw*P_D*H_yaw' + R_yaw;
        K_yaw = P_D*H_yaw'/S_yaw;

        x_hat_D = x_hat_D + K_yaw*innov_yaw;

        x_hat_D(7:9) = wrapToPi_local(x_hat_D(7:9));

        P_D = (eye(nx) - K_yaw*H_yaw)*P_D*(eye(nx) - K_yaw*H_yaw)' + K_yaw*R_yaw*K_yaw';
        P_D = 0.5*(P_D + P_D');

    end
end

%% Atualização do tempo

t_prev_D = t_now;

%% Saídas

euler_out = x_hat_D(7:9);
vel_out   = x_hat_D(4:6);
pos_out   = ekf_pos_output(x_hat_D(1:3));
acc_n_out = a_hat_n;
xhat_out  = x_hat_D;

end

%-=-=-=
```

```
theta = eta(2);
psi = eta(3);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);
sth = sin(theta);
cpsi = cos(psi);
spsi = sin(psi);

R = [ ...
    cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
    cth*spsi,  sphi*sth*spsi + cphi*cpsi,  cphi*sth*spsi - sphi*cpsi;
   -sth,       sphi*cth,                   cphi*cth ...
];

end

function T = T_321(eta)

phi = eta(1);
theta = eta(2);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    cth = sign(cth)*1e-6;
    if cth == 0
        cth = 1e-6;
    end
end

T = [ ...
    1, sphi*tan(theta), cphi*tan(theta);
    0, cphi,           -sphi;
    0, sphi/cth,        cphi/cth ...
];

end

function [Rphi,Rtheta,Rpsi] = dRb2n_321(eta)

phi = eta(1);
theta = eta(2);
psi = eta(3);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);
sth = sin(theta);
cpsi = cos(psi);
spsi = sin(psi);

Rphi = [ ...
    0,  cphi*sth*cpsi + sphi*spsi, -sphi*sth*cpsi + cphi*spsi;
    0,  cphi*sth*spsi - sphi*cpsi, -sphi*sth*spsi - cphi*cpsi;
    0,  cphi*cth,                  -sphi*cth ...
];

Rtheta = [ ...
   -sth*cpsi,  sphi*cth*cpsi,  cphi*cth*cpsi;
   -sth*spsi,  sphi*cth*spsi,  cphi*cth*spsi;
   -cth,      -sphi*sth,      -cphi*sth ...
];

Rpsi = [ ...
   -cth*spsi, -sphi*sth*spsi - cphi*cpsi, -cphi*sth*spsi + sphi*cpsi;
    cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
    0,         0,                          0 ...
];

end

function [Tphi,Ttheta,Tpsi] = dT_321(eta)

phi = eta(1);
theta = eta(2);

cphi = cos(phi);
sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    cth = sign(cth)*1e-6;
```

```
    if cth == 0
        cth = 1e-6;
    end
end

sec_th = 1/cth;
tan_th = tan(theta);

Tphi = [ ...
    0,  cphi*tan_th,        -sphi*tan_th;
    0, -sphi,               -cphi;
    0,  cphi*sec_th,        -sphi*sec_th ...
];

Ttheta = [ ...
    0, sphi*(sec_th^2),          cphi*(sec_th^2);
    0, 0,                        0;
    0, sphi*sec_th*tan_th,       cphi*sec_th*tan_th ...
];

Tpsi = zeros(3);

end
function pos_out = ekf_pos_output(pos_ned)

N = pos_ned(1);
E = pos_ned(2);
D = pos_ned(3);

altitude = -D;

pos_out = [N; E; altitude];

end
% para o magnetometro
function ang = wrapToPi_local(ang)

ang = mod(ang + pi, 2*pi) - pi;

end
```

4.29. navigation/FK/EKF_INDI_EM_solver.m

Arquivo: navigation/FK/EKF_INDI_EM_solver.m

Extensão: .m

Conteúdo:

```
function [euler_out, vel_out, pos_out, acc_n_out, xhat_out] = EKF_INDI_EM_solver( ...
    acc_b_in, gyro_b_in, pos_meas_in, vel_meas_in, eul_meas_in, reset, sample_valid, t_now, EKF_INDI_params)
% EKF_INDI_EM_solver
%
% Adaptacao para Simulink do EKF indireto 3D isolado.
%
% Estado mantido:
%   x_hat = [p^n; v^n; eta; ba; bg; by; alpha_a]
%
% Entradas:
%   acc_b_in     = aceleracao no corpo [m/s^2]
%   gyro_b_in    = [p; q; r] [rad/s]
%   pos_meas_in  = [N; E; h] ou [N; E; D], conforme params.pos_meas_mode
%   vel_meas_in  = [vN; vE; vD] [m/s]
%   eul_meas_in  = [phi; theta; psi] [rad], usa apenas yaw/psi
%   reset
%   sample_valid
%   t_now
%   EKF_INDI_params
%
% Saidas:
%   euler_out = [phi; theta; psi]
%   vel_out   = [vN; vE; vD]
%   pos_out   = [N; E; altitude]
%   acc_n_out = aceleracao estimada em NED
%   xhat_out  = estado completo 21x1

persistent x_hat_IEM
persistent P_IEM
persistent t_prev_IEM
persistent initialized_IEM
persistent last_reset_token_IEM
persistent next_gps_time_IEM
persistent next_mag_time_IEM
persistent h0_meas_IEM

nx = 21;
I3 = eye(3);
Z3 = zeros(3);

euler_out = zeros(3,1);
vel_out   = zeros(3,1);
pos_out   = zeros(3,1);
acc_n_out = zeros(3,1);
xhat_out  = zeros(nx,1);

if isempty(initialized_IEM)
    initialized_IEM = false;
end

if ~isfield(EKF_INDI_params, 'initialized') || ~EKF_INDI_params.initialized
    return;
end

if isempty(last_reset_token_IEM)
    last_reset_token_IEM = -1;
end

new_params_loaded = false;
if isfield(EKF_INDI_params, 'reset_token')
    if EKF_INDI_params.reset_token ~= last_reset_token_IEM
        new_params_loaded = true;
        last_reset_token_IEM = EKF_INDI_params.reset_token;
    end
end

%% Inicializacao
if reset || ~initialized_IEM || new_params_loaded

    x_hat_IEM = EKF_INDI_params.x0;
    P_IEM = EKF_INDI_params.P0;

    t_prev_IEM = t_now;
    initialized_IEM = true;

    next_gps_time_IEM = t_now + EKF_INDI_params.gps_period;

    if isfield(EKF_INDI_params, 'mag_period')
```

```
        next_mag_time_IEM = t_now + EKF_INDI_params.mag_period;
    else
        next_mag_time_IEM = t_now + 0.02;
    end

    % pos_meas_in tipicamente vem como [N; E; h] do X-Plane.
    h0_meas_IEM = pos_meas_in(3);

    euler_out = x_hat_IEM(7:9);
    vel_out   = x_hat_IEM(4:6);
    pos_out   = ekf_pos_output(x_hat_IEM(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_IEM;

    return;
end

%% Sem amostra valida: mantem estados
if sample_valid == 0
    euler_out = x_hat_IEM(7:9);
    vel_out   = x_hat_IEM(4:6);
    pos_out   = ekf_pos_output(x_hat_IEM(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_IEM;
    return;
end

%% Tempo
dt = t_now - t_prev_IEM;
if dt <= 0
    dt = 0;
end

%% Parametros
Qw = EKF_INDI_params.Qw;
R_pos = EKF_INDI_params.R_pos;
R_vel = EKF_INDI_params.R_vel;
lambda_y = EKF_INDI_params.lambda_y;
beta_y = exp(-lambda_y*dt);

if isfield(EKF_INDI_params, 'R_yaw')
    R_yaw = EKF_INDI_params.R_yaw;
else
    R_yaw = deg2rad(5.0)^2;
end

% Piso de seguranca para evitar correcao agressiva demais
R_yaw = max(R_yaw, deg2rad(3.0)^2);

if isfield(EKF_INDI_params, 'yaw_gate_rad')
    yaw_gate_rad = EKF_INDI_params.yaw_gate_rad;
else
    yaw_gate_rad = deg2rad(30.0);
end

if isfield(EKF_INDI_params, 'acc_input_is_translational')
    acc_input_is_translational = EKF_INDI_params.acc_input_is_translational;
else
    acc_input_is_translational = true;
end

gn = [0; 0; EKF_INDI_params.g0];

%% Separar estados
p_hat     = x_hat_IEM(1:3);
v_hat     = x_hat_IEM(4:6);
eta_hat   = x_hat_IEM(7:9);
ba_hat    = x_hat_IEM(10:12);
bg_hat    = x_hat_IEM(13:15);
by_hat    = x_hat_IEM(16:18);
alpha_hat = x_hat_IEM(19:21);

%% Corrigir IMU
den = 1 - alpha_hat;
for k = 1:3
    if abs(den(k)) < 1e-6
        if den(k) >= 0
            den(k) = 1e-6;
        else
            den(k) = -1e-6;
        end
    end
end

M_alpha = diag(1./den);
```

```
f_hat_b = M_alpha*(acc_b_in + ba_hat);
omega_hat_b = gyro_b_in + bg_hat;

%% Propagacao nominal
Rbn = Rb2n_321(eta_hat);
Teta = T_321(eta_hat);

if acc_input_is_translational
    % acc_b_in ja e aceleracao translacional em corpo.
    a_hat_n = Rbn*f_hat_b;
else
    % acc_b_in e forca especifica em corpo.
    a_hat_n = Rbn*f_hat_b + gn;
end

x_pred = x_hat_IEM;
x_pred(1:3)   = p_hat + v_hat*dt + 0.5*a_hat_n*dt^2;
x_pred(4:6)   = v_hat + a_hat_n*dt;
x_pred(7:9)   = eta_hat + Teta*omega_hat_b*dt;
x_pred(10:12) = ba_hat;
x_pred(13:15) = bg_hat;
x_pred(16:18) = beta_y*by_hat;
x_pred(19:21) = alpha_hat;

x_pred(7:9) = wrapToPi_local(x_pred(7:9));

%% Linearizacao
F = zeros(nx,nx);
F(1:3,4:6) = I3;

[Rphi,Rtheta,Rpsi] = dRb2n_321(eta_hat);
Fv_eta = [Rphi*f_hat_b, Rtheta*f_hat_b, Rpsi*f_hat_b];
Fv_ba = Rbn*M_alpha;
Fv_alpha = Rbn*diag((acc_b_in + ba_hat)./((1 - alpha_hat).^2));

[Tphi,Ttheta,Tpsi] = dT_321(eta_hat);
Feta_eta = [Tphi*omega_hat_b, Ttheta*omega_hat_b, Tpsi*omega_hat_b];
Feta_bg = Teta;

F(4:6,7:9)     = Fv_eta;
F(4:6,10:12)   = Fv_ba;
F(4:6,19:21)   = Fv_alpha;
F(7:9,7:9)     = Feta_eta;
F(7:9,13:15)   = Feta_bg;
F(16:18,16:18) = -lambda_y*I3;

G = zeros(nx,15);
G(4:6,1:3)     = Rbn*M_alpha;
G(7:9,4:6)     = Teta;
G(10:12,7:9)   = I3;
G(13:15,10:12) = I3;
G(16:18,13:15) = I3;

Phi = eye(nx) + F*dt;
Qd = G*Qw*G'*dt;

P_IEM = Phi*P_IEM*Phi' + Qd;
P_IEM = 0.5*(P_IEM + P_IEM');

x_hat_IEM = x_pred;

%% Consideração ou não de medida de correção
correction_allowed = true;

range_no_corr = EKF_INDI_params.range_time_without_correction;

for kk = 1:size(range_no_corr,1)
    ti = range_no_corr(kk,1);
    tf = range_no_corr(kk,2);

    if ~(ti == 0 && tf == 0)
        if t_now >= ti && t_now < tf
            correction_allowed = false;
        end
    end
end

%% Atualizacao auxiliar GPS a 1 Hz
do_gps_update = false;
if t_now >= next_gps_time_IEM
    do_gps_update = true;
    while next_gps_time_IEM <= t_now
        next_gps_time_IEM = next_gps_time_IEM + EKF_INDI_params.gps_period;
    end
end
```

```
if do_gps_update && correction_allowed

    %% Posicao auxiliar
    N_meas = pos_meas_in(1);
    E_meas = pos_meas_in(2);
    z3_meas = pos_meas_in(3);

    if isfield(EKF_INDI_params, 'pos_meas_mode') && strcmpi(EKF_INDI_params.pos_meas_mode, 'NED')
        D_meas = z3_meas;
    else
        % Padrao: pos_meas_in = [N; E; h]
        D0 = EKF_INDI_params.pos0_ned(3);
        h_meas = z3_meas;
        D_meas = D0 - (h_meas - h0_meas_IEM);
    end

    z_pos = [N_meas; E_meas; D_meas];

    H_pos = [I3 Z3 Z3 Z3 Z3 I3 Z3];
    y_hat_pos = x_hat_IEM(1:3) + x_hat_IEM(16:18);
    innov_pos = z_pos - y_hat_pos;

    S_pos = H_pos*P_IEM*H_pos' + R_pos;
    K_pos = P_IEM*H_pos'/S_pos;

    dx_hat = K_pos*innov_pos;
    x_hat_IEM = x_hat_IEM + dx_hat;
    x_hat_IEM(7:9) = wrapToPi_local(x_hat_IEM(7:9));

    P_IEM = (eye(nx) - K_pos*H_pos)*P_IEM*(eye(nx) - K_pos*H_pos)' + K_pos*R_pos*K_pos';
    P_IEM = 0.5*(P_IEM + P_IEM');

    %% Velocidade auxiliar
    z_vel = vel_meas_in;

    H_vel = [Z3 I3 Z3 Z3 Z3 Z3 Z3];
    innov_vel = z_vel - x_hat_IEM(4:6);

    S_vel = H_vel*P_IEM*H_vel' + R_vel;
    K_vel = P_IEM*H_vel'/S_vel;

    dx_hat_v = K_vel*innov_vel;
    x_hat_IEM = x_hat_IEM + dx_hat_v;
    x_hat_IEM(7:9) = wrapToPi_local(x_hat_IEM(7:9));

    P_IEM = (eye(nx) - K_vel*H_vel)*P_IEM*(eye(nx) - K_vel*H_vel)' + K_vel*R_vel*K_vel';
    P_IEM = 0.5*(P_IEM + P_IEM');

end

%% Atualizacao de yaw pelo magnetometro
% Usa apenas eul_meas_in(3). Roll e pitch medidos nao sao usados.

do_mag_update = false;

if isempty(next_mag_time_IEM)
    if isfield(EKF_INDI_params, 'mag_period')
        next_mag_time_IEM = t_now + EKF_INDI_params.mag_period;
    else
        next_mag_time_IEM = t_now + 0.02;
    end
end

if t_now >= next_mag_time_IEM
    do_mag_update = true;

    if isfield(EKF_INDI_params, 'mag_period')
        mag_period = EKF_INDI_params.mag_period;
    else
        mag_period = 0.02;
    end

    while next_mag_time_IEM <= t_now
        next_mag_time_IEM = next_mag_time_IEM + mag_period;
    end
end

if do_mag_update

    yaw_meas = eul_meas_in(3);

    if isfinite(yaw_meas)

        yaw_hat = x_hat_IEM(9);
        innov_yaw = wrapToPi_local(yaw_meas - yaw_hat);
```

```
        % Gate de seguranca para evitar correcao com medida absurda
        if abs(innov_yaw) <= yaw_gate_rad

            H_yaw = zeros(1,nx);
            H_yaw(9) = 1;

            S_yaw = H_yaw*P_IEM*H_yaw' + R_yaw;
            K_yaw = P_IEM*H_yaw'/S_yaw;

            dx_hat_yaw = K_yaw * innov_yaw;

            x_hat_IEM = x_hat_IEM + dx_hat_yaw;
            x_hat_IEM(7:9) = wrapToPi_local(x_hat_IEM(7:9));

            P_IEM = (eye(nx) - K_yaw*H_yaw)*P_IEM*(eye(nx) - K_yaw*H_yaw)' + K_yaw*R_yaw*K_yaw';
            P_IEM = 0.5*(P_IEM + P_IEM');

        end
    end
end

%% Atualizar tempo
t_prev_IEM = t_now;

%% Saidas
euler_out = x_hat_IEM(7:9);
vel_out   = x_hat_IEM(4:6);
pos_out   = ekf_pos_output(x_hat_IEM(1:3));  % [N; E; altitude]
acc_n_out = a_hat_n;
xhat_out  = x_hat_IEM;

end

%% HELPERS
function R = Rb2n_321(eta)
phi = eta(1); theta = eta(2); psi = eta(3);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta); sth = sin(theta);
cpsi = cos(psi); spsi = sin(psi);

R = [ cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
      cth*spsi,  sphi*sth*spsi + cphi*cpsi,  cphi*sth*spsi - sphi*cpsi;
     -sth,       sphi*cth,                    cphi*cth ];
end

function T = T_321(eta)
phi = eta(1); theta = eta(2);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    if cth >= 0
        cth = 1e-6;
    else
        cth = -1e-6;
    end
end

T = [1, sphi*tan(theta), cphi*tan(theta);
     0, cphi,           -sphi;
     0, sphi/cth,        cphi/cth];
end

function [Rphi,Rtheta,Rpsi] = dRb2n_321(eta)
phi = eta(1); theta = eta(2); psi = eta(3);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta); sth = sin(theta);
cpsi = cos(psi); spsi = sin(psi);

Rphi = [0, cphi*sth*cpsi + sphi*spsi, -sphi*sth*cpsi + cphi*spsi;
        0, cphi*sth*spsi - sphi*cpsi, -sphi*sth*spsi - cphi*cpsi;
        0, cphi*cth,                  -sphi*cth];

Rtheta = [-sth*cpsi, sphi*cth*cpsi, cphi*cth*cpsi;
          -sth*spsi, sphi*cth*spsi, cphi*cth*spsi;
          -cth,     -sphi*sth,     -cphi*sth];

Rpsi = [-cth*spsi, -sphi*sth*spsi - cphi*cpsi, -cphi*sth*spsi + sphi*cpsi;
         cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
         0,         0,                           0];
end

function [Tphi,Ttheta,Tpsi] = dT_321(eta)
phi = eta(1); theta = eta(2);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta);
```

```
if abs(cth) < 1e-6
    if cth >= 0
        cth = 1e-6;
    else
        cth = -1e-6;
    end
end

sec_th = 1/cth;
tan_th = tan(theta);

Tphi = [0, cphi*tan_th, -sphi*tan_th;
        0, -sphi,       -cphi;
        0, cphi*sec_th, -sphi*sec_th];

Ttheta = [0, sphi*sec_th^2,      cphi*sec_th^2;
          0, 0,                  0;
          0, sphi*sec_th*tan_th, cphi*sec_th*tan_th];

Tpsi = zeros(3);
end

function ang = wrapToPi_local(ang)
ang = mod(ang + pi, 2*pi) - pi;
end

function pos_out = ekf_pos_output(pos_ned)

N = pos_ned(1);
E = pos_ned(2);
D = pos_ned(3);

altitude = -D;

pos_out = [N; E; altitude];

end
```

4.30. navigation/FK/EKF_INDI_solver.m

Arquivo: navigation/FK/EKF_INDI_solver.m

Extensão: .m

Conteúdo:

```
function [euler_out, vel_out, pos_out, acc_n_out, xhat_out] = EKF_INDI_solver( ...
    acc_b_in, gyro_b_in, pos_meas_in, vel_meas_in, eul_meas_in, reset, sample_valid, t_now, EKF_INDI_params)
% EKF_INDI_solver
%
% Adaptacao para Simulink do EKF indireto 3D isolado.
%
% Estado mantido:
%   x_hat = [p^n; v^n; eta; ba; bg; by; alpha_a]
%
% Entradas:
%   acc_b_in     = aceleracao no corpo [m/s^2]
%   gyro_b_in    = [p; q; r] [rad/s]
%   pos_meas_in  = [N; E; h] ou [N; E; D], conforme params.pos_meas_mode
%   vel_meas_in  = [vN; vE; vD] [m/s]
%   eul_meas_in  = [roll; pitch; yaw] [rad], usa apenas yaw
%   reset
%   sample_valid
%   t_now
%   EKF_INDI_params
%
% Saidas:
%   euler_out = [phi; theta; psi]
%   vel_out   = [vN; vE; vD]
%   pos_out   = [N; E; altitude]
%   acc_n_out = aceleracao estimada em NED
%   xhat_out  = estado completo 21x1

persistent x_hat_ID
persistent P_ID
persistent t_prev_ID
persistent initialized_ID
persistent last_reset_token_ID
persistent next_gps_time_ID
persistent next_mag_time_ID
persistent h0_meas_ID

nx = 21;
I3 = eye(3);
Z3 = zeros(3);

euler_out = zeros(3,1);
vel_out   = zeros(3,1);
pos_out   = zeros(3,1);
acc_n_out = zeros(3,1);
xhat_out  = zeros(nx,1);

if isempty(initialized_ID)
    initialized_ID = false;
end

if ~isfield(EKF_INDI_params, 'initialized') || ~EKF_INDI_params.initialized
    return;
end

if isempty(last_reset_token_ID)
    last_reset_token_ID = -1;
end

new_params_loaded = false;
if isfield(EKF_INDI_params, 'reset_token')
    if EKF_INDI_params.reset_token ~= last_reset_token_ID
        new_params_loaded = true;
        last_reset_token_ID = EKF_INDI_params.reset_token;
    end
end

%% Inicializacao
if reset || ~initialized_ID || new_params_loaded

    x_hat_ID = EKF_INDI_params.x0;
    P_ID = EKF_INDI_params.P0;

    t_prev_ID = t_now;
    initialized_ID = true;

    next_gps_time_ID = t_now + EKF_INDI_params.gps_period;

    if isfield(EKF_INDI_params, 'mag_period')
```

```
        next_mag_time_ID = t_now + EKF_INDI_params.mag_period;
    else
        next_mag_time_ID = t_now + 0.02;
    end

    % pos_meas_in tipicamente vem como [N; E; h] do X-Plane.
    h0_meas_ID = pos_meas_in(3);

    euler_out = x_hat_ID(7:9);
    vel_out   = x_hat_ID(4:6);
    pos_out   = ekf_pos_output(x_hat_ID(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_ID;

    return;
end

%% Sem amostra valida: mantem estados
if sample_valid == 0
    euler_out = x_hat_ID(7:9);
    vel_out   = x_hat_ID(4:6);
    pos_out   = ekf_pos_output(x_hat_ID(1:3));
    acc_n_out = zeros(3,1);
    xhat_out  = x_hat_ID;
    return;
end

%% Tempo
dt = t_now - t_prev_ID;
if dt <= 0
    dt = 0;
end

%% Parametros
Qw = EKF_INDI_params.Qw;
R_pos = EKF_INDI_params.R_pos;
R_vel = EKF_INDI_params.R_vel;
lambda_y = EKF_INDI_params.lambda_y;
beta_y = exp(-lambda_y*dt);

if isfield(EKF_INDI_params, 'R_yaw')
    R_yaw = EKF_INDI_params.R_yaw;
else
    R_yaw = deg2rad(2.0)^2;
end

if isfield(EKF_INDI_params, 'acc_input_is_translational')
    acc_input_is_translational = EKF_INDI_params.acc_input_is_translational;
else
    acc_input_is_translational = true;
end

gn = [0; 0; EKF_INDI_params.g0];

%% Separar estados
p_hat     = x_hat_ID(1:3);
v_hat     = x_hat_ID(4:6);
eta_hat   = x_hat_ID(7:9);
ba_hat    = x_hat_ID(10:12);
bg_hat    = x_hat_ID(13:15);
by_hat    = x_hat_ID(16:18);
alpha_hat = x_hat_ID(19:21);

%% Corrigir IMU
den = 1 - alpha_hat;
for k = 1:3
    if abs(den(k)) < 1e-6
        if den(k) >= 0
            den(k) = 1e-6;
        else
            den(k) = -1e-6;
        end
    end
end

M_alpha = diag(1./den);

f_hat_b = M_alpha*(acc_b_in + ba_hat);
omega_hat_b = gyro_b_in + bg_hat;

%% Propagacao nominal
Rbn = Rb2n_321(eta_hat);
Teta = T_321(eta_hat);

if acc_input_is_translational
    % acc_b_in ja e aceleracao translacional em corpo.
```

```
    a_hat_n = Rbn*f_hat_b;
else
    % acc_b_in e forca especifica em corpo.
    a_hat_n = Rbn*f_hat_b + gn;
end

x_pred = x_hat_ID;
x_pred(1:3)   = p_hat + v_hat*dt + 0.5*a_hat_n*dt^2;
x_pred(4:6)   = v_hat + a_hat_n*dt;
x_pred(7:9)   = eta_hat + Teta*omega_hat_b*dt;
x_pred(10:12) = ba_hat;
x_pred(13:15) = bg_hat;
x_pred(16:18) = beta_y*by_hat;
x_pred(19:21) = alpha_hat;

x_pred(7:9) = wrapToPi_local(x_pred(7:9));

%% Linearizacao
F = zeros(nx,nx);
F(1:3,4:6) = I3;

[Rphi,Rtheta,Rpsi] = dRb2n_321(eta_hat);
Fv_eta = [Rphi*f_hat_b, Rtheta*f_hat_b, Rpsi*f_hat_b];
Fv_ba = Rbn*M_alpha;
Fv_alpha = Rbn*diag((acc_b_in + ba_hat)./((1 - alpha_hat).^2));

[Tphi,Ttheta,Tpsi] = dT_321(eta_hat);
Feta_eta = [Tphi*omega_hat_b, Ttheta*omega_hat_b, Tpsi*omega_hat_b];
Feta_bg = Teta;

F(4:6,7:9)     = Fv_eta;
F(4:6,10:12)   = Fv_ba;
F(4:6,19:21)   = Fv_alpha;
F(7:9,7:9)     = Feta_eta;
F(7:9,13:15)   = Feta_bg;
F(16:18,16:18) = -lambda_y*I3;

G = zeros(nx,15);
G(4:6,1:3)     = Rbn*M_alpha;
G(7:9,4:6)     = Teta;
G(10:12,7:9)   = I3;
G(13:15,10:12) = I3;
G(16:18,13:15) = I3;

Phi = eye(nx) + F*dt;
Qd = G*Qw*G'*dt;

P_ID = Phi*P_ID*Phi' + Qd;
P_ID = 0.5*(P_ID + P_ID');

x_hat_ID = x_pred;

%% Consideração ou não de medida de correção
correction_allowed = true;

range_no_corr = EKF_INDI_params.range_time_without_correction;

for kk = 1:size(range_no_corr,1)
    ti = range_no_corr(kk,1);
    tf = range_no_corr(kk,2);

    if ~(ti == 0 && tf == 0)
        if t_now >= ti && t_now < tf
            correction_allowed = false;
        end
    end
end

%% Atualizacao auxiliar GPS
do_gps_update = false;
if t_now >= next_gps_time_ID
    do_gps_update = true;
    while next_gps_time_ID <= t_now
        next_gps_time_ID = next_gps_time_ID + EKF_INDI_params.gps_period;
    end
end

if do_gps_update && correction_allowed

    %% Posicao auxiliar
    N_meas = pos_meas_in(1);
    E_meas = pos_meas_in(2);
    z3_meas = pos_meas_in(3);

    if isfield(EKF_INDI_params, 'pos_meas_mode') && strcmpi(EKF_INDI_params.pos_meas_mode, 'NED')
        D_meas = z3_meas;
```

```
    else
        % Padrao: pos_meas_in = [N; E; h]
        D0 = EKF_INDI_params.pos0_ned(3);
        h_meas = z3_meas;
        D_meas = D0 - (h_meas - h0_meas_ID);
    end

    z_pos = [N_meas; E_meas; D_meas];

    H_pos = [I3 Z3 Z3 Z3 Z3 I3 Z3];
    y_hat_pos = x_hat_ID(1:3) + x_hat_ID(16:18);
    innov_pos = z_pos - y_hat_pos;

    S_pos = H_pos*P_ID*H_pos' + R_pos;
    K_pos = P_ID*H_pos'/S_pos;

    dx_hat = K_pos*innov_pos;
    x_hat_ID = x_hat_ID + dx_hat;
    x_hat_ID(7:9) = wrapToPi_local(x_hat_ID(7:9));

    P_ID = (eye(nx) - K_pos*H_pos)*P_ID*(eye(nx) - K_pos*H_pos)' + K_pos*R_pos*K_pos';
    P_ID = 0.5*(P_ID + P_ID');

    %% Velocidade auxiliar
    z_vel = vel_meas_in;

    H_vel = [Z3 I3 Z3 Z3 Z3 Z3 Z3];
    innov_vel = z_vel - x_hat_ID(4:6);

    S_vel = H_vel*P_ID*H_vel' + R_vel;
    K_vel = P_ID*H_vel'/S_vel;

    dx_hat_v = K_vel*innov_vel;
    x_hat_ID = x_hat_ID + dx_hat_v;
    x_hat_ID(7:9) = wrapToPi_local(x_hat_ID(7:9));

    P_ID = (eye(nx) - K_vel*H_vel)*P_ID*(eye(nx) - K_vel*H_vel)' + K_vel*R_vel*K_vel';
    P_ID = 0.5*(P_ID + P_ID');

end

%% Atualizacao de yaw pelo magnetometro
do_mag_update = false;

if t_now >= next_mag_time_ID
    do_mag_update = true;

    if isfield(EKF_INDI_params, 'mag_period')
        mag_period = EKF_INDI_params.mag_period;
    else
        mag_period = 0.02;
    end

    while next_mag_time_ID <= t_now
        next_mag_time_ID = next_mag_time_ID + mag_period;
    end
end

if do_mag_update

    % eul_meas_in = [roll; pitch; yaw]
    % Apenas yaw e usado na correcao.
    yaw_meas = eul_meas_in(3);

    if isfinite(yaw_meas)

        H_yaw = zeros(1,nx);
        H_yaw(9) = 1;

        yaw_hat = x_hat_ID(9);

        innov_yaw = wrapToPi_local(yaw_meas - yaw_hat);

        S_yaw = H_yaw*P_ID*H_yaw' + R_yaw;
        K_yaw = P_ID*H_yaw'/S_yaw;

        dx_hat_yaw = K_yaw*innov_yaw;
        x_hat_ID = x_hat_ID + dx_hat_yaw;
        x_hat_ID(7:9) = wrapToPi_local(x_hat_ID(7:9));

        P_ID = (eye(nx) - K_yaw*H_yaw)*P_ID*(eye(nx) - K_yaw*H_yaw)' + K_yaw*R_yaw*K_yaw';
        P_ID = 0.5*(P_ID + P_ID');

    end
end
```

```
%% Atualizar tempo
t_prev_ID = t_now;

%% Saidas
euler_out = x_hat_ID(7:9);
vel_out   = x_hat_ID(4:6);
pos_out   = ekf_pos_output(x_hat_ID(1:3));  % [N; E; altitude]
acc_n_out = a_hat_n;
xhat_out  = x_hat_ID;

end

%% HELPERS
function R = Rb2n_321(eta)
phi = eta(1); theta = eta(2); psi = eta(3);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta); sth = sin(theta);
cpsi = cos(psi); spsi = sin(psi);

R = [ cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
      cth*spsi,  sphi*sth*spsi + cphi*cpsi,  cphi*sth*spsi - sphi*cpsi;
     -sth,       sphi*cth,                    cphi*cth ];
end

function T = T_321(eta)
phi = eta(1); theta = eta(2);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    if cth >= 0
        cth = 1e-6;
    else
        cth = -1e-6;
    end
end

T = [1, sphi*tan(theta), cphi*tan(theta);
     0, cphi,           -sphi;
     0, sphi/cth,        cphi/cth];
end

function [Rphi,Rtheta,Rpsi] = dRb2n_321(eta)
phi = eta(1); theta = eta(2); psi = eta(3);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta); sth = sin(theta);
cpsi = cos(psi); spsi = sin(psi);

Rphi = [0, cphi*sth*cpsi + sphi*spsi, -sphi*sth*cpsi + cphi*spsi;
        0, cphi*sth*spsi - sphi*cpsi, -sphi*sth*spsi - cphi*cpsi;
        0, cphi*cth,                  -sphi*cth];

Rtheta = [-sth*cpsi, sphi*cth*cpsi, cphi*cth*cpsi;
          -sth*spsi, sphi*cth*spsi, cphi*cth*spsi;
          -cth,     -sphi*sth,     -cphi*sth];

Rpsi = [-cth*spsi, -sphi*sth*spsi - cphi*cpsi, -cphi*sth*spsi + sphi*cpsi;
         cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi;
         0,         0,                           0];
end

function [Tphi,Ttheta,Tpsi] = dT_321(eta)
phi = eta(1); theta = eta(2);
cphi = cos(phi); sphi = sin(phi);
cth = cos(theta);

if abs(cth) < 1e-6
    if cth >= 0
        cth = 1e-6;
    else
        cth = -1e-6;
    end
end

sec_th = 1/cth;
tan_th = tan(theta);

Tphi = [0, cphi*tan_th, -sphi*tan_th;
        0, -sphi,       -cphi;
        0, cphi*sec_th, -sphi*sec_th];

Ttheta = [0, sphi*sec_th^2,      cphi*sec_th^2;
          0, 0,                  0;
          0, sphi*sec_th*tan_th, cphi*sec_th*tan_th];

Tpsi = zeros(3);
```

```
end

function ang = wrapToPi_local(ang)
ang = mod(ang + pi, 2*pi) - pi;
end

function pos_out = ekf_pos_output(pos_ned)

N = pos_ned(1);
E = pos_ned(2);
D = pos_ned(3);

altitude = -D;

pos_out = [N; E; altitude];

end
```

4.31. navigation/FK/init_EKF_DI.m

Arquivo: navigation/FK/init_EKF_DI.m

Extensão: .m

Conteúdo:

```
function EKF_DI_params = init_EKF_DI(xplane_init)
% init_EKF_DI
%
% Inicializa parâmetros do EKF direto.
%
% Entrada:
%   xplane_init.euler0    = [phi0; theta0; psi0] [rad]
%   xplane_init.pos0_ned  = [N0; E0; D0] [m]
%   xplane_init.vel0_ned  = [vN0; vE0; vD0] [m/s]
%
% Saída:
%   EKF_DI_params no workspace base.

EKF_DI_params = struct();
EKF_DI_params.initialized = true;

%% Condições iniciais

EKF_DI_params.euler0   = xplane_init.euler0(:);
EKF_DI_params.pos0_ned = xplane_init.pos0_ned(:);
EKF_DI_params.vel0_ned = xplane_init.vel0_ned(:);

EKF_DI_params.g0 = 9.807;

%% Carregar sensores

try
    sensors = evalin('base', 'sensors');
    EKF_DI_params.sensors = sensors;
catch
    sensors = struct();
    EKF_DI_params.sensors = sensors;
end

%% Parâmetros do sensor

if isfield(sensors, 'icm20689')

    acc = sensors.icm20689.acc;
    gyro = sensors.icm20689.gyro;

    sigma_acc = acc.sigma_noise_ms2;
    sigma_gyro = deg2rad(gyro.sigma_noise_dps);

    sigma_ba = acc.zero_g_temp_mg_per_C * 1e-3 * acc.gravity * 0.05;
    sigma_bg = deg2rad(gyro.bias_temp) * 0.05;

    sigma_ba0 = acc.zero_g_sigma_mg * 1e-3 * acc.gravity;
    sigma_bg0 = deg2rad(5/3);

else

    sigma_acc = 0.02;
    sigma_gyro = deg2rad(0.05);

    sigma_ba = 1e-4;
    sigma_bg = deg2rad(1e-3);

    sigma_ba0 = 0.1;
    sigma_bg0 = deg2rad(0.5);

end

%% Parâmetros GPS/pseudo-GPS

EKF_DI_params.gps_Fs = 1;                  % GPS a 1 Hz
EKF_DI_params.gps_period = 1/EKF_DI_params.gps_Fs;

EKF_DI_params.R_pos = diag([1.0^2, 1.0^2, 1.5^2]);
EKF_DI_params.R_vel = (0.05^2)*eye(3);

lambda_y = 1/60;
EKF_DI_params.lambda_y = lambda_y;

sigma_by = sqrt(3)/30;
EKF_DI_params.sigma_by = sigma_by;

%% Parâmetros Magnetômetro / yaw
```

```
if isfield(sensors, 'ist8310') && isfield(sensors.ist8310, 'mag')

    mag = sensors.ist8310.mag;

    if isfield(mag, 'Fs')
        EKF_DI_params.mag_Fs = mag.Fs;
    elseif isfield(mag, 'Ts')
        EKF_DI_params.mag_Fs = 1/mag.Ts;
    else
        EKF_DI_params.mag_Fs = EKF_DI_params.gps_Fs;
    end

    EKF_DI_params.mag_period = 1/EKF_DI_params.mag_Fs;

    if isfield(mag, 'mag_n_ref_uT')
        mag_horiz_norm = norm(mag.mag_n_ref_uT(1:2));
    else
        mag_horiz_norm = norm([23; -5]);
    end

    if isfield(mag, 'sigma_noise_uT')
        sigma_yaw_mag = max(mag.sigma_noise_uT / max(mag_horiz_norm, 1e-6), deg2rad(0.8));
    else
        sigma_yaw_mag = deg2rad(2.0);
    end

    EKF_DI_params.R_yaw = sigma_yaw_mag^2;

else

    EKF_DI_params.mag_Fs = 50;
    EKF_DI_params.mag_period = 1/EKF_DI_params.mag_Fs;
    EKF_DI_params.R_yaw = deg2rad(2.0)^2;

end

%% Estado inicial

nx = 21;

x0 = zeros(nx,1);

x0(1:3)   = EKF_DI_params.pos0_ned;
x0(4:6)   = EKF_DI_params.vel0_ned;
x0(7:9)   = EKF_DI_params.euler0;
x0(10:12) = zeros(3,1);   % bias acelerômetro
x0(13:15) = zeros(3,1);   % bias giroscópio
x0(16:18) = zeros(3,1);   % bias posição/GPS
x0(19:21) = zeros(3,1);   % fator de escala acelerômetro

EKF_DI_params.x0 = x0;

%% Covariância inicial

P0 = diag([ ...
    1.0^2*ones(1,3), ...
    0.5^2*ones(1,3), ...
    deg2rad(2)^2*ones(1,3), ...
    sigma_ba0^2*ones(1,3), ...
    sigma_bg0^2*ones(1,3), ...
    3.0^2*ones(1,3), ...
    0.01^2*ones(1,3) ...
]);

EKF_DI_params.P0 = P0;

%% Ruídos de processo

Qw = diag([ ...
    sigma_acc^2*ones(1,3), ...
    sigma_gyro^2*ones(1,3), ...
    sigma_ba^2*ones(1,3), ...
    sigma_bg^2*ones(1,3), ...
    sigma_by^2*ones(1,3) ...
]);

EKF_DI_params.Qw = Qw;

%% Convenção da aceleração

% acc_b_in vindo do ins_read_xplane é aceleração translacional no corpo.
% Portanto, no EKF:
%   a_n = Rb2n * acc_b_corrigida
% sem adicionar/subtrair gravidade.
EKF_DI_params.acc_input_is_translational = true;
```

```
%% Time ranges without GPS/yaw correction
try
    EKF_DI_params.range_time_without_correction = ...
        evalin('base','range_time_without_correction');
catch
    EKF_DI_params.range_time_without_correction = [0 0];
end

%% Token de reinicialização

EKF_DI_params.reset_token = now;

assignin('base', 'EKF_DI_params', EKF_DI_params);

clear EKF_DI_solver;
clear EKF_DI_EM_solver;

disp('--- EKF_DI_params inicializado ---');
fprintf('Euler0 [deg] = [%.3f %.3f %.3f]\n', rad2deg(EKF_DI_params.euler0));
fprintf('Pos0 NED [m] = [%.3f %.3f %.3f]\n', EKF_DI_params.pos0_ned);
fprintf('Vel0 NED [m/s] = [%.3f %.3f %.3f]\n', EKF_DI_params.vel0_ned);

end
```

4.32. navigation/FK/init_EKF_INDI.m

Arquivo: navigation/FK/init_EKF_INDI.m

Extensão: .m

Conteúdo:

```
function EKF_INDI_params = init_EKF_INDI(xplane_init)
% init_EKF_INDI
%
% Inicializa os parametros do EKF indireto para uso no Simulink.
%
% Entrada:
%   xplane_init.euler0    = [phi0; theta0; psi0] [rad]
%   xplane_init.pos0_ned  = [N0; E0; D0] [m]
%   xplane_init.vel0_ned  = [vN0; vE0; vD0] [m/s]
%
% Saida:
%   EKF_INDI_params publicado no workspace base.

EKF_INDI_params = struct();
EKF_INDI_params.initialized = true;

%% Condicoes iniciais
EKF_INDI_params.euler0   = xplane_init.euler0(:);
EKF_INDI_params.pos0_ned = xplane_init.pos0_ned(:);
EKF_INDI_params.vel0_ned = xplane_init.vel0_ned(:);

EKF_INDI_params.g0 = 9.80665;

%% Sensores
try
    sensors = evalin('base', 'sensors');
catch
    sensors = struct();
end

EKF_INDI_params.sensors = sensors;

%% Parametros de ruido vindos da struct sensors, com fallback
if isfield(sensors, 'icm20689') && isfield(sensors.icm20689, 'acc')
    acc = sensors.icm20689.acc;
else
    acc = struct();
end

if isfield(sensors, 'icm20689') && isfield(sensors.icm20689, 'gyro')
    gyro = sensors.icm20689.gyro;
else
    gyro = struct();
end

% Acelerometro
if isfield(acc, 'sigma_noise_ms2')
    sigma_acc = acc.sigma_noise_ms2;
else
    sigma_acc = 0.02;
end

if isfield(acc, 'gravity')
    g_acc = acc.gravity;
else
    g_acc = EKF_INDI_params.g0;
end

if isfield(acc, 'zero_g_sigma_mg')
    sigma_ba0 = acc.zero_g_sigma_mg * 1e-3 * g_acc;
else
    sigma_ba0 = 0.30;
end

if isfield(acc, 'kT_bias_g_per_C')
    sigma_ba_vec = abs(acc.kT_bias_g_per_C(:)) * g_acc * 0.05;
else
    sigma_ba_vec = 0.002 * ones(3,1);
end

% Giroscopio
if isfield(gyro, 'noise_density')
    sigma_gyro = deg2rad(gyro.noise_density);
elseif isfield(gyro, 'sigma_noise_dps')
    sigma_gyro = deg2rad(gyro.sigma_noise_dps);
else
    sigma_gyro = deg2rad(0.05);
end
```

```
if isfield(gyro, 'b0')
    sigma_bg0 = max(std(deg2rad(gyro.b0(:))), deg2rad(0.5));
else
    sigma_bg0 = deg2rad(5);
end

if isfield(gyro, 'k_zro')
    sigma_bg_vec = abs(deg2rad(gyro.k_zro(:))) * 0.05;
else
    sigma_bg_vec = deg2rad(0.01) * ones(3,1);
end

%% Medidas auxiliares tipo GPS
EKF_INDI_params.gps_Fs = 1;
EKF_INDI_params.gps_period = 1/EKF_INDI_params.gps_Fs;

% Posicao: [N E D], com terceiro eixo mais ruidoso
EKF_INDI_params.R_pos = diag([1.0^2, 1.0^2, 1.5^2]);

% Velocidade 3D NED
EKF_INDI_params.R_vel = (0.05^2) * eye(3);

% Bias de posicao tipo Gauss-Markov
EKF_INDI_params.lambda_y = 1/60;
EKF_INDI_params.sigma_by = sqrt(2*EKF_INDI_params.lambda_y) * 3.0;

%% Medida auxiliar de yaw / magnetometro

if isfield(sensors, 'ist8310') && isfield(sensors.ist8310, 'mag')

    mag = sensors.ist8310.mag;

    if isfield(mag, 'Fs')
        EKF_INDI_params.mag_Fs = mag.Fs;
    elseif isfield(mag, 'Ts')
        EKF_INDI_params.mag_Fs = 1/mag.Ts;
    else
        EKF_INDI_params.mag_Fs = 50;
    end

    EKF_INDI_params.mag_period = 1/EKF_INDI_params.mag_Fs;

    if isfield(mag, 'mag_n_ref_uT')
        mag_horiz_norm = norm(mag.mag_n_ref_uT(1:2));
    else
        mag_horiz_norm = norm([23; -5]);
    end

    if isfield(mag, 'sigma_noise_uT')
        sigma_yaw_mag = mag.sigma_noise_uT / max(mag_horiz_norm, 1e-6);
        sigma_yaw_mag = max(sigma_yaw_mag, deg2rad(0.8));
    else
        sigma_yaw_mag = deg2rad(2.0);
    end

    EKF_INDI_params.R_yaw = sigma_yaw_mag^2;

else

    EKF_INDI_params.mag_Fs = 50;
    EKF_INDI_params.mag_period = 1/EKF_INDI_params.mag_Fs;
    EKF_INDI_params.R_yaw = deg2rad(2.0)^2;

end

%% Estado inicial completo
nx = 21;
x0 = zeros(nx,1);

x0(1:3)   = EKF_INDI_params.pos0_ned;
x0(4:6)   = EKF_INDI_params.vel0_ned;
x0(7:9)   = EKF_INDI_params.euler0;
x0(10:12) = zeros(3,1);  % bias acelerometro
x0(13:15) = zeros(3,1);  % bias giroscopio
x0(16:18) = zeros(3,1);  % bias da medida de posicao
x0(19:21) = zeros(3,1);  % fator de escala acelerometro

EKF_INDI_params.x0 = x0;

%% Covariancia inicial - baseada no script isolado do EKF indireto
P0 = diag([ ...
    10.0^2*ones(1,3), ...
     1.0^2*ones(1,3), ...
     deg2rad(5)^2*ones(1,3), ...
     sigma_ba0^2*ones(1,3), ...
     sigma_bg0^2*ones(1,3), ...
```

```
     3.0^2*ones(1,3), ...
     0.02^2*ones(1,3) ...
]);

EKF_INDI_params.P0 = P0;

%% Ruido de processo continuo
Qw = diag([ ...
    sigma_acc^2*ones(3,1); ...
    sigma_gyro^2*ones(3,1); ...
    sigma_ba_vec(:).^2; ...
    sigma_bg_vec(:).^2; ...
    EKF_INDI_params.sigma_by^2*ones(3,1) ...
]);

EKF_INDI_params.Qw = Qw;

%% Convencao da aceleracao
% Pela sua leitura do X-Plane, acc_b_in ja e aceleracao translacional em corpo.
% Portanto, por padrao nao se soma gravidade na propagacao.
EKF_INDI_params.acc_input_is_translational = true;

%% Time ranges without GPS/yaw correction
try
    EKF_INDI_params.range_time_without_correction = ...
        evalin('base','range_time_without_correction');
catch
    EKF_INDI_params.range_time_without_correction = [0 0];
end

%% Token para forcar reinicializacao entre simulacoes
EKF_INDI_params.reset_token = now;

assignin('base', 'EKF_INDI_params', EKF_INDI_params);

clear EKF_INDI_solver;
clear EKF_INDI_EM_solver;

disp('--- EKF_INDI_params inicializado ---');
fprintf('Euler0 [deg] = [%.3f %.3f %.3f]\n', rad2deg(EKF_INDI_params.euler0));
fprintf('Pos0 NED [m] = [%.3f %.3f %.3f]\n', EKF_INDI_params.pos0_ned);
fprintf('Vel0 NED [m/s] = [%.3f %.3f %.3f]\n', EKF_INDI_params.vel0_ned);

end
```

4.33. navigation/ins_init_block.m

Arquivo: navigation/ins_init_block.m

Extensão: .m

Conteúdo:

```
%% INICIANDO SENSORES
ins_init_sensors(); % load variables para o modelo e os dados dos sensores
%% ========== DBN / Inertial Navigation Initialization ==========
DBN_params = struct();
DBN_params.initialized = false;
assignin('base', 'DBN_params', DBN_params);
% Placeholder. O DBN_params real será criado por init_DBN,
% chamado dentro de ins_initial_state_xplane.m após reposicionar o X-Plane.
% Essa struct também guarda os parametros de inicialização para os EKFs
% Os EKFs também são inicializados em ins_initial_state_xplane.m
%% INFORMANDO
fprintf("\nins_init_block: DBN_params carregado no path.\n" + ...
    "A inicialização real ocorrerá via:\ninit_DBN, init_EKF_DI, " + ...
    "init_EKF_INDI após posicionamento inicial no X-Plane.\n");
```

4.34. navigation/README.md

Arquivo: navigation/README.md

Extensão: .md

Conteúdo:

```
# Inertial Navigation Block
```

4.35. navigation/sensors/ICM20689/ins_acc_errors_icm20689.m

Arquivo: navigation/sensors/ICM20689/ins_acc_errors_icm20689.m

Extensão: .m

Conteúdo:

```
function acc_meas_ms2 = ins_acc_errors_icm20689(acc_true_ms2, T, noise_ms2, sensors)
%#codegen
% INS_ACC_ERROS_ICM20689 - núcleo do modelo de erros do acelerômetro ICM20689.
%
% Entradas:
%   acc_true_ms2 : [3x1] aceleração verdadeira em m/s^2
%   T            : [1x1] temperatura em °C
%   noise_ms2    : [3x1] ruído externo em m/s^2, já amostrado em Ts
%   sensors      : struct de sensores inicializada por ins_init_sensors()
%
% Saída:
%   acc_meas_ms2 : [3x1] aceleração medida em m/s^2

acc_true_ms2 = acc_true_ms2(:);
noise_ms2    = noise_ms2(:);

% Lê parâmetros fixos do workspace/modelo
a = sensors.icm20689.acc;

% ------------------------------------------------------------------------
% (1) Montagem nominal corpo -> sensor
% ------------------------------------------------------------------------
roll  = deg2rad(a.roll_mount_deg);
pitch = deg2rad(a.pitch_mount_deg);
yaw   = deg2rad(a.yaw_mount_deg);

R_roll = [ ...
    1,          0,           0; ...
    0,  cos(roll),   sin(roll); ...
    0, -sin(roll),   cos(roll)];

R_pitch = [ ...
     cos(pitch), 0, -sin(pitch); ...
              0, 1,           0; ...
     sin(pitch), 0,  cos(pitch)];

R_yaw = [ ...
     cos(yaw),  sin(yaw), 0; ...
    -sin(yaw),  cos(yaw), 0; ...
            0,         0, 1];

R_b2s = R_roll * R_pitch * R_yaw;

acc_nom = R_b2s * acc_true_ms2;

% ------------------------------------------------------------------------
% (2) Misalignment fino do acelerômetro
% ------------------------------------------------------------------------
acc_mis = a.R_mis * acc_nom;

% ------------------------------------------------------------------------
% (3) Cross-axis
% ------------------------------------------------------------------------
acc_cross = a.Ca * acc_mis;

% ------------------------------------------------------------------------
% (4) Scale factor fixo + variação térmica
% ------------------------------------------------------------------------
sf = a.sf_fixed + (T - a.T_ref) .* a.k_sf;
acc_sf = acc_cross .* sf(:);

% ------------------------------------------------------------------------
% (5) Não-linearidade cúbica
% ------------------------------------------------------------------------
FS_ms2 = a.fundo_escala;

acc_nl = acc_sf + (acc_sf.^3) .* (a.k3(:) / (FS_ms2^2));

% ------------------------------------------------------------------------
% (6) Bias zero-g + drift térmico
% ------------------------------------------------------------------------
bias_g = a.bias0_g + (T - a.T_ref) .* a.kT_bias_g_per_C;
bias_ms2 = bias_g(:) * a.gravity;

% ------------------------------------------------------------------------
% (7) Soma com ruído externo
% ------------------------------------------------------------------------
acc_ana = acc_nl + bias_ms2 + noise_ms2;
```

```
% ------------------------------------------------------------------------
% (8) Saturação em ±FS
% ------------------------------------------------------------------------
acc_sat = min(max(acc_ana, -FS_ms2), FS_ms2);

% ------------------------------------------------------------------------
% (9) Quantização em LSB
% ------------------------------------------------------------------------
acc_sat_g = acc_sat / a.gravity;

counts = round(acc_sat_g * a.sens_LSB_per_g);
counts = min(max(counts, -32768), 32767);

% ------------------------------------------------------------------------
% (10) Zona morta pós-quantização
% ------------------------------------------------------------------------
D = a.deadzone_LSB;

abs_c = abs(counts);
counts_dz = zeros(size(counts));

mask = abs_c > D;
counts_dz(mask) = sign(counts(mask)) .* (abs_c(mask) - D);

% ------------------------------------------------------------------------
% (11) Volta para m/s^2
% ------------------------------------------------------------------------
acc_meas_g = counts_dz / a.sens_LSB_per_g;
acc_meas_ms2 = acc_meas_g * a.gravity;

end
```

4.36. navigation/sensors/ICM20689/ins_gyro_errors_icm20689.m

Arquivo: navigation/sensors/ICM20689/ins_gyro_errors_icm20689.m

Extensão: .m

Conteúdo:

```
function omega_meas_dps = ins_gyro_errors_icm20689(omega_true_dps, T, noise_dps, sensors)
omega_true_dps = omega_true_dps(:);
noise_dps      = noise_dps(:);
%#codegen
% GyroSensor_ICM20689 - núcleo do modelo em °/s.
% Entradas:
%   omega_true_dps : [3x1] taxa verdadeira em °/s
%   T              : [1x1] temperatura em °C
%   noise_dps      : [3x1] ruído externo em °/s (já amostrado em Ts)
% Saída:
%   omega_meas_dps : [3x1] taxa medida em °/s
% Lê parâmetros fixos do workspace (congelados em tempo de execução)
g = sensors.icm20689.gyro;
% --- (1) Montagem nominal ---
roll = deg2rad(g.yaw_mount_deg);
pit = deg2rad(g.yaw_mount_deg);
yaw = deg2rad(g.yaw_mount_deg);
R_roll = [1 0 0; 0 cos(roll) sin(roll);0 -sin(roll) cos(roll)];
R_pitch = [cos(pit) 0 -sin(pit); 0 1 0; sin(pit) 0 cos(pit)];
R_yaw = [ cos(yaw) sin(yaw) 0; -sin(yaw) cos(yaw) 0;0 0 1];
R_b2s = R_roll*R_pitch*R_yaw;
omega_nom = R_b2s* omega_true_dps;
% --- (2) Cross-axis ---
omega_cross = g.Cg * omega_nom;
% --- (3) Scale factor (fixo + variação com T, se você estiver usando) ---
sf = g.sf_fixed + (T - g.T_ref).*g.k_sf;   % 1x3
omega_sf = omega_cross .* sf(:);
% --- (4) Não-linearidade (opcional) ---
FS = g.FS_dps;
omega_nl = omega_sf + (omega_sf.^3) .* (g.k3(:) / (FS^2));
% --- (5) Bias (ZRO + drift térmico) ---
bias = g.b0 + (T - g.T_ref).*g.k_zro;      % 1x3
bias = bias(:);
% --- (6) Soma com ruído externo ---
omega_ana = omega_nl + bias + noise_dps;
% --- (7) Saturação em ±FS ---
omega_sat = min(max(omega_ana, -FS), FS);
% --- (8) Quantização (LSB) ---
counts = round(omega_sat * g.sens_LSB_per_dps);
counts = min(max(counts, -32768), 32767);
% --- (9) Zona morta (soft deadband) ---
D = g.deadzone_LSB;
abs_c = abs(counts);
counts_dz = zeros(size(counts));
mask = abs_c > D;
counts_dz(mask) = sign(counts(mask)) .* (abs_c(mask) - D);
% --- (10) Volta para °/s ---
omega_meas_dps = counts_dz / g.sens_LSB_per_dps;
end
```

4.37. navigation/sensors/ICM20689/ins_init_icm20689.m

Arquivo: navigation/sensors/ICM20689/ins_init_icm20689.m

Extensão: .m

Conteúdo:

```
function icm20689 = ins_init_icm20689(seeds_icm20689)
%INIT_ICM20689 Inicializa parâmetros do ICM20689 (por enquanto: gyro).
% Esta função é chamada por init_sensors() e NÃO publica nada no Base Workspace.
% Ela apenas retorna a struct icm20689 com subestruturas (ex.: icm20689.gyro).
%
% Entrada:
%   seeds_icm20689: struct com seeds do ICM20689, por exemplo:
%       seeds_icm20689.gyro.params
%       seeds_icm20689.gyro.noise (opcional, usado no Simulink nos blocos Random Number)
%       seeds_icm20689.acc.params (futuro)
%
% Saída:
%   icm20689: struct com parâmetros do sensor (ex.: icm20689.gyro.*)
    %% (0) Reprodutibilidade (parâmetros sorteados)
    % Usamos o seed específico do GYRO e ACC para sortear os parâmetros fixos do sensor
    % (cross-axis, bias inicial, k3, etc.). Isso garante que o "mesmo sensor"
    % seja reproduzido sempre que a simulação rodar com o mesmo seed.
    if nargin >= 1 && isfield(seeds_icm20689,'gyro') && isfield(seeds_icm20689.gyro,'params')
        rng(seeds_icm20689.gyro.params);
    else
        % fallback (não recomendado): mantém comportamento previsível
        rng(1);
    end
        % ACC - Reprodutibilidade
    if nargin >= 1 && isfield(seeds_icm20689,'acc') && isfield(seeds_icm20689.acc,'params')
        rng(seeds_icm20689.acc.params);
    else
        rng(2);
    end
    %% (1) Girometros - Parâmetros basicos
    gyro = struct();
    gyro.Ts = 1/200;                % [s] taxa do sensor/PA (200 Hz)
    gyro.Fs = 1/gyro.Ts;            % [Hz] correção do bug (antes estava 1/Ts)
    gyro.FS_dps = 1000;             % [°/s] fundo de escala escolhido
    gyro.bits = 16;                 % [bits] resolução
    gyro.sens_LSB_per_dps = 32.8;   % [LSB/(°/s)] típico para ±1000 °/s
    gyro.sf_tol = 0.02;             % [-] erro de escala (±2% típico)
    gyro.sf_temp_dev_typ = 0.015;   % [-] variação típica vs temperatura (±1.5%)
    gyro.bias_temp = 0.05;          % [°/s/°C] drift térmico do bias (±0.05)
    gyro.noise_density = 0.006;     % [°/s/√Hz] noise density
    gyro.noise_BW = 59;             % [Hz] Noise BW (DLPF_CFG=3)
    gyro.sigma_noise_dps = gyro.noise_density * sqrt(gyro.noise_BW);
    gyro.cross_axis_max = 0.05;     % [-] cross-axis (pior caso; típico seria 0.02)
    gyro.nonlin_max = 0.001;        % [-] não-linearidade (±0.1%)
    gyro.deadzone_LSB = 1;          % [LSB] zona morta pós-quantização
    gyro.T_ref = 25;                % [°C] referência
    gyro.yaw_mount_deg = 0;         % [deg] montagem nominal (0 => implícito)
    gyro.pitch_mount_deg = 0;       % [deg] montagem nominal (0 => implícito)
    gyro.roll_mount_deg = 0;        % [deg] montagem nominal (0 => implícito)
    % Sorteios por eixo (fixos por simulação)
    % distribuicoes tipicas via 3 sigma
    gyro.sf_fixed = 1 + (gyro.sf_tol/3)*randn(1,3);
    gyro.k_sf     = (gyro.sf_temp_dev_typ/3)/65 * randn(1,3);    % slope por eixo
    gyro.b0       = (5/3)*randn(1,3);                            % ZRO inicial típico
    gyro.k_zro    = (2*rand(1,3)-1)*gyro.bias_temp;              % coef térmico do bias
    gyro.k3       = (2*rand(1,3)-1)*gyro.nonlin_max;             % não-linearidade por eixo
    % Matriz cross-axis 3x3 (Cg)
    E = zeros(3);
    for i = 1:3
        for j = 1:3
            if i ~= j
                E(i,j) = (2*rand-1)*gyro.cross_axis_max;
            end
        end
    end
    gyro.Cg = eye(3) + E;
    %% (2) Acelerometros - Parâmetros basicos
    acc = struct();
    acc.Ts = 1/200;
    acc.Fs = 1/acc.Ts;
    acc.gravity = 9.807;
    acc.FS_g = 8;                       % ±8g
    acc.bits = 16;
    acc.sens_LSB_per_g = 4096;          % LSB/g para ±8g
    acc.fundo_escala = acc.FS_g * acc.gravity; % m/s^2
    acc.zero_g_max_mg = 80;
    acc.zero_g_sigma_mg = acc.zero_g_max_mg/3;
    acc.zero_g_temp_mg_per_C = 0.75;
```

```
    acc.noise_density_ug_sqrtHz = 150;
    acc.noise_density_g_sqrtHz = acc.noise_density_ug_sqrtHz * 1e-6;
    acc.noise_BW_Hz = 61.5;
    acc.sigma_noise_g = acc.noise_density_g_sqrtHz * sqrt(acc.noise_BW_Hz);
    acc.sigma_noise_ms2 = acc.sigma_noise_g * acc.gravity;
    acc.sf_tol_typ = 0.02;
    acc.sf_temp_dev_typ = 0.01;
    acc.nonlin_max = 0.005;
    acc.cross_axis_max = 0.05;
    acc.deadzone_LSB = 1;
    acc.T_ref = 25;
    % desalinhamento do eixo
    acc.roll_mount_deg = 0;
    acc.pitch_mount_deg = 0;
    acc.yaw_mount_deg = 0;
    % ACC - Bias e drift térmico
    acc.bias0_g = (acc.zero_g_sigma_mg*1e-3) * randn(1,3);
    acc.kT_bias_g_per_C = ...
        (acc.zero_g_temp_mg_per_C*1e-3) * (2*rand(1,3)-1);
    % ACC - Misalignment
    % O datasheet não fornece desalinhamento angular diretamente.
    % Este bloco fica como parâmetro controlável do modelo.
    % Se quiser desligar, usar acc.misalign_deg_max = 0.
    acc.misalign_deg_max = 0.0;
    acc.misalign_sigma_rad = deg2rad(acc.misalign_deg_max)/3;
    alpha = acc.misalign_sigma_rad * randn;
    beta  = acc.misalign_sigma_rad * randn;
    gamma = acc.misalign_sigma_rad * randn;
    acc.R_mis = eye(3) + [ ...
          0     -gamma    beta;
        gamma      0     -alpha;
       -beta     alpha      0 ];
    % ACC - Cross-axis
    E = zeros(3);
    for i = 1:3
        for j = 1:3
            if i ~= j
                E(i,j) = (2*rand-1)*acc.cross_axis_max;
            end
        end
    end
    acc.Ca = eye(3) + E;
    % ACC - Scale factor
    sf_sigma = acc.sf_tol_typ/3;
    acc.sf_fixed = 1 + sf_sigma*randn(1,3);
    dT_max = 65;
    k_sf_sigma = (acc.sf_temp_dev_typ/3)/dT_max;
    acc.k_sf = k_sf_sigma*randn(1,3);
    % ACC - Não-linearidade
    acc.k3 = (2*rand(1,3)-1) * acc.nonlin_max;
    %% (3) Monta a struct final do ICM20689
    icm20689 = struct();
    icm20689.gyro = gyro;
    icm20689.acc = acc;
    disp("icm20689 sensor successfully creadted")
end
```

4.38. navigation/sensors/ins_init_sensors.m

Arquivo: navigation/sensors/ins_init_sensors.m

Extensão: .m

Conteúdo:

```
%% Funcao Principal
function sensors = ins_init_sensors()
    disp("---------- Sensors ----------")
    %INIT_SENSORS Inicializa e consolida parâmetros do sistema e sensores.
    % Publica UMA ÚNICA variável no Base Workspace: 'sensors'.

    %% (0) Configuração local
    Ts = 1/200;          % sample time do PA/IMU
    Fs = 1/Ts;
    seed_master = 1;     % seed mestre do experimento

    %% (1) Estrutura do sistema
    sensors = struct();
    sensors.sys = struct();
    sensors.sys.Ts = Ts;
    sensors.sys.Fs = Fs;
    sensors.sys.seed_master = seed_master;
    sensors.seeds = struct();

    %% seeds específicos para cada sensor
    sensors = icm20689_seeds_generator(sensors); % ICM20689
    sensors = gps_seeds_generator(sensors);      % GPS / GNSS
    sensors = ist8310_seeds_generator(sensors);  % IST8310 / Magnetometro

    %% Inicializando sensores
    sensors.icm20689 = ins_init_icm20689(sensors.seeds.icm20689); % ICM20689 - ICM
    sensors.gps      = ins_init_gps(sensors.seeds.gps);           % u-blox NEO-M8
    sensors.ist8310  = ins_init_ist8310(sensors.seeds.ist8310);   % magnetometro

    %% Publica no workspace
    assignin('base', 'sensors', sensors);
    clear all;
    disp("-----------------------------")
end

%% SEEDS ICM20689
function sensors = icm20689_seeds_generator(sensors)
    % =========================
    % (2) Seeds globais (separa parâmetros de ruído)
    % =========================
    seed_master = sensors.sys.seed_master;
    seed_params = seed_master + 1000;  % para sorteio de parâmetros (Cg, b0, k3, etc.)
    seed_noise  = seed_master + 2000;  % para ruído temporal (Random Number no Simulink)
    sensors.sys.seed_params = seed_params;
    sensors.sys.seed_noise = seed_noise;
    % =========================
    % (3) Offsets (padrão simples e escalável)
    % =========================
    OFF.icm20689 = 100;
    OFF.gyro     = 10;
    OFF.acc      = 20;

    % gerador de seeds específicos para cada sensor
     % para evitar poluir função principal conforme o número de sensores
    % aumentar
    % =========================
    % (4) Seeds por sensor/componente
    % =========================
    sensors.seeds.icm20689 = struct();

    % ---- ICM20689 / GYRO ----
    sensors.seeds.icm20689.gyro = struct();
    sensors.seeds.icm20689.gyro.params = seed_params + OFF.icm20689 + OFF.gyro;
    sensors.seeds.icm20689.gyro.noise  = [ ...
        seed_noise + OFF.icm20689 + OFF.gyro + 1, ... % X
        seed_noise + OFF.icm20689 + OFF.gyro + 2, ... % Y
        seed_noise + OFF.icm20689 + OFF.gyro + 3  ... % Z
    ];

    % ---- ICM20689 / ACC ----
    sensors.seeds.icm20689.acc = struct();
    sensors.seeds.icm20689.acc.params = seed_params + OFF.icm20689 + OFF.acc;
    sensors.seeds.icm20689.acc.noise  = [ ...
        seed_noise + OFF.icm20689 + OFF.acc + 1, ... % X
        seed_noise + OFF.icm20689 + OFF.acc + 2, ... % Y
        seed_noise + OFF.icm20689 + OFF.acc + 3  ... % Z
    ];
end
```

```
%% SEEDS GPS/GNSS - u-blox NEO-M8
function sensors = gps_seeds_generator(sensors)

seed_master = sensors.sys.seed_master;
seed_params = seed_master + 1000;
seed_noise  = seed_master + 2000;

OFF.gps = 300;

sensors.seeds.gps = struct();

sensors.seeds.gps.params = seed_params + OFF.gps;

% Ruído branco de posição NED
sensors.seeds.gps.noise_pos = [ ...
    seed_noise + OFF.gps + 1, ... % N
    seed_noise + OFF.gps + 2, ... % E
    seed_noise + OFF.gps + 3  ... % D
    ];

% Ruído branco de velocidade NED
sensors.seeds.gps.noise_vel = [ ...
    seed_noise + OFF.gps + 4, ... % vN
    seed_noise + OFF.gps + 5, ... % vE
    seed_noise + OFF.gps + 6  ... % vD
    ];

% Ruído unitário para o bias Gauss-Markov
sensors.seeds.gps.noise_bias = [ ...
    seed_noise + OFF.gps + 7, ... % bias N
    seed_noise + OFF.gps + 8, ... % bias E
    seed_noise + OFF.gps + 9  ... % bias D
    ];

end
%% SEEDS IST8310
function sensors = ist8310_seeds_generator(sensors)

    seed_master = sensors.sys.seed_master;
    seed_params = seed_master + 1000;
    seed_noise  = seed_master + 2000;

    OFF.ist8310 = 400;
    OFF.mag     = 10;

    sensors.seeds.ist8310 = struct();
    sensors.seeds.ist8310.mag = struct();

    sensors.seeds.ist8310.mag.params = seed_params + OFF.ist8310 + OFF.mag;

    sensors.seeds.ist8310.mag.noise = [ ...
        seed_noise + OFF.ist8310 + OFF.mag + 1, ... % X
        seed_noise + OFF.ist8310 + OFF.mag + 2, ... % Y
        seed_noise + OFF.ist8310 + OFF.mag + 3  ... % Z
    ];
end
```

4.39. navigation/sensors/ins_telemetry.m

Arquivo: navigation/sensors/ins_telemetry.m

Extensão: .m

Conteúdo:

```
function ins_telemetry(time, h, Vt, V3D, V3D_EKF, cmds, euler)
%INS_TELEMETRY Telemetria textual limitada por intervalo de tempo.
%
% Entradas:
%   time     : tempo usado para telemetria, preferencialmente t_nav ou t_xplane_rel [s]
%   h        : altitude [m]
%   Vt       : velocidade aerodinamica [m/s]
%   V3D      : velocidade X-Plane/NED [vN; vE; vD] ou equivalente
%   V3D_EKF  : velocidade estimada EKF [vN; vE; vD]
%   cmds     : comandos [thr; elev; ail; rud]
%   euler    : [phi_x; theta_x; psi_x; phi_ekf; theta_ekf; psi_ekf]

persistent last_print_time

if isempty(last_print_time)
    last_print_time = -inf;
end

%% Configuracoes do workspace

try
    activate_telemetry_f = evalin('base', 'activate_telemetry');
catch
    activate_telemetry_f = false;
end

try
    consider_FK_in_telemetry_f = evalin('base', 'consider_FK_in_telemetry');
catch
    consider_FK_in_telemetry_f = false;
end

try
    ins_telemetry_time_interval_f = evalin('base', 'ins_telemetry_time_interval');
catch
    ins_telemetry_time_interval_f = 1.0;
end

%% Validacoes basicas

if ~activate_telemetry_f
    return;
end

if ~isfinite(time)
    return;
end

if ins_telemetry_time_interval_f <= 0
    ins_telemetry_time_interval_f = 1.0;
end

%% Controle de intervalo de impressao

if time < last_print_time
    % Caso a simulacao reinicie
    last_print_time = -inf;
end

if (time - last_print_time) < ins_telemetry_time_interval_f
    return;
end

last_print_time = time;

%% Garantir vetores coluna

V3D = V3D(:);
V3D_EKF = V3D_EKF(:);
cmds = cmds(:);
euler = euler(:);

%% Protecao contra tamanhos inesperados

if numel(V3D) < 3
    V3D = [V3D; zeros(3 - numel(V3D), 1)];
end
```

```
if numel(V3D_EKF) < 3
    V3D_EKF = [V3D_EKF; zeros(3 - numel(V3D_EKF), 1)];
end

if numel(cmds) < 4
    cmds = [cmds; zeros(4 - numel(cmds), 1)];
end

if numel(euler) < 6
    euler = [euler; zeros(6 - numel(euler), 1)];
end

%% Velocidades

Vx = V3D(1);
Vy = V3D(2);
Vz = V3D(3);

Vmod = sqrt(Vx^2 + Vy^2 + Vz^2);

Vxfk = V3D_EKF(1);
Vyfk = V3D_EKF(2);
Vzfk = V3D_EKF(3);

Vmod_fk = sqrt(Vxfk^2 + Vyfk^2 + Vzfk^2);

%% Comandos

thr  = cmds(1);
elev = cmds(2);
ail  = cmds(3);
rud  = cmds(4);

%% Euler

phix   = euler(1);
thetax = euler(2);
psix   = euler(3);

phie   = euler(4);
thetae = euler(5);
psie   = euler(6);

%% Impressao

if consider_FK_in_telemetry_f

    fprintf(['\nt: %.2f s | h: %.2f m | VT: %.2f m/s | VXP: %.2f m/s | ' ...
             'VEKF: %.2f m/s | ' ...
             'Thr: %.2f | Ele: %.2f | Ail: %.2f | Rud: %.2f | ' ...
             'phiX: %.2f deg | thetaX: %.2f deg | psiX: %.2f deg | ' ...
             'phiEKF: %.2f deg | thetaEKF: %.2f deg | psiEKF: %.2f deg\n'], ...
             time, h, Vt, Vmod, Vmod_fk, ...
             thr, elev, ail, rud, ...
             rad2deg(phix), rad2deg(thetax), rad2deg(psix), ...
             rad2deg(phie), rad2deg(thetae), rad2deg(psie));

else

    fprintf(['\nt: %.2f s | h: %.2f m | VT: %.2f m/s | VXP_3D: %.2f m/s | ' ...
             'Thr: %.2f | Ele: %.2f | Ail: %.2f | Rud: %.2f | ' ...
             'phiX: %.2f deg | thetaX: %.2f deg | psiX: %.2f deg\n'], ...
             time, h, Vt, Vmod, ...
             thr, elev, ail, rud, ...
             rad2deg(phix), rad2deg(thetax), rad2deg(psix));

end

end
```

4.40. navigation/sensors/IST8310/ins_init_ist8310.m

Arquivo: navigation/sensors/IST8310/ins_init_ist8310.m

Extensão: .m

Conteúdo:

```
function ist8310 = ins_init_ist8310(seeds_ist8310)
%INS_INIT_IST8310 Inicializa parametros do magnetometro IST8310.

%% (0) Reprodutibilidade
if nargin >= 1 && isfield(seeds_ist8310,'mag') && isfield(seeds_ist8310.mag,'params')
    rng(seeds_ist8310.mag.params);
else
    rng(11);
end

%% (1) Parametros basicos
mag = struct();

mag.Ts = 1/200;
mag.Fs = 1/mag.Ts;

mag.noise_BW_Hz = 50;

mag.range_xy_uT = 1600;
mag.range_z_uT  = 2500;
mag.range_uT = [mag.range_xy_uT, mag.range_xy_uT, mag.range_z_uT];

mag.bits = 16;
mag.sens_LSB_per_uT = 3.3;
mag.resolution_uT_per_LSB = 1/mag.sens_LSB_per_uT;

%% (2) Parametros de erro
mag.linearity = [0.01 0.001 0.001];

mag.offset0_uT = [0.3 -0.3 0.3];

mag.sens_temp_drift = 0.00016;
mag.offset_temp_drift_uT_per_C = 0.024;

mag.hysteresis = 0.001;

mag.misalignment_rad = deg2rad(0.05);
mag.cross_axis = 0.01;

mag.noise_density_uT_sqrtHz = 0.05;
mag.sigma_noise_uT = mag.noise_density_uT_sqrtHz * sqrt(mag.noise_BW_Hz);

mag.dead_zone_uT = 0.05;
mag.T_ref = 25;

%% (3) Campo magnetico local NED
% Fallback interno. Na simulacao online, a conversao para heading verdadeiro
% deve usar magnetic_variation_rad vindo do X-Plane.
mag.mag_n_ref_uT = [23; -5; -38];

mag.declination_rad = atan2(mag.mag_n_ref_uT(2), mag.mag_n_ref_uT(1));

%% (4) Matrizes de desalinhamento e cross-axis
a = mag.misalignment_rad;

mag.M_align = [ ...
    1,  a, 0; ...
   -a,  1, 0; ...
    0,  0, 1];

c = mag.cross_axis;

mag.M_cross = [ ...
    1, c, c; ...
    c, 1, c; ...
    c, c, 1];

mag.M_mag = mag.M_cross * mag.M_align;

%% (5) Coeficientes fixos sorteados
mag.bias0_uT = mag.offset0_uT;

mag.kT_bias_uT_per_C = ...
    mag.offset_temp_drift_uT_per_C * (2*rand(1,3)-1);

mag.sf_fixed = 1 + (mag.linearity/3).*randn(1,3);

mag.k_sf = (mag.sens_temp_drift/3) * randn(1,3);
```

```
mag.k3 = (2*rand(1,3)-1).*mag.linearity;

%% (6) Limites digitais
mag.raw_min_LSB = -2^(mag.bits-1);
mag.raw_max_LSB =  2^(mag.bits-1) - 1;

%% (7) Seeds para ruido no Simulink
if nargin >= 1 && isfield(seeds_ist8310,'mag') && isfield(seeds_ist8310.mag,'noise')
    mag.noise_seed = seeds_ist8310.mag.noise;
else
    mag.noise_seed = [4011 4012 4013];
end

%% (8) Struct final
ist8310 = struct();
ist8310.mag = mag;

disp("ist8310 sensor successfully created")
end
```

4.41. navigation/sensors/IST8310/ins_mag_meas.m

Arquivo: navigation/sensors/IST8310/ins_mag_meas.m

Extensão: .m

Conteúdo:

```
function yaw_true_degraded_rad = ins_mag_meas(euler_true_rad, T, noise_mag_uT, mag_var, sensors)
%#codegen
%INS_MAG_MEAS Modelo de erro do magnetometro IST8310.
%
% Entradas:
%   euler_true_rad        : [3x1] [phi; theta; psi] do X-Plane [rad]
%   T                     : temperatura [deg C]
%   noise_mag_uT          : [3x1] ruido externo do magnetometro [uT]
%   mag_var               : variacao magnetica local do X-Plane [rad]
%   sensors               : struct sensors
%
% Saida:
%   yaw_true_degraded_rad : yaw verdadeiro degradado pelo modelo do magnetometro [rad]

euler_true_rad = euler_true_rad(:);
noise_mag_uT   = noise_mag_uT(:);

mag = sensors.ist8310.mag;

mag_var = wrapToPi_local(mag_var);

%% (1) Campo magnetico local NED usando magnetic_variation do X-Plane
% Usa a magnitude horizontal e vertical definidas em ins_init_ist8310,
% mas substitui a direcao horizontal pela variacao magnetica do X-Plane.

H_mag_uT = norm(mag.mag_n_ref_uT(1:2));
D_mag_uT = mag.mag_n_ref_uT(3);

mag_n_ref_uT = [ ...
    H_mag_uT * cos(mag_var); ...
    H_mag_uT * sin(mag_var); ...
    D_mag_uT];

%% (2) Campo magnetico ideal no corpo
Rbn = Rb2n_321_local(euler_true_rad);

mag_true_b_uT = Rbn' * mag_n_ref_uT;

%% (3) Desalinhamento e cross-axis
mag_mis = mag.M_mag * mag_true_b_uT;

%% (4) Fator de escala fixo + termico
sf = mag.sf_fixed(:) + (T - mag.T_ref) .* mag.k_sf(:);

mag_scale = mag_mis .* sf;

%% (5) Nao-linearidade
range_uT = mag.range_uT(:);
k3 = mag.k3(:);

mag_nl = k3 .* (mag_scale.^2 ./ range_uT) .* sign(mag_scale);

%% (6) Bias fixo + drift termico
bias_uT = mag.bias0_uT(:) + (T - mag.T_ref) .* mag.kT_bias_uT_per_C(:);

%% (7) Histerese
% Mantida zerada para evitar offset artificial constante no yaw.
mag_hyst = zeros(3,1);

%% (8) Soma dos erros
mag_ana = mag_scale + mag_nl + bias_uT + mag_hyst + noise_mag_uT;

%% (9) Zona morta
D = mag.dead_zone_uT;

mag_dz = mag_ana;
mag_dz(abs(mag_dz) < D) = 0;

%% (10) Saturacao
mag_sat = min(max(mag_dz, -range_uT), range_uT);

%% (11) Quantizacao
raw = round(mag_sat * mag.sens_LSB_per_uT);

raw = min(max(raw, mag.raw_min_LSB), mag.raw_max_LSB);

mag_meas_b_uT = raw / mag.sens_LSB_per_uT;
```

```
%% (12) Tilt compensation usando roll e pitch do X-Plane
roll  = euler_true_rad(1);
pitch = euler_true_rad(2);

R_level = Rb2n_321_local([roll; pitch; 0]);

mag_level = R_level * mag_meas_b_uT;

%% (13) Yaw verdadeiro degradado
% Modelo coerente com:
%   yaw_true = magnetic_variation - atan2(E_mag_level, N_mag_level)
%
% Sem erros, essa expressao retorna aproximadamente o psi verdadeiro
% usado para gerar mag_true_b_uT.

yaw_true_degraded_rad = mag_var - atan2(mag_level(2), mag_level(1));

yaw_true_degraded_rad = wrapToPi_local(yaw_true_degraded_rad);

end

%% Funcoes locais
function R = Rb2n_321_local(euler_rad)

    phi   = euler_rad(1);
    theta = euler_rad(2);
    psi   = euler_rad(3);

    cphi = cos(phi);
    sphi = sin(phi);
    cth  = cos(theta);
    sth  = sin(theta);
    cpsi = cos(psi);
    spsi = sin(psi);

    R = [ ...
        cth*cpsi,  sphi*sth*cpsi - cphi*spsi,  cphi*sth*cpsi + sphi*spsi; ...
        cth*spsi,  sphi*sth*spsi + cphi*cpsi,  cphi*sth*spsi - sphi*cpsi; ...
        -sth,      sphi*cth,                    cphi*cth];
end

function ang = wrapToPi_local(ang)

    ang = mod(ang + pi, 2*pi) - pi;

end
```

4.42. navigation/sensors/NEOM8/ins_gps_measurements.m

Arquivo: navigation/sensors/NEOM8/ins_gps_measurements.m

Extensão: .m

Conteúdo:

```
function y_gps = ins_gps_measurements(pos_ned_true, vel_ned_true, t, ...
                                      noise_pos_m, noise_vel_ms, w_bias_pos, sensors)
%INS_GPS_MEASUREMENTS Aplica modelo de erro GNSS.
%
% Entradas variantes no tempo:
%   pos_ned_true : [3x1] posição verdadeira NED [m]
%   vel_ned_true : [3x1] velocidade verdadeira NED [m/s]
%   t            : [1x1] tempo [s]
%   noise_pos_m  : [3x1] ruído branco de posição [m]
%   noise_vel_ms : [3x1] ruído branco de velocidade [m/s]
%   w_bias_pos   : [3x1] ruído normal unitário para excitar bias
%
% Parâmetros fixos:
%   gps          : sensors.gps
%
% Saída:
%   y_gps        : [6x1] [pos_NED; vel_NED] com erro aplicado

persistent bias_pos last_t initialized

gps = sensors.gps; % parametros do gps
pos_ned_true = pos_ned_true(:);
vel_ned_true = vel_ned_true(:);
noise_pos_m  = noise_pos_m(:);
noise_vel_ms = noise_vel_ms(:);
w_bias_pos   = w_bias_pos(:);

%% (0) Inicialização persistente
if isempty(initialized) || t < last_t
    bias_pos = gps.bias0_pos(:);
    last_t = t;
    initialized = true;
end

%% (1) Passo de tempo
dt = t - last_t;

if dt <= 0
    dt = gps.Ts;
end

%% (2) Bias de posição - Gauss-Markov
phi = exp(-gps.lambda_bias * dt);

qd = gps.sigma_bias_ss_NED(:) * sqrt(max(0, 1 - phi^2));

bias_pos = phi * bias_pos + qd .* w_bias_pos;

last_t = t;

%% (3) Medição de posição
pos_meas = pos_ned_true + bias_pos + noise_pos_m;

%% (4) Medição de velocidade
vel_meas = vel_ned_true + noise_vel_ms;

%% (5) Saída
y_gps = [pos_meas; vel_meas];

end
```

4.43. navigation/sensors/NEOM8/ins_init_gps.m

Arquivo: navigation/sensors/NEOM8/ins_init_gps.m

Extensão: .m

Conteúdo:

```
function gps = ins_init_gps(seeds_gps, model_name)
%INS_INIT_GPS Inicializa parâmetros fixos de erro do GNSS u-blox NEO-M8.
%
% Modelo:
%   pos_meas = pos_ned_true + bias_pos + noise_pos_m
%   vel_meas = vel_ned_true + noise_vel_ms
%
% Os ruídos variantes no tempo entram pelo Simulink.
% Os parâmetros fixos ficam em sensors.gps.

    %% (0) Modelo padrão
    if nargin < 2 || isempty(model_name)
        model_name = "NEO-M8N";
    end

    %% (1) Reprodutibilidade dos parâmetros fixos
    if nargin >= 1 && isfield(seeds_gps, 'params')
        rng(seeds_gps.params);
    else
        rng(3);
    end

    %% (2) Identificação básica
    gps = struct();

    gps.manufacturer = "u-blox";
    gps.family       = "NEO-M8";
    gps.model        = string(model_name);
    gps.frame        = "NED";

    %% (3) Frequência nominal do GNSS
    switch upper(char(gps.model))
        case 'NEO-M8N'
            gps.Fs = 5;
        case 'NEO-M8Q'
            gps.Fs = 10;
        case 'NEO-M8J'
            gps.Fs = 5;
        case 'NEO-M8M'
            gps.Fs = 10;
        otherwise
            gps.model = "NEO-M8N";
            gps.Fs = 5;
    end

    gps.Ts = 1/gps.Fs;

    %% (4) Erro de posição
    gps.horizontal_accuracy_cep50_m = 2.0;

    gps.sigma_pos_NE_total = gps.horizontal_accuracy_cep50_m / sqrt(2*log(2));

    gps.vertical_sigma_ratio = 1.5;

    gps.sigma_pos_N = gps.sigma_pos_NE_total;
    gps.sigma_pos_E = gps.sigma_pos_NE_total;
    gps.sigma_pos_D = gps.vertical_sigma_ratio * gps.sigma_pos_NE_total;

    gps.sigma_pos_total_NED = [ ...
        gps.sigma_pos_N; ...
        gps.sigma_pos_E; ...
        gps.sigma_pos_D];

    %% (5) Separação entre ruído branco e bias correlacionado
    gps.white_noise_variance_fraction = 0.60;
    gps.bias_variance_fraction        = 0.40;

    gps.sigma_eta_pos_NED = sqrt(gps.white_noise_variance_fraction) ...
        * gps.sigma_pos_total_NED;

    gps.sigma_bias_ss_NED = sqrt(gps.bias_variance_fraction) ...
        * gps.sigma_pos_total_NED;

    %% (6) Bias Gauss-Markov de posição
    gps.lambda_bias = 1/60;  % [1/s]

    gps.bias0_pos = gps.sigma_bias_ss_NED .* randn(3,1);
```

```
    %% (7) Erro de velocidade
    gps.velocity_accuracy_50pct_ms = 0.05;

    gps.sigma_vel_N = gps.velocity_accuracy_50pct_ms;
    gps.sigma_vel_E = gps.velocity_accuracy_50pct_ms;
    gps.sigma_vel_D = 1.5 * gps.velocity_accuracy_50pct_ms;

    gps.sigma_vel_NED = [ ...
        gps.sigma_vel_N; ...
        gps.sigma_vel_E; ...
        gps.sigma_vel_D];

    %% (8) Seeds para blocos Random Number no Simulink
    if nargin >= 1 && isfield(seeds_gps, 'noise_pos')
        gps.noise_seed_pos = seeds_gps.noise_pos;
    else
        gps.noise_seed_pos = [3001 3002 3003];
    end

    if nargin >= 1 && isfield(seeds_gps, 'noise_vel')
        gps.noise_seed_vel = seeds_gps.noise_vel;
    else
        gps.noise_seed_vel = [3004 3005 3006];
    end

    if nargin >= 1 && isfield(seeds_gps, 'noise_bias')
        gps.noise_seed_bias = seeds_gps.noise_bias;
    else
        gps.noise_seed_bias = [3007 3008 3009];
    end

    %% (9) Matrizes úteis
    gps.R_pos = diag(gps.sigma_pos_total_NED.^2);
    gps.R_vel = diag(gps.sigma_vel_NED.^2);
    gps.R     = blkdiag(gps.R_pos, gps.R_vel);

    gps.R_pos_white = diag(gps.sigma_eta_pos_NED.^2);

    disp("u-blox NEO-M8 GNSS sensor successfully created")

end
```

4.44. navigation/sensors/README.md

Arquivo: navigation/sensors/README.md

Extensão: .md

Conteúdo:

```
# SENSORS BASED ON X-Plane
## Here are some explanations and isntructions about the structure builded to acquire and hold data of the used
sensors
## Also here are some explanantions about the erros models used
```

4.45. plots/init_plots.m

Arquivo: plots/init_plots.m

Extensão: .m

Conteúdo:

```
%% PLOTAGEM DOS RESULTADOS
% Plotar resultados do Modelo Não Linear
% evalin('base', 'plot3d_voo');
% Plotar resultados do Xplane
% evalin('base', 'plot3d_voo_xplane');
% Plotar resultados do DBN
% evalin('base', 'plot3d_voo_DBN');
% Plotar os 3 em um só - comparação
evalin('base', 'plot_compare_all');
%evalin('base', 'plot_compare_all2'); % desatualizado
evalin('base', 'plot_compare_all3');
evalin('base', 'plot_compare_all4');
evalin('base', 'plot_compare_all5');
```

4.46. plots/plot3d_voo.m

Arquivo: plots/plot3d_voo.m

Extensão: .m

Conteúdo:

```
%% plot3d_voo.m - Plota trajetória 3D da aeronave
% Rodar APÓS a simulação terminar.
% Requer variável POS no workspace (To Workspace com [xN, xE, alt]).
%
% Se POS não existir, tenta extrair de 'out' ou dos estados.
%
% Uso: >> plot3d_voo

fprintf('\n========== PLOT 3D DO VOO ==========\n');
%% ========== Localizar dados de posição ==========
try
    pos_data = out.Y.signals.values;
    t_pos = out.Y.time;
    xN = pos_data(:,10);
    xE = pos_data(:,11);
    alt = pos_data(:,12);
catch
   disp("Error: data not founded to plot")
end
%% ========== Figura 1: Trajetória 3D ==========
figure('Name', 'Trajetória 3D', 'Position', [50 100 800 600]);
plot3(xE, xN, alt, 'b-', 'LineWidth', 1.5);
hold on;

% Waypoints (apenas markers)
if exist('WPs', 'var')
    plot3(WPs(:,2), WPs(:,1), WPs(:,3), 'rs', 'MarkerSize', 10, ...
          'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, WPs(i,3)+2, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end
end

% Marcar início e fim
plot3(xE(1), xN(1), alt(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot3(xE(end), xN(end), alt(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);

xlabel('Leste (m)'); ylabel('Norte (m)'); zlabel('Altitude (m)');
title('Trajetória 3D da Aeronave');
legend('Trajetória', 'Waypoints', 'Início', 'Fim', 'Location', 'best');
grid on; axis equal;
view(30, 25);
hold off;

%% ========== Figura 2: Vista Superior (Ground Track) ==========
figure('Name', 'Vista Superior', 'Position', [900 100 700 600]);
plot(xE, xN, 'b-', 'LineWidth', 1.5);
hold on;
if exist('WPs', 'var')
    plot(WPs(:,2), WPs(:,1), 'rs', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end
    % Círculo de aceitação
    if exist('R_accept', 'var')
        th = linspace(0, 2*pi, 100);
        for i = 1:size(WPs, 1)
            plot(WPs(i,2) + R_accept*cos(th), WPs(i,1) + R_accept*sin(th), ...
                 'r--', 'LineWidth', 0.5);
        end
    end
end
plot(xE(1), xN(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot(xE(end), xN(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);
xlabel('Leste (m)'); ylabel('Norte (m)');
title('Vista Superior (Ground Track)');
grid on; axis equal;
hold off;

%% ========== Figura 3: Altitude vs Tempo ==========
figure('Name', 'Altitude', 'Position', [50 750 800 400]);
if ~isempty(t_pos)
    plot(t_pos, alt, 'b-', 'LineWidth', 1.5);
    xlabel('Tempo (s)');
else
    plot(alt, 'b-', 'LineWidth', 1.5);
```

```
    xlabel('Amostra');
end
hold on;
% Linhas de referência dos waypoints
if exist('WPs', 'var')
    alt_wps = unique(WPs(:,3));
    for i = 1:length(alt_wps)
        yline(alt_wps(i), 'r--', sprintf('%.0f m', alt_wps(i)), ...
              'LineWidth', 0.8, 'LabelHorizontalAlignment', 'left');
    end
end
ylabel('Altitude (m)');
title('Altitude ao Longo do Voo');
grid on;
hold off;

%% ========== Estatísticas ==========
fprintf('\n--- Estatísticas do Voo ---\n');
fprintf('  Posição inicial: N=%.1f  E=%.1f  Alt=%.1f m\n', xN(1), xE(1), alt(1));
fprintf('  Posição final:   N=%.1f  E=%.1f  Alt=%.1f m\n', xN(end), xE(end), alt(end));
fprintf('  Altitude: min=%.2f  max=%.2f  média=%.2f m\n', min(alt), max(alt), mean(alt));
fprintf('  Distância total percorrida: %.1f m\n', ...
    sum(sqrt(diff(xN).^2 + diff(xE).^2 + diff(alt).^2)));

if exist('WPs', 'var')
    % Distância ao último WP
    d_final = sqrt((xN(end)-WPs(end,1))^2 + (xE(end)-WPs(end,2))^2);
    fprintf('  Distância ao WP final: %.1f m\n', d_final);
end

fprintf('\n========== FIM DO PLOT 3D ==========\n');
```

4.47. plots/plot3d_voo_DBN.m

Arquivo: plots/plot3d_voo_DBN.m

Extensão: .m

Conteúdo:

```
%% plot3d_voo_dbn.m - Plota trajetória 3D calculada pelo DBN
% Rodar APÓS a simulação terminar.
% Requer variável 'out' no workspace, contendo out.BDN_data.
%
% Formato esperado de out.BDN_data.signals.values:
%   colunas  1:3   -> Euler DBN [phi, theta, psi] [rad]
%   colunas  4:6   -> Velocidade DBN em NED [vN, vE, vD] [m/s]
%   colunas  7:9   -> Posição DBN em NED [N, E, D] [m]
%   colunas 10:12  -> Aceleração DBN em NED [aN, aE, aD] [m/s²]
%   colunas 13:16  -> Quaternion Farrel [b1, b2, b3, b4]
%
% Uso:
%   >> plot3d_voo_dbn

fprintf('\n========== PLOT 3D DO VOO A PARTIR DE DADOS DO DBN ==========' );
fprintf('\n');

%% ========== Localizar dados do DBN ==========
try
    dbn_data = out.BDN_data.signals.values;
    t_dbn = out.BDN_data.time;
catch
    error('Erro: dados out.BDN_data não encontrados para plotagem do DBN.');
end

%% ========== Separar sinais ==========
euler_dbn = dbn_data(:,1:3);        % [phi theta psi] [rad]
vel_dbn   = dbn_data(:,4:6);        % [vN vE vD] [m/s]
pos_dbn   = dbn_data(:,7:9);        % [N E D] [m]
acc_dbn   = dbn_data(:,10:12);      % [aN aE aD] [m/s²]

if size(dbn_data,2) >= 16
    qb_dbn = dbn_data(:,13:16);     % % [b1 b2 b3 b4]
end

xN = pos_dbn(:,1);
xE = pos_dbn(:,2);
D  = pos_dbn(:,3);                  % NED: Down positivo para baixo

%% ========== Converter Down NED para altitude positiva para cima ==========
% Preferência: usar DBN_params, pois contém a condição inicial aplicada após
% reposicionar o X-Plane.

if exist('DBN_params', 'var') && isfield(DBN_params, 'altitude0') && isfield(DBN_params, 'pos0_ned')
    altitude0 = DBN_params.altitude0;
    D0 = DBN_params.pos0_ned(3);
    alt = altitude0 - (D - D0);
else
    % Se a posição DBN já estiver em NED absoluto, altitude = -D.
    alt = -D;
end

%% ========== Figura 1: Trajetória 3D ==========
figure('Name', 'Trajetória 3D DBN', 'Position', [50 100 800 600]);
plot3(xE, xN, alt, 'b-', 'LineWidth', 1.5);
hold on;

% Waypoints
if exist('WPs', 'var')
    plot3(WPs(:,2), WPs(:,1), WPs(:,3), 'rs', 'MarkerSize', 10, ...
          'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, WPs(i,3)+2, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end
end

% Início e fim
plot3(xE(1), xN(1), alt(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot3(xE(end), xN(end), alt(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);

xlabel('Leste (m)');
ylabel('Norte (m)');
zlabel('Altitude (m)');
title('Trajetória 3D da Aeronave - DBN');
legend('Trajetória DBN', 'Waypoints', 'Início', 'Fim', 'Location', 'best');
grid on;
axis equal;
```

```
view(30, 25);
hold off;

%% ========== Figura 2: Vista Superior ==========
figure('Name', 'Vista Superior DBN', 'Position', [900 100 700 600]);
plot(xE, xN, 'b-', 'LineWidth', 1.5);
hold on;

if exist('WPs', 'var')
    plot(WPs(:,2), WPs(:,1), 'rs', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end

    if exist('R_accept', 'var')
        th = linspace(0, 2*pi, 100);
        for i = 1:size(WPs, 1)
            plot(WPs(i,2) + R_accept*cos(th), WPs(i,1) + R_accept*sin(th), ...
                 'r--', 'LineWidth', 0.5);
        end
    end
end

plot(xE(1), xN(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot(xE(end), xN(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);
xlabel('Leste (m)');
ylabel('Norte (m)');
title('Vista Superior - DBN');
grid on;
axis equal;
hold off;

%% ========== Figura 3: Altitude vs Tempo ==========
figure('Name', 'Altitude - DBN', 'Position', [50 750 800 400]);
if ~isempty(t_dbn)
    plot(t_dbn, alt, 'b-', 'LineWidth', 1.5);
    xlabel('Tempo (s)');
else
    plot(alt, 'b-', 'LineWidth', 1.5);
    xlabel('Amostra');
end
hold on;

if exist('WPs', 'var')
    alt_wps = unique(WPs(:,3));
    for i = 1:length(alt_wps)
        yline(alt_wps(i), 'r--', sprintf('%.0f m', alt_wps(i)), ...
              'LineWidth', 0.8, 'LabelHorizontalAlignment', 'left');
    end
end

ylabel('Altitude (m)');
title('Altitude ao Longo do Voo - DBN');
grid on;
hold off;

%% ========== Figura 4: Ângulos de Euler ==========
figure('Name', 'Euler - DBN', 'Position', [900 750 800 400]);
if ~isempty(t_dbn)
    plot(t_dbn, rad2deg(euler_dbn(:,1)), 'r-', 'LineWidth', 1.2); hold on;
    plot(t_dbn, rad2deg(euler_dbn(:,2)), 'g-', 'LineWidth', 1.2);
    plot(t_dbn, rad2deg(euler_dbn(:,3)), 'b-', 'LineWidth', 1.2);
    xlabel('Tempo (s)');
else
    plot(rad2deg(euler_dbn(:,1)), 'r-', 'LineWidth', 1.2); hold on;
    plot(rad2deg(euler_dbn(:,2)), 'g-', 'LineWidth', 1.2);
    plot(rad2deg(euler_dbn(:,3)), 'b-', 'LineWidth', 1.2);
    xlabel('Amostra');
end
grid on;
ylabel('Euler (graus)');
legend('\phi roll', '\theta pitch', '\psi yaw', 'Location', 'best');
title('Ângulos de Euler - DBN');
hold off;

%% ========== Figura 5: Velocidades NED ==========
figure('Name', 'Velocidades NED - DBN', 'Position', [50 1200 800 400]);
if ~isempty(t_dbn)
    plot(t_dbn, vel_dbn(:,1), 'r-', 'LineWidth', 1.2); hold on;
    plot(t_dbn, vel_dbn(:,2), 'g-', 'LineWidth', 1.2);
    plot(t_dbn, vel_dbn(:,3), 'b-', 'LineWidth', 1.2);
    xlabel('Tempo (s)');
else
    plot(vel_dbn(:,1), 'r-', 'LineWidth', 1.2); hold on;
    plot(vel_dbn(:,2), 'g-', 'LineWidth', 1.2);
```

```
    plot(vel_dbn(:,3), 'b-', 'LineWidth', 1.2);
    xlabel('Amostra');
end
grid on;
ylabel('Velocidade NED (m/s)');
legend('v_N', 'v_E', 'v_D', 'Location', 'best');
title('Velocidades em NED - DBN');
hold off;

%% ========== Figura 6: Acelerações NED ==========
figure('Name', 'Acelerações NED - DBN', 'Position', [900 1200 800 400]);
if ~isempty(t_dbn)
    plot(t_dbn, acc_dbn(:,1), 'r-', 'LineWidth', 1.2); hold on;
    plot(t_dbn, acc_dbn(:,2), 'g-', 'LineWidth', 1.2);
    plot(t_dbn, acc_dbn(:,3), 'b-', 'LineWidth', 1.2);
    xlabel('Tempo (s)');
else
    plot(acc_dbn(:,1), 'r-', 'LineWidth', 1.2); hold on;
    plot(acc_dbn(:,2), 'g-', 'LineWidth', 1.2);
    plot(acc_dbn(:,3), 'b-', 'LineWidth', 1.2);
    xlabel('Amostra');
end
grid on;
ylabel('Aceleração NED (m/s²)');
legend('a_N', 'a_E', 'a_D', 'Location', 'best');
title('Acelerações em NED - DBN');
hold off;

%% ========== Estatísticas ==========
fprintf('\n--- Estatísticas do Voo - DADOS DO DBN ---\n');
fprintf('  Posição inicial: N=%.1f  E=%.1f  Alt=%.1f m\n', xN(1), xE(1), alt(1));
fprintf('  Posição final:   N=%.1f  E=%.1f  Alt=%.1f m\n', xN(end), xE(end), alt(end));
fprintf('  Altitude: min=%.2f  max=%.2f  média=%.2f m\n', min(alt), max(alt), mean(alt));
fprintf('  Distância total percorrida: %.1f m\n', ...
    sum(sqrt(diff(xN).^2 + diff(xE).^2 + diff(alt).^2)));

if exist('WPs', 'var')
    d_final = sqrt((xN(end)-WPs(end,1))^2 + (xE(end)-WPs(end,2))^2);
    fprintf('  Distância ao WP final: %.1f m\n', d_final);
end

fprintf('\n========== FIM DO PLOT 3D DBN ==========' );
fprintf('\n');
```

4.48. plots/plot3d_voo_xplane.m

Arquivo: plots/plot3d_voo_xplane.m

Extensão: .m

Conteúdo:

```
%% plot3d_voo.m - Plota trajetória 3D da aeronave
% Rodar APÓS a simulação terminar.
% Requer variável POS no workspace (To Workspace com [xN, xE, alt]).
%
% Se POS não existir, tenta extrair de 'out' ou dos estados.
%
% Uso: >> plot3d_voo

fprintf('\n========== PLOT 3D DO VOO A PARTIR DE DADOS DO XPLANE==========\n');
%% ========== Localizar dados de posição ==========
try
pos_data = out.XplaneSimulationData.signals.values;
t_pos = out.XplaneSimulationData.time;
xN = pos_data(:,9);
xE = pos_data(:,10);
alt = pos_data(:,4);
catch
    disp("Error: data not founded to plot")
end
%% ========== Figura 1: Trajetória 3D ==========
figure('Name', 'Trajetória 3D dados Xplane', 'Position', [50 100 800 600]);
plot3(xE, xN, alt, 'b-', 'LineWidth', 1.5);
hold on;

% Waypoints (apenas markers)
if exist('WPs', 'var')
    plot3(WPs(:,2), WPs(:,1), WPs(:,3), 'rs', 'MarkerSize', 10, ...
          'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, WPs(i,3)+2, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end
end

% Marcar início e fim
plot3(xE(1), xN(1), alt(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot3(xE(end), xN(end), alt(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);

xlabel('Leste (m)'); ylabel('Norte (m)'); zlabel('Altitude (m)');
title('Trajetória 3D da Aeronave - Xplane');
legend('Trajetória', 'Waypoints', 'Início', 'Fim', 'Location', 'best');
grid on; axis equal;
view(30, 25);
hold off;

%% ========== Figura 2: Vista Superior (Ground Track) ==========
figure('Name', 'Vista Superior Xplane', 'Position', [900 100 700 600]);
plot(xE, xN, 'b-', 'LineWidth', 1.5);
hold on;
if exist('WPs', 'var')
    plot(WPs(:,2), WPs(:,1), 'rs', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    for i = 1:size(WPs, 1)
        text(WPs(i,2)+5, WPs(i,1)+5, sprintf('WP%d', i), ...
             'FontSize', 9, 'FontWeight', 'bold', 'Color', 'r');
    end
    % Círculo de aceitação
    if exist('R_accept', 'var')
        th = linspace(0, 2*pi, 100);
        for i = 1:size(WPs, 1)
            plot(WPs(i,2) + R_accept*cos(th), WPs(i,1) + R_accept*sin(th), ...
                 'r--', 'LineWidth', 0.5);
        end
    end
end
plot(xE(1), xN(1), 'go', 'MarkerSize', 12, 'MarkerFaceColor', 'g');
plot(xE(end), xN(end), 'kx', 'MarkerSize', 12, 'LineWidth', 2);
xlabel('Leste (m)'); ylabel('Norte (m)');
title('Vista Superior (Xplane)');
grid on; axis equal;
hold off;

%% ========== Figura 3: Altitude vs Tempo ==========
figure('Name', 'Altitude - XPLANE', 'Position', [50 750 800 400]);
if ~isempty(t_pos)
    plot(t_pos, alt, 'b-', 'LineWidth', 1.5);
    xlabel('Tempo (s)');
else
    plot(alt, 'b-', 'LineWidth', 1.5);
```

```
    xlabel('Amostra');
end
hold on;
% Linhas de referência dos waypoints
if exist('WPs', 'var')
    alt_wps = unique(WPs(:,3));
    for i = 1:length(alt_wps)
        yline(alt_wps(i), 'r--', sprintf('%.0f m', alt_wps(i)), ...
               'LineWidth', 0.8, 'LabelHorizontalAlignment', 'left');
    end
end
ylabel('Altitude (m)');
title('Altitude ao Longo do Voo - Xplane');
grid on;
hold off;

%% ========== Estatísticas ==========
fprintf('\n--- Estatísticas do Voo - DADOS DO XPLANE---\n');
fprintf('  Posição inicial: N=%.1f  E=%.1f  Alt=%.1f m\n', xN(1), xE(1), alt(1));
fprintf('  Posição final:   N=%.1f  E=%.1f  Alt=%.1f m\n', xN(end), xE(end), alt(end));
fprintf('  Altitude: min=%.2f  max=%.2f  média=%.2f m\n', min(alt), max(alt), mean(alt));
fprintf('  Distância total percorrida: %.1f m\n', ...
    sum(sqrt(diff(xN).^2 + diff(xE).^2 + diff(alt).^2)));

if exist('WPs', 'var')
    % Distância ao último WP
    d_final = sqrt((xN(end)-WPs(end,1))^2 + (xE(end)-WPs(end,2))^2);
    fprintf('  Distância ao WP final: %.1f m\n', d_final);
end

fprintf('\n========== FIM DO PLOT 3D ==========\n');
```

4.49. plots/plot_compare_all.m

Arquivo: plots/plot_compare_all.m

Extensão: .m

Conteúdo:

```
%% plot_compare_all.m
% Comparação configurável entre XPlane, DBN e EKFs

fprintf('\n========== COMPARAÇÃO CONFIGURÁVEL DE NAVEGAÇÃO ==========\n');

%% ==================== ESCOLHA DOS MODELOS ====================

use_modelo      = false;
use_xplane_ref  = true;

use_dbn         = false;
use_dbn_em      = false;

use_ekf_di      = true;
use_ekf_di_em   = true;

use_ekf_indi    = true;
use_ekf_indi_em = true;

%% ==================== CORES ====================

colors.modelo      = [1.000 0.000 0.600];  % rosa
colors.xplane      = [0.000 0.500 0.000];  % verde escuro

colors.dbn         = [1.000 0.000 1.000];  % magenta
colors.dbn_em      = [0.500 0.500 0.500];  % cinza

colors.ekf_di      = [0.000 0.000 1.000];  % azul
colors.ekf_di_em   = [0.500 0.000 0.500];  % roxo

colors.ekf_indi    = [1.000 0.000 0.000];  % vermelho
colors.ekf_indi_em = [0.500 0.250 0.000];  % marrom

lineWidth = 1.4;

%% ==================== INTERVALOS SEM CORREÇÃO ====================

try
    range_time_without_correction_local = evalin('base','range_time_without_correction');
catch
    range_time_without_correction_local = [0 0];
end

if isempty(range_time_without_correction_local) || size(range_time_without_correction_local,2) ~= 2
    range_time_without_correction_local = [0 0];
end

%% ==================== WAYPOINTS ====================

try
    WPs_local = WPs;
catch
    WPs_local = [];
end

try
    R_accept_local = R_accept;
catch
    R_accept_local = [];
end

%% ==================== VERIFICAR OUT ====================

if ~evalin('base','exist(''out'',''var'')')
    error('Base workspace variable ''out'' not found. Load simulation data before running this script.');
end

out_local = evalin('base','out');

%% ==================== CARREGAR SÉRIES ====================

series = struct( ...
    'name', {}, ...
    'key', {}, ...
    't', {}, ...
    'N', {}, ...
    'E', {}, ...
    'alt', {}, ...
```

```
    'vel', {}, ...
    'VT', {}, ...
    'acc', {}, ...
    'euler', {}, ...
    'xhat', {}, ...
    'color', {} ...
);

%% Modelo matemático
if use_modelo
    try
        [data, t] = get_out_signal(out_local, {'Y'});

        s = new_series('Modelo', 'modelo', colors.modelo);
        s.t = t;
        s.N = data(:,10);
        s.E = data(:,11);
        s.alt = data(:,12);

        if size(data,2) >= 3
            s.VT = sqrt(data(:,1).^2 + data(:,2).^2 + data(:,3).^2);
        end

        series(end+1) = s;
    catch ME
        warning('Modelo out.Y não encontrado. Erro: %s', ME.message);
    end
end

%% XPlane
if use_xplane_ref
    try
        [data, t] = get_out_signal(out_local, {'XplaneSimulationData'});

        s = new_series('XPlane', 'xplane', colors.xplane);
        s.t = t;
        s.VT = data(:,1);
        s.alt = data(:,4);
        s.N = data(:,9);
        s.E = data(:,10);

        series(end+1) = s;
    catch ME
        warning('XPlane out.XplaneSimulationData não encontrado. Erro: %s', ME.message);
    end
end

%% DBN sem erro
if use_dbn
    try
        [data, t] = get_out_signal(out_local, {'DBN_Data','DBN_data','BDN_data'});

        s = read_dbn_format(data, t, 'DBN', 'dbn', colors.dbn);

        series(end+1) = s;
    catch ME
        warning('DBN não encontrado. Erro: %s', ME.message);
    end
end

%% DBN com erro
if use_dbn_em
    try
        [data, t] = get_out_signal(out_local, {'DBN_data_with_error','DBN_Data_with_error','BDN_data_with_error'});

        s = read_dbn_format(data, t, 'DBN EM', 'dbn_em', colors.dbn_em);

        series(end+1) = s;
    catch ME
        warning('DBN com erro não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto sem erro
if use_ekf_di
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_data','EKF_DI'});

        s = read_ekf_format(data, t, 'EKF DI', 'ekf_di', colors.ekf_di);

        series(end+1) = s;
    catch ME
        warning('EKF_DI não encontrado. Erro: %s', ME.message);
    end
end
```

```
%% EKF direto com erro
if use_ekf_di_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_EM_data','EKF_DI_EM'});

        s = read_ekf_format(data, t, 'EKF DI EM', 'ekf_di_em', colors.ekf_di_em);

        series(end+1) = s;
    catch ME
        warning('EKF_DI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto sem erro
if use_ekf_indi
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_data','EKF_INDI'});

        s = read_ekf_format(data, t, 'EKF INDI', 'ekf_indi', colors.ekf_indi);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto com erro
if use_ekf_indi_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_EM_data','EKF_INDI_EM'});

        s = read_ekf_format(data, t, 'EKF INDI EM', 'ekf_indi_em', colors.ekf_indi_em);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% Verificação

if isempty(series)
    error('Nenhuma série carregada.');
end

idx_ref = find(strcmp({series.key}, 'xplane'), 1);

if isempty(idx_ref)
    warning('XPlane não foi carregado. Erros contra referência não serão plotados.');
end

%% ==================== FIGURA ====================

figure('Name','Comparação Navegação', 'Position',[80 40 1650 950]);

%% 1 - Trajetória 3D

subplot(3,2,1)
hold on

for k = 1:numel(series)
    plot3(series(k).E, series(k).N, series(k).alt, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_waypoints_3d(WPs_local);

grid on
axis equal
xlabel('Leste [m]')
ylabel('Norte [m]')
zlabel('Altitude [m]')
title('Trajetória 3D')
legend(build_legend(series, WPs_local), 'Location','best')
view(30,25)
hold off

%% 2 - Vista superior

subplot(3,2,5)
hold on

for k = 1:numel(series)
    plot(series(k).E, series(k).N, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end
```

```
plot_waypoints_2d(WPs_local, R_accept_local);

grid on
axis equal
xlabel('Leste [m]')
ylabel('Norte [m]')
title('Vista Superior')
legend(build_legend(series, WPs_local), 'Location','best')
hold off

%% 3 - Altitude

subplot(3,2,3)
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).alt, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_altitude_waypoints(WPs_local);
plot_correction_windows(gca, range_time_without_correction_local, true);

grid on
xlabel('Tempo [s]')
ylabel('Altitude [m]')
title('Altitude')
legend({series.name}, 'Location','best')
hold off

%% 4 - Velocidade escalar

subplot(3,2,2)
hold on

for k = 1:numel(series)
    if ~isempty(series(k).VT)
        plot(series(k).t, series(k).VT, ...
            'Color', series(k).color, 'LineWidth', lineWidth);
    end
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('Velocidade [m/s]')
title('Velocidade Escalar')
legend(build_vt_legend(series), 'Location','best')
hold off

%% 5 - Erro horizontal vs XPlane

subplot(3,2,4)
hold on

if ~isempty(idx_ref)

    ref = series(idx_ref);

    for k = 1:numel(series)

        if k == idx_ref
            continue;
        end

        [tc, Nref, Nk] = align_by_time(ref.t, ref.N, series(k).t, series(k).N);
        [~,  Eref, Ek] = align_by_time(ref.t, ref.E, series(k).t, series(k).E);

        erro_h = sqrt((Nref - Nk).^2 + (Eref - Ek).^2);

        plot(tc, erro_h, ...
            'Color', series(k).color, 'LineWidth', lineWidth);
    end

    yline(0, 'k--', 'XPlane ref');
    plot_correction_windows(gca, range_time_without_correction_local, false);

    grid on
    xlabel('Tempo [s]')
    ylabel('Erro horizontal [m]')
    title('Erro Horizontal vs XPlane')
    legend(build_error_legend(series, idx_ref), 'Location','best')
end

hold off
```

```
%% 6 - Erro altitude vs XPlane

subplot(3,2,6)
hold on

if ~isempty(idx_ref)

    ref = series(idx_ref);

    for k = 1:numel(series)

        if k == idx_ref
            continue;
        end

        [tc, alt_ref, alt_k] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

        erro_alt = alt_ref - alt_k;

        plot(tc, erro_alt, ...
            'Color', series(k).color, 'LineWidth', lineWidth);
    end

    yline(0, 'k--', 'XPlane ref');
    plot_correction_windows(gca, range_time_without_correction_local, false);

    grid on
    xlabel('Tempo [s]')
    ylabel('Erro altitude [m]')
    title('Erro de Altitude vs XPlane')
    legend(build_error_legend(series, idx_ref), 'Location','best')
end

hold off

sgtitle('XPlane x EKFs')

%% ==================== ESTATÍSTICAS ====================

fprintf('\n--- Séries carregadas ---\n');

for k = 1:numel(series)
    fprintf('%-12s | N0=%8.2f E0=%8.2f Alt0=%8.2f | Nf=%8.2f Ef=%8.2f Altf=%8.2f\n', ...
        series(k).name, ...
        series(k).N(1), series(k).E(1), series(k).alt(1), ...
        series(k).N(end), series(k).E(end), series(k).alt(end));
end

fprintf('\n========== FIM DA COMPARAÇÃO ==========\n');

%% ========================================================================
% FUNÇÕES LOCAIS
%% ========================================================================

function s = new_series(name, key, color)

    s.name = name;
    s.key = key;
    s.t = [];
    s.N = [];
    s.E = [];
    s.alt = [];
    s.vel = [];
    s.VT = [];
    s.acc = [];
    s.euler = [];
    s.xhat = [];
    s.color = color;

end

function s = read_dbn_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    s.euler = data(:,1:3);
    s.vel   = data(:,4:6);

    pos = data(:,7:9);

    s.N = pos(:,1);
    s.E = pos(:,2);
    s.alt = -pos(:,3);
```

```
    if size(data,2) >= 12
        s.acc = data(:,10:12);
    end

    s.VT = sqrt(s.vel(:,1).^2 + s.vel(:,2).^2 + s.vel(:,3).^2);

end

function s = read_ekf_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % Novo formato dos EKFs:
    % 1:3    pos_out   = [N E altitude]
    % 4:6    euler_out
    % 7:9    vel_out
    % 10:12  acc_n_out
    % 13:33  xhat_out

    pos = data(:,1:3);

    s.N = pos(:,1);
    s.E = pos(:,2);
    s.alt = pos(:,3);

    s.euler = data(:,4:6);
    s.vel   = data(:,7:9);
    s.acc   = data(:,10:12);

    s.VT = sqrt(s.vel(:,1).^2 + s.vel(:,2).^2 + s.vel(:,3).^2);

    if size(data,2) >= 33
        s.xhat = data(:,13:33);
    end

end

function [data, t] = get_out_signal(out_local, names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = out_local.(name_i);
        catch
            try
                sig = out_local.get(name_i);
            catch
                continue;
            end
        end

        try
            if isstruct(sig) && isfield(sig,'signals') && isfield(sig,'time')
                data = sig.signals.values;
                t = sig.time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'timeseries')
                data = sig.Data;
                t = sig.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'Simulink.SimulationData.Signal')
                data = sig.Values.Data;
                t = sig.Values.Time;
```

```
                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

        catch
        end
    end

    error('Nenhum dos sinais solicitados foi encontrado em out.');

end

function [tc, yref_i, y_i] = align_by_time(tref, yref, t, y)

    tref = tref(:);
    t = t(:);
    yref = yref(:);
    y = y(:);

    t0 = max(tref(1), t(1));
    tf = min(tref(end), t(end));

    N = min([length(tref), length(t), 5000]);

    tc = linspace(t0, tf, N).';

    yref_i = interp1(tref, yref, tc, 'linear');
    y_i    = interp1(t, y, tc, 'linear');

end

function labels = build_legend(series, WPs_local)

    labels = cell(1,numel(series));

    for k = 1:numel(series)
        labels{k} = series(k).name;
    end

    if ~isempty(WPs_local)
        labels{end+1} = 'Waypoints';
    end

end

function labels = build_vt_legend(series)

    labels = {};

    for k = 1:numel(series)

        if ~isempty(series(k).VT)

            if strcmp(series(k).key, 'xplane')
                labels{end+1} = 'XPlane true airspeed'; %#ok<AGROW>
            else
                labels{end+1} = series(k).name; %#ok<AGROW>
            end
        end
    end

end

function labels = build_error_legend(series, idx_ref)

    labels = {};

    for k = 1:numel(series)

        if k ~= idx_ref
            labels{end+1} = series(k).name; %#ok<AGROW>
        end
    end

end

function plot_waypoints_3d(WPs_local)

    if ~isempty(WPs_local)
        plot3(WPs_local(:,2), WPs_local(:,1), WPs_local(:,3), ...
            'ks', 'MarkerSize', 8, 'MarkerFaceColor', 'y');
```

```
    end

end

function plot_waypoints_2d(WPs_local, R_accept_local)

    if ~isempty(WPs_local)

        plot(WPs_local(:,2), WPs_local(:,1), ...
            'ks', 'MarkerSize', 8, 'MarkerFaceColor', 'y');

        if ~isempty(R_accept_local)

            th = linspace(0, 2*pi, 100);

            for j = 1:size(WPs_local,1)
                plot(WPs_local(j,2) + R_accept_local*cos(th), ...
                     WPs_local(j,1) + R_accept_local*sin(th), ...
                     'k--', 'LineWidth', 0.5);
            end
        end
    end

end

function plot_altitude_waypoints(WPs_local)

    if ~isempty(WPs_local)

        alt_wps = unique(WPs_local(:,3));

        for j = 1:length(alt_wps)
            yline(alt_wps(j), 'k--', sprintf('%.0f m', alt_wps(j)), ...
                'LineWidth', 0.6, 'LabelHorizontalAlignment', 'left');
        end
    end

end

function plot_correction_windows(ax, range_no_corr, show_labels, label_y_value)

    if nargin < 4
        label_y_value = [];
    end

    if isempty(range_no_corr) || size(range_no_corr,2) ~= 2
        return;
    end

    axes(ax); %#ok<LAXES>

    yl = ylim(ax);
    y_span = yl(2) - yl(1);

    if y_span <= 0
        y_span = 1;
    end

    %% Calcula posicao vertical do texto
    if ~isempty(label_y_value)
        label_y = label_y_value;
    else
        line_handles = findobj(ax, 'Type', 'line');

        y_all = [];

        for ii = 1:numel(line_handles)
            try
                y_i = line_handles(ii).YData;
                y_i = y_i(:);
                y_i = y_i(isfinite(y_i));

                if ~isempty(y_i)
                    y_all = [y_all; y_i]; %#ok<AGROW>
                end
            catch
            end
        end

        if ~isempty(y_all)
            label_y = 0.5 * (min(y_all) + max(y_all));
        else
            label_y = 0.5 * (yl(1) + yl(2));
        end
    end
```

```
    %% Plota janelas sem correcao
    for kk = 1:size(range_no_corr,1)

        ti = range_no_corr(kk,1);
        tf = range_no_corr(kk,2);

        if ti == 0 && tf == 0
            continue;
        end

        if tf <= ti
            continue;
        end

        hp = patch(ax, ...
            [ti tf tf ti], ...
            [yl(1) yl(1) yl(2) yl(2)], ...
            [0.85 0.85 0.85], ...
            'FaceAlpha', 0.22, ...
            'EdgeColor', 'none', ...
            'HandleVisibility', 'off');

        try
            uistack(hp, 'bottom');
        catch
        end

        xline(ax, ti, 'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
        xline(ax, tf, 'k-',  'LineWidth', 1.0, 'HandleVisibility', 'off');

        if show_labels
            text(ax, ...
                (ti + tf)/2, label_y, ...
                sprintf('sem GPS e/ou Yaw (Psi) %d', kk), ...
                'HorizontalAlignment', 'center', ...
                'VerticalAlignment', 'middle', ...
                'FontSize', 8, ...
                'Color', [0.15 0.15 0.15], ...
                'BackgroundColor', [1 1 1], ...
                'Margin', 1, ...
                'HandleVisibility', 'off');
        end
    end

    ylim(ax, yl);

end
```

4.50. plots/plot_compare_all2.m

Arquivo: plots/plot_compare_all2.m

Extensão: .m

Conteúdo:

```
%% plot_compare_all2.m
% Comparação de Euler e MSE de trajetória:
%   XPlane referência
%   EKF Direto sem erro
%   EKF Direto com erro
%   EKF Indireto sem erro
%   EKF Indireto com erro

fprintf('\n========== COMPARAÇÃO EULER + MSE DE TRAJETÓRIA ==========\n');

%% ==================== ESCOLHA DOS MODELOS ====================

use_xplane_ref  = true;

use_ekf_di      = true;
use_ekf_di_em   = false;

use_ekf_indi    = true;
use_ekf_indi_em = false;

%% ==================== CORES ====================

colors.xplane      = [0.000 0.500 0.000];  % verde escuro
colors.ekf_di      = [0.000 0.000 1.000];  % azul
colors.ekf_di_em   = [0.500 0.000 0.500];  % roxo
colors.ekf_indi    = [1.000 0.000 0.000];  % vermelho
colors.ekf_indi_em = [0.500 0.250 0.000];  % marrom

lineWidth = 1.4;

%% ==================== CARREGAR SÉRIES ====================

series = struct( ...
    'name', {}, ...
    'key', {}, ...
    't', {}, ...
    'N', {}, ...
    'E', {}, ...
    'alt', {}, ...
    'phi', {}, ...
    'theta', {}, ...
    'psi', {}, ...
    'color', {} ...
);

%% XPlane referência
if use_xplane_ref
    try
        data = out.XplaneSimulationData.signals.values;
        t = out.XplaneSimulationData.time;

        s = new_series('XPlane', 'xplane', colors.xplane);

        s.t = t;

        % XPlane:
        % phi   -> coluna 5
        % theta -> coluna 2
        % psi   -> coluna 7
        s.phi   = data(:,5);
        s.theta = data(:,2);
        s.psi   = data(:,7);

        % posição:
        % altitude -> coluna 4
        % Norte    -> coluna 9
        % Leste    -> coluna 10
        s.alt = data(:,4);
        s.N   = data(:,9);
        s.E   = data(:,10);

        series(end+1) = s;
    catch
        warning('XPlane out.XplaneSimulationData não encontrado.');
    end
end

%% EKF Direto sem erro
```

```
if use_ekf_di
    try
        data = get_first_available({'EKF_DI_data','EKF_DI'});
        t = get_time_first_available({'EKF_DI_data','EKF_DI'});

        s = read_ekf_format(data, t, 'EKF DI', 'ekf_di', colors.ekf_di);

        series(end+1) = s;
    catch
        warning('EKF_DI não encontrado.');
    end
end

%% EKF Direto com erro
if use_ekf_di_em
    try
        data = get_first_available({'EKF_DI_EM_data','EKF_DI_EM'});
        t = get_time_first_available({'EKF_DI_EM_data','EKF_DI_EM'});

        s = read_ekf_format(data, t, 'EKF DI EM', 'ekf_di_em', colors.ekf_di_em);

        series(end+1) = s;
    catch
        warning('EKF_DI_EM não encontrado.');
    end
end

%% EKF Indireto sem erro
if use_ekf_indi
    try
        data = get_first_available({'EKF_INDI_data','EKF_INDI'});
        t = get_time_first_available({'EKF_INDI_data','EKF_INDI'});

        s = read_ekf_format(data, t, 'EKF INDI', 'ekf_indi', colors.ekf_indi);

        series(end+1) = s;
    catch
        warning('EKF_INDI não encontrado.');
    end
end

%% EKF Indireto com erro
if use_ekf_indi_em
    try
        data = get_first_available({'EKF_INDI_EM_data','EKF_INDI_EM'});
        t = get_time_first_available({'EKF_INDI_EM_data','EKF_INDI_EM'});

        s = read_ekf_format(data, t, 'EKF INDI EM', 'ekf_indi_em', colors.ekf_indi_em);

        series(end+1) = s;
    catch
        warning('EKF_INDI_EM não encontrado.');
    end
end

%% Verificação

if isempty(series)
    error('Nenhuma série carregada.');
end

idx_ref = find(strcmp({series.key}, 'xplane'), 1);

if isempty(idx_ref)
    error('XPlane precisa estar carregado como referência.');
end

ref = series(idx_ref);

%% ==================== FIGURA ====================

figure('Name','Euler e MSE de Trajetória', 'Position',[80 40 1650 950]);

%% 1 - Roll

subplot(3,2,1)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).phi), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

grid on
xlabel('Tempo [s]')
ylabel('\phi roll [deg]')
```

```
title('Roll')
legend({series.name}, 'Location','best')
hold off

%% 2 - Pitch

subplot(3,2,3)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).theta), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

grid on
xlabel('Tempo [s]')
ylabel('\theta pitch [deg]')
title('Pitch')
legend({series.name}, 'Location','best')
hold off

%% 3 - Yaw

subplot(3,2,5)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).psi), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

grid on
xlabel('Tempo [s]')
ylabel('\psi yaw [deg]')
title('Yaw')
legend({series.name}, 'Location','best')
hold off

%% 4 - MSE horizontal acumulado

subplot(3,2,6)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, Nref, Nk] = align_by_time(ref.t, ref.N, series(k).t, series(k).N);
    [~,  Eref, Ek] = align_by_time(ref.t, ref.E, series(k).t, series(k).E);

    e_h2 = (Nref - Nk).^2 + (Eref - Ek).^2;
    mse_h = cumulative_mean(e_h2);

    plot(tc, mse_h, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

grid on
xlabel('Tempo [s]')
ylabel('MSE horizontal [m²]')
title('MSE Horizontal Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 5 - MSE altitude acumulado

subplot(3,2,4)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, alt_ref, alt_k] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_alt2 = (alt_ref - alt_k).^2;
    mse_alt = cumulative_mean(e_alt2);

    plot(tc, mse_alt, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end
```

```
grid on
xlabel('Tempo [s]')
ylabel('MSE altitude [m²]')
title('MSE de Altitude Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 6 - MSE 3D acumulado

subplot(3,2,2)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, Nref, Nk]     = align_by_time(ref.t, ref.N,   series(k).t, series(k).N);
    [~,  Eref, Ek]     = align_by_time(ref.t, ref.E,   series(k).t, series(k).E);
    [~,  altref, altk] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_3d2 = (Nref - Nk).^2 + (Eref - Ek).^2 + (altref - altk).^2;
    mse_3d = cumulative_mean(e_3d2);

    plot(tc, mse_3d, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

grid on
xlabel('Tempo [s]')
ylabel('MSE 3D [m²]')
title('MSE 3D Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

sgtitle('Euler e Erro Médio Quadrático - XPlane x EKFs')

%% ==================== ESTATÍSTICAS ====================

fprintf('\n--- Métricas finais vs XPlane ---\n');

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [~, Nref, Nk]       = align_by_time(ref.t, ref.N,   series(k).t, series(k).N);
    [~, Eref, Ek]       = align_by_time(ref.t, ref.E,   series(k).t, series(k).E);
    [~, altref, altk]   = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_h2   = (Nref - Nk).^2 + (Eref - Ek).^2;
    e_alt2 = (altref - altk).^2;
    e_3d2  = e_h2 + e_alt2;

    mse_h_final   = mean(e_h2);
    mse_alt_final = mean(e_alt2);
    mse_3d_final  = mean(e_3d2);

    rmse_h_final   = sqrt(mse_h_final);
    rmse_alt_final = sqrt(mse_alt_final);
    rmse_3d_final  = sqrt(mse_3d_final);

    fprintf('%-12s | MSE H=%.3f m² | RMSE H=%.3f m | MSE Alt=%.3f m² | RMSE Alt=%.3f m | MSE 3D=%.3f m² | RMSE
3D=%.3f m\n', ...
        series(k).name, ...
        mse_h_final, rmse_h_final, ...
        mse_alt_final, rmse_alt_final, ...
        mse_3d_final, rmse_3d_final);
end

fprintf('\n========== FIM DO PLOT_COMPARE_ALL2 ==========\n');

%% ========================================================================
% FUNÇÕES LOCAIS
%% ========================================================================

function s = new_series(name, key, color)

    s.name = name;
    s.key = key;
    s.t = [];
    s.N = [];
    s.E = [];
    s.alt = [];
```

```
    s.phi = [];
    s.theta = [];
    s.psi = [];
    s.color = color;

end

function s = read_ekf_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % EKF:
    % 1:3    pos_out   = [N E altitude]
    % 4:6    euler_out = [phi theta psi]
    % 7:9    vel_out
    % 10:12  acc_n_out
    % 13:33  xhat_out

    s.N   = data(:,1);
    s.E   = data(:,2);
    s.alt = data(:,3);

    s.phi   = data(:,4);
    s.theta = data(:,5);
    s.psi   = data(:,6);

end

function data = get_first_available(names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = evalin('base', ['out.' name_i]);
            data = sig.signals.values;
            return;
        catch
        end
    end

    error('Nenhum dos sinais solicitados foi encontrado.');

end

function t = get_time_first_available(names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = evalin('base', ['out.' name_i]);
            t = sig.time;
            return;
        catch
        end
    end

    error('Nenhum tempo encontrado.');

end

function [tc, yref_i, y_i] = align_by_time(tref, yref, t, y)

    tref = tref(:);
    t = t(:);
    yref = yref(:);
    y = y(:);

    t0 = max(tref(1), t(1));
    tf = min(tref(end), t(end));

    N = min([length(tref), length(t), 5000]);

    tc = linspace(t0, tf, N).';

    yref_i = interp1(tref, yref, tc, 'linear');
    y_i    = interp1(t, y, tc, 'linear');

end

function mse = cumulative_mean(x)
```

```
    n = (1:length(x)).';
    mse = cumsum(x) ./ n;

end

function labels = build_error_legend(series, idx_ref)

    labels = {};

    for k = 1:numel(series)

        if k ~= idx_ref
            labels{end+1} = series(k).name; %#ok<AGROW>
        end
    end

end
```

4.51. plots/plot_compare_all3.m

Arquivo: plots/plot_compare_all3.m

Extensão: .m

Conteúdo:

```
%% plot_compare_all3.m
% Comparação compacta:
%   Altitude
%   Vista superior
%   MSE 3D acumulado
%   MSE altitude acumulado
%   MSE horizontal acumulado

fprintf('\n========== COMPARAÇÃO ALL3: ALTITUDE, TRAJETÓRIA E MSE ==========\n');

%% ==================== ESCOLHA DOS MODELOS ====================

use_xplane_ref  = true;

use_ekf_di      = true;
use_ekf_di_em   = true;

use_ekf_indi    = true;
use_ekf_indi_em = true;

use_dbn         = false;
use_dbn_em      = false;

%% ==================== CORES ====================

colors.xplane      = [0.000 0.500 0.000];  % verde escuro

colors.dbn         = [1.000 0.000 1.000];  % magenta
colors.dbn_em      = [0.500 0.500 0.500];  % cinza

colors.ekf_di      = [0.000 0.000 1.000];  % azul
colors.ekf_di_em   = [0.500 0.000 0.500];  % roxo

colors.ekf_indi    = [1.000 0.000 0.000];  % vermelho
colors.ekf_indi_em = [0.500 0.250 0.000];  % marrom

lineWidth = 1.4;

%% ==================== INTERVALOS SEM CORREÇÃO ====================

try
    range_time_without_correction_local = evalin('base','range_time_without_correction');
catch
    range_time_without_correction_local = [0 0];
end

if isempty(range_time_without_correction_local) || size(range_time_without_correction_local,2) ~= 2
    range_time_without_correction_local = [0 0];
end

%% ==================== WAYPOINTS ====================

try
    WPs_local = WPs;
catch
    WPs_local = [];
end

try
    R_accept_local = R_accept;
catch
    R_accept_local = [];
end

%% ==================== VERIFICAR OUT ====================

if ~evalin('base','exist(''out'',''var'')')
    error('Base workspace variable ''out'' not found. Load simulation data before running this script.');
end

out_local = evalin('base','out');

%% ==================== CARREGAR SÉRIES ====================

series = struct( ...
    'name', {}, ...
    'key', {}, ...
    't', {}, ...
```

```
    'N', {}, ...
    'E', {}, ...
    'alt', {}, ...
    'vel', {}, ...
    'VT', {}, ...
    'euler', {}, ...
    'color', {} ...
);

%% XPlane referência
if use_xplane_ref
    try
        [data, t] = get_out_signal(out_local, {'XplaneSimulationData'});

        s = new_series('XPlane', 'xplane', colors.xplane);

        s.t = t;
        s.VT = data(:,1);
        s.alt = data(:,4);
        s.N = data(:,9);
        s.E = data(:,10);

        s.euler = [data(:,5), data(:,2), data(:,7)];

        series(end+1) = s;
    catch ME
        warning('XPlane out.XplaneSimulationData não encontrado. Erro: %s', ME.message);
    end
end

%% DBN sem erro
if use_dbn
    try
        % teste nomes diferentes da mesma variavel
        [data, t] = get_out_signal(out_local, {'DBN_Data','DBN_data','BDN_data'});

        s = read_dbn_format(data, t, 'DBN', 'dbn', colors.dbn);

        series(end+1) = s;
    catch ME
        warning('DBN não encontrado. Erro: %s', ME.message);
    end
end

%% DBN com erro
if use_dbn_em
    try
        [data, t] = get_out_signal(out_local, {'DBN_data_with_error','DBN_Data_with_error','BDN_data_with_error'});

        s = read_dbn_format(data, t, 'DBN EM', 'dbn_em', colors.dbn_em);

        series(end+1) = s;
    catch ME
        warning('DBN com erro não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto sem erro
if use_ekf_di
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_data','EKF_DI'});

        s = read_ekf_format(data, t, 'EKF DI', 'ekf_di', colors.ekf_di);

        series(end+1) = s;
    catch ME
        warning('EKF_DI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto com erro
if use_ekf_di_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_EM_data','EKF_DI_EM'});

        s = read_ekf_format(data, t, 'EKF DI EM', 'ekf_di_em', colors.ekf_di_em);

        series(end+1) = s;
    catch ME
        warning('EKF_DI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto sem erro
if use_ekf_indi
    try
```

```
        [data, t] = get_out_signal(out_local, {'EKF_INDI_data','EKF_INDI'});

        s = read_ekf_format(data, t, 'EKF INDI', 'ekf_indi', colors.ekf_indi);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto com erro
if use_ekf_indi_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_EM_data','EKF_INDI_EM'});

        s = read_ekf_format(data, t, 'EKF INDI EM', 'ekf_indi_em', colors.ekf_indi_em);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% Verificação

if isempty(series)
    error('Nenhuma série carregada.');
end

idx_ref = find(strcmp({series.key}, 'xplane'), 1);

if isempty(idx_ref)
    error('XPlane precisa estar carregado como referência.');
end

ref = series(idx_ref);

%% ==================== FIGURA ====================

figure('Name','Comparação ALL3 - Altitude, Vista Superior e MSE', ...
       'Position',[80 40 1650 950]);

%% 1 - Altitude

subplot(3,2,1)
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).alt, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_altitude_waypoints(WPs_local);
plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('Altitude [m]')
title('Altitude')
legend({series.name}, 'Location','best')
hold off

%% 2 - MSE 3D acumulado

subplot(3,2,2)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, Nref, Nk]     = align_by_time(ref.t, ref.N,   series(k).t, series(k).N);
    [~,  Eref, Ek]     = align_by_time(ref.t, ref.E,   series(k).t, series(k).E);
    [~,  altref, altk] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_3d2 = (Nref - Nk).^2 + (Eref - Ek).^2 + (altref - altk).^2;
    mse_3d = cumulative_mean(e_3d2);

    plot(tc, mse_3d, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end
plot_correction_windows(gca, range_time_without_correction_local, true);
grid on
xlabel('Tempo [s]')
```

```
ylabel('MSE 3D [m^2]')
title('MSE 3D Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 3 - Vista superior em subplot(3,2,[3 5])

subplot(3,2,[3 5])
hold on

for k = 1:numel(series)
    plot(series(k).E, series(k).N, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_waypoints_2d(WPs_local, R_accept_local);

grid on
axis equal
xlabel('Leste [m]')
ylabel('Norte [m]')
title('Vista Superior')
legend(build_legend(series, WPs_local), 'Location','best')
hold off

%% 4 - MSE altitude acumulado

subplot(3,2,4)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, alt_ref, alt_k] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_alt2 = (alt_ref - alt_k).^2;
    mse_alt = cumulative_mean(e_alt2);

    plot(tc, mse_alt, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);
grid on
xlabel('Tempo [s]')
ylabel('MSE altitude [m^2]')
title('MSE de Altitude Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 5 - MSE horizontal acumulado

subplot(3,2,6)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, Nref, Nk] = align_by_time(ref.t, ref.N, series(k).t, series(k).N);
    [~,  Eref, Ek] = align_by_time(ref.t, ref.E, series(k).t, series(k).E);

    e_h2 = (Nref - Nk).^2 + (Eref - Ek).^2;
    mse_h = cumulative_mean(e_h2);

    plot(tc, mse_h, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);
grid on
xlabel('Tempo [s]')
ylabel('MSE horizontal [m^2]')
title('MSE Horizontal Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% Título geral da figura

annotation('textbox', [0 0.955 1 0.035], ...
    'String', 'Altitude, Vista Superior e MSE - XPlane x Estimadores', ...
```

```
    'HorizontalAlignment', 'center', ...
    'VerticalAlignment', 'middle', ...
    'FontWeight', 'bold', ...
    'FontSize', 14, ...
    'EdgeColor', 'none');

%% ==================== ESTATÍSTICAS ====================

fprintf('\n--- Métricas finais vs XPlane ---\n');

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [~, Nref, Nk]     = align_by_time(ref.t, ref.N,   series(k).t, series(k).N);
    [~, Eref, Ek]     = align_by_time(ref.t, ref.E,   series(k).t, series(k).E);
    [~, altref, altk] = align_by_time(ref.t, ref.alt, series(k).t, series(k).alt);

    e_h2   = (Nref - Nk).^2 + (Eref - Ek).^2;
    e_alt2 = (altref - altk).^2;
    e_3d2  = e_h2 + e_alt2;

    mse_h_final   = mean(e_h2);
    mse_alt_final = mean(e_alt2);
    mse_3d_final  = mean(e_3d2);

    rmse_h_final   = sqrt(mse_h_final);
    rmse_alt_final = sqrt(mse_alt_final);
    rmse_3d_final  = sqrt(mse_3d_final);

    fprintf('%-12s | MSE H=%.3f m^2 | RMSE H=%.3f m | MSE Alt=%.3f m^2 | RMSE Alt=%.3f m | MSE 3D=%.3f m^2 | RMSE
3D=%.3f m\n', ...
        series(k).name, ...
        mse_h_final, rmse_h_final, ...
        mse_alt_final, rmse_alt_final, ...
        mse_3d_final, rmse_3d_final);
end

fprintf('\n========== FIM DO PLOT_COMPARE_ALL3 ==========\n');

%% ========================================================================
% FUNÇÕES LOCAIS
%% ========================================================================
%% new_series
function s = new_series(name, key, color)

    s.name = name;
    s.key = key;
    s.t = [];
    s.N = [];
    s.E = [];
    s.alt = [];
    s.vel = [];
    s.VT = [];
    s.euler = [];
    s.color = color;

end
%% read_dbn_format
function s = read_dbn_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    s.euler = data(:,1:3);
    s.vel   = data(:,4:6);

    pos = data(:,7:9);

    s.N = pos(:,1);
    s.E = pos(:,2);
    s.alt = -pos(:,3);

    if size(data,2) >= 12
        s.acc = data(:,10:12);
    end

    s.VT = sqrt(s.vel(:,1).^2 + s.vel(:,2).^2 + s.vel(:,3).^2);

end
%% read_ekf_format
function s = read_ekf_format(data, t, name, key, color)
```

```
    s = new_series(name, key, color);

    s.t = t;

    % EKF:
    % 1:3    pos_out   = [N E altitude]
    % 4:6    euler_out = [phi theta psi]
    % 7:9    vel_out
    % 10:12  acc_n_out
    % 13:33  xhat_out

    pos = data(:,1:3);

    s.N = pos(:,1);
    s.E = pos(:,2);
    s.alt = pos(:,3);

    s.euler = data(:,4:6);
    s.vel   = data(:,7:9);

    s.VT = sqrt(s.vel(:,1).^2 + s.vel(:,2).^2 + s.vel(:,3).^2);

end
%% get_out_signal
function [data, t] = get_out_signal(out_local, names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = out_local.(name_i);
        catch
            try
                sig = out_local.get(name_i);
            catch
                continue;
            end
        end

        try
            if isstruct(sig) && isfield(sig,'signals') && isfield(sig,'time')
                data = sig.signals.values;
                t = sig.time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'timeseries')
                data = sig.Data;
                t = sig.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'Simulink.SimulationData.Signal')
                data = sig.Values.Data;
                t = sig.Values.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

        catch
        end
    end

    error('Nenhum dos sinais solicitados foi encontrado em out.');
```

```
end
%% align_by_time
function [tc, yref_i, y_i] = align_by_time(tref, yref, t, y)

    tref = tref(:);
    t = t(:);
    yref = yref(:);
    y = y(:);

    t0 = max(tref(1), t(1));
    tf = min(tref(end), t(end));

    N = min([length(tref), length(t), 5000]);

    tc = linspace(t0, tf, N).';

    yref_i = interp1(tref, yref, tc, 'linear');
    y_i    = interp1(t, y, tc, 'linear');

end
%% cumulative_mean
function mse = cumulative_mean(x)

    n = (1:length(x)).';
    mse = cumsum(x) ./ n;

end
%% build_legend
function labels = build_legend(series, WPs_local)

    labels = cell(1,numel(series));

    for k = 1:numel(series)
        labels{k} = series(k).name;
    end

    if ~isempty(WPs_local)
        labels{end+1} = 'Waypoints';
    end

end
%% build_error_legend
function labels = build_error_legend(series, idx_ref)

    labels = {};

    for k = 1:numel(series)

        if k ~= idx_ref
            labels{end+1} = series(k).name; %#ok<AGROW>
        end
    end

end
%% plot_waypoints_2d
function plot_waypoints_2d(WPs_local, R_accept_local)

    if ~isempty(WPs_local)

        plot(WPs_local(:,2), WPs_local(:,1), ...
            'ks', 'MarkerSize', 8, 'MarkerFaceColor', 'y');

        if ~isempty(R_accept_local)

            th = linspace(0, 2*pi, 100);

            for j = 1:size(WPs_local,1)
                plot(WPs_local(j,2) + R_accept_local*cos(th), ...
                     WPs_local(j,1) + R_accept_local*sin(th), ...
                     'k--', 'LineWidth', 0.5);
            end
        end
    end

end
%% plot_altitude_waypoints
function plot_altitude_waypoints(WPs_local)

    if ~isempty(WPs_local)

        alt_wps = unique(WPs_local(:,3));

        for j = 1:length(alt_wps)
            yline(alt_wps(j), 'k--', sprintf('%.0f m', alt_wps(j)), ...
                'LineWidth', 0.6, 'LabelHorizontalAlignment', 'left');
        end
```

```
    end

end
%% plot_correction_windows
function plot_correction_windows(ax, range_no_corr, show_labels, label_y_value)

if nargin < 4
    label_y_value = [];
end

if isempty(range_no_corr) || size(range_no_corr,2) ~= 2
    return;
end

axes(ax);

yl = ylim(ax);
y_span = yl(2) - yl(1);

if y_span <= 0
    y_span = 1;
end

%% Calcula posição vertical do texto
if ~isempty(label_y_value)
    label_y = label_y_value;
else
    line_handles = findobj(ax, 'Type', 'line');

    y_all = [];

    for ii = 1:numel(line_handles)
        try
            y_i = line_handles(ii).YData;
            y_i = y_i(:);
            y_i = y_i(isfinite(y_i));

            if ~isempty(y_i)
                y_all = [y_all; y_i]; %#ok<AGROW>
            end
        catch
        end
    end

    if ~isempty(y_all)
        label_y = 0.5 * (min(y_all) + max(y_all));
    else
        label_y = 0.5 * (yl(1) + yl(2));
    end
end

%% Plota janelas sem correção
for kk = 1:size(range_no_corr,1)

    ti = range_no_corr(kk,1);
    tf = range_no_corr(kk,2);

    if ti == 0 && tf == 0
        continue;
    end

    if tf <= ti
        continue;
    end

    hp = patch(ax, ...
        [ti tf tf ti], ...
        [yl(1) yl(1) yl(2) yl(2)], ...
        [0.85 0.85 0.85], ...
        'FaceAlpha', 0.22, ...
        'EdgeColor', 'none', ...
        'HandleVisibility', 'off');

    try
        uistack(hp, 'bottom');
    catch
    end

    xline(ax, ti, 'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
    xline(ax, tf, 'k-',  'LineWidth', 1.0, 'HandleVisibility', 'off');

    if show_labels
        text(ax, ...
            (ti + tf)/2, label_y, ...
            sprintf('sem correção %d', kk), ...
            'HorizontalAlignment', 'center', ...
```

```
            'VerticalAlignment', 'middle', ...
            'FontSize', 8, ...
            'Color', [0.15 0.15 0.15], ...
            'BackgroundColor', [1 1 1], ...
            'Margin', 1, ...
            'HandleVisibility', 'off');
    end
end

ylim(ax, yl);

end
```

4.52. plots/plot_compare_all4.m

Arquivo: plots/plot_compare_all4.m

Extensão: .m

Conteúdo:

```
%% plot_compare_all4.m
% Comparação de angulos de Euler e MSE angular:
%   Roll, Pitch, Yaw
%   MSE Roll, MSE Pitch, MSE Yaw

fprintf('\n========== COMPARAÇÃO ALL4: EULER E MSE ANGULAR ==========\n');

%% ==================== ESCOLHA DOS MODELOS ====================

use_xplane_ref  = true;

use_dbn         = false;
use_dbn_em      = false;

use_ekf_di      = true;
use_ekf_di_em   = false;

use_ekf_indi    = true;
use_ekf_indi_em = false;

%% ==================== CORES ====================

colors.xplane      = [0.000 0.500 0.000];  % verde escuro

colors.dbn         = [1.000 0.000 1.000];  % magenta
colors.dbn_em      = [0.500 0.500 0.500];  % cinza

colors.ekf_di      = [0.000 0.000 1.000];  % azul
colors.ekf_di_em   = [0.500 0.000 0.500];  % roxo

colors.ekf_indi    = [1.000 0.000 0.000];  % vermelho
colors.ekf_indi_em = [0.500 0.250 0.000];  % marrom

lineWidth = 1.4;

%% ==================== INTERVALOS SEM CORREÇÃO ====================

try
    range_time_without_correction_local = evalin('base','range_time_without_correction');
catch
    range_time_without_correction_local = [0 0];
end

if isempty(range_time_without_correction_local) || size(range_time_without_correction_local,2) ~= 2
    range_time_without_correction_local = [0 0];
end

%% ==================== VERIFICAR OUT ====================

if ~evalin('base','exist(''out'',''var'')')
    error('Base workspace variable ''out'' not found. Load simulation data before running this script.');
end

out_local = evalin('base','out');

%% ==================== CARREGAR SÉRIES ====================

series = struct( ...
    'name', {}, ...
    'key', {}, ...
    't', {}, ...
    'phi', {}, ...
    'theta', {}, ...
    'psi', {}, ...
    'color', {} ...
);

%% XPlane referência
if use_xplane_ref
    try
        [data, t] = get_out_signal(out_local, {'XplaneSimulationData'});

        s = new_series('XPlane', 'xplane', colors.xplane);

        s.t = t;

        % XPlane:
        % phi   -> coluna 5
```

```
        % theta -> coluna 2
        % psi   -> coluna 7
        s.phi   = data(:,5);
        s.theta = data(:,2);
        s.psi   = data(:,7);

        series(end+1) = s;
    catch ME
        warning('XPlane out.XplaneSimulationData não encontrado. Erro: %s', ME.message);
    end
end

%% DBN sem erro
if use_dbn
    try
        [data, t] = get_out_signal(out_local, {'DBN_Data','DBN_data','BDN_data'});

        s = read_dbn_format(data, t, 'DBN', 'dbn', colors.dbn);

        series(end+1) = s;
    catch ME
        warning('DBN não encontrado. Erro: %s', ME.message);
    end
end

%% DBN com erro
if use_dbn_em
    try
        [data, t] = get_out_signal(out_local, {'DBN_data_with_error','DBN_Data_with_error','BDN_data_with_error'});

        s = read_dbn_format(data, t, 'DBN EM', 'dbn_em', colors.dbn_em);

        series(end+1) = s;
    catch ME
        warning('DBN com erro não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto sem erro
if use_ekf_di
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_data','EKF_DI'});

        s = read_ekf_format(data, t, 'EKF DI', 'ekf_di', colors.ekf_di);

        series(end+1) = s;
    catch ME
        warning('EKF_DI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto com erro
if use_ekf_di_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_EM_data','EKF_DI_EM'});

        s = read_ekf_format(data, t, 'EKF DI EM', 'ekf_di_em', colors.ekf_di_em);

        series(end+1) = s;
    catch ME
        warning('EKF_DI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto sem erro
if use_ekf_indi
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_data','EKF_INDI'});

        s = read_ekf_format(data, t, 'EKF INDI', 'ekf_indi', colors.ekf_indi);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto com erro
if use_ekf_indi_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_EM_data','EKF_INDI_EM'});

        s = read_ekf_format(data, t, 'EKF INDI EM', 'ekf_indi_em', colors.ekf_indi_em);

        series(end+1) = s;
    catch ME
```

```
        warning('EKF_INDI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% Verificação

if isempty(series)
    error('Nenhuma série carregada.');
end

idx_ref = find(strcmp({series.key}, 'xplane'), 1);

if isempty(idx_ref)
    error('XPlane precisa estar carregado como referência.');
end

ref = series(idx_ref);

%% ==================== FIGURA ====================

figure('Name','Comparação ALL4 - Euler e MSE Angular', ...
       'Position',[80 40 1650 950]);

%% 1 - Roll

subplot(3,2,1)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).phi), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('\phi roll [deg]')
title('Roll')
legend({series.name}, 'Location','best')
hold off

%% 2 - MSE Roll

subplot(3,2,2)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.phi, series(k).t, series(k).phi);

    err = wrapToPi_local(ref_i - y_i);
    mse_roll = cumulative_mean(err.^2);

    plot(tc, rad2deg(sqrt(mse_roll)), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, true);

grid on
xlabel('Tempo [s]')
ylabel('RMSE roll [deg]')
title('RMSE Roll Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 3 - Pitch

subplot(3,2,3)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).theta), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('\theta pitch [deg]')
```

```
title('Pitch')
legend({series.name}, 'Location','best')
hold off

%% 4 - MSE Pitch

subplot(3,2,4)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.theta, series(k).t, series(k).theta);

    err = wrapToPi_local(ref_i - y_i);
    mse_pitch = cumulative_mean(err.^2);

    plot(tc, rad2deg(sqrt(mse_pitch)), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('RMSE pitch [deg]')
title('RMSE Pitch Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 5 - Yaw

subplot(3,2,5)
hold on

for k = 1:numel(series)
    plot(series(k).t, rad2deg(series(k).psi), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('\psi yaw [deg]')
title('Yaw')
legend({series.name}, 'Location','best')
hold off

%% 6 - MSE Yaw

subplot(3,2,6)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.psi, series(k).t, series(k).psi);

    err = wrapToPi_local(ref_i - y_i);
    mse_yaw = cumulative_mean(err.^2);

    plot(tc, rad2deg(sqrt(mse_yaw)), ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('RMSE yaw [deg]')
title('RMSE Yaw Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

annotation('textbox', [0 0.955 1 0.035], ...
    'String', 'Euler e RMSE Angular - XPlane x Estimadores', ...
    'HorizontalAlignment', 'center', ...
    'VerticalAlignment', 'middle', ...
    'FontWeight', 'bold', ...
```

```
    'FontSize', 14, ...
    'EdgeColor', 'none');

%% ==================== ESTATÍSTICAS ====================

fprintf('\n--- Métricas finais angulares vs XPlane ---\n');

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [~, phi_ref, phi_k]     = align_by_time(ref.t, ref.phi,   series(k).t, series(k).phi);
    [~, theta_ref, theta_k] = align_by_time(ref.t, ref.theta, series(k).t, series(k).theta);
    [~, psi_ref, psi_k]     = align_by_time(ref.t, ref.psi,   series(k).t, series(k).psi);

    err_phi   = wrapToPi_local(phi_ref - phi_k);
    err_theta = wrapToPi_local(theta_ref - theta_k);
    err_psi   = wrapToPi_local(psi_ref - psi_k);

    rmse_phi   = rad2deg(sqrt(mean(err_phi.^2)));
    rmse_theta = rad2deg(sqrt(mean(err_theta.^2)));
    rmse_psi   = rad2deg(sqrt(mean(err_psi.^2)));

    fprintf('%-12s | RMSE roll=%.3f deg | RMSE pitch=%.3f deg | RMSE yaw=%.3f deg\n', ...
        series(k).name, rmse_phi, rmse_theta, rmse_psi);
end

fprintf('\n========== FIM DO PLOT_COMPARE_ALL4 ==========\n');

%% ========================================================================
% FUNÇÕES LOCAIS
%% ========================================================================

function s = new_series(name, key, color)

    s.name = name;
    s.key = key;
    s.t = [];
    s.phi = [];
    s.theta = [];
    s.psi = [];
    s.color = color;

end

function s = read_dbn_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % DBN:
    % 1:3 = [phi theta psi]
    s.phi   = data(:,1);
    s.theta = data(:,2);
    s.psi   = data(:,3);

end

function s = read_ekf_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % EKF:
    % 4:6 = [phi theta psi]
    s.phi   = data(:,4);
    s.theta = data(:,5);
    s.psi   = data(:,6);

end

function [data, t] = get_out_signal(out_local, names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = out_local.(name_i);
        catch
            try
                sig = out_local.get(name_i);
```

```
            catch
                continue;
            end
        end

        try
            if isstruct(sig) && isfield(sig,'signals') && isfield(sig,'time')
                data = sig.signals.values;
                t = sig.time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'timeseries')
                data = sig.Data;
                t = sig.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'Simulink.SimulationData.Signal')
                data = sig.Values.Data;
                t = sig.Values.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

        catch
        end
    end

    error('Nenhum dos sinais solicitados foi encontrado em out.');

end

function [tc, yref_i, y_i] = align_by_time(tref, yref, t, y)

    tref = tref(:);
    t = t(:);
    yref = yref(:);
    y = y(:);

    t0 = max(tref(1), t(1));
    tf = min(tref(end), t(end));

    N = min([length(tref), length(t), 5000]);

    tc = linspace(t0, tf, N).';

    yref_i = interp1(tref, yref, tc, 'linear');
    y_i    = interp1(t, y, tc, 'linear');

end

function mse = cumulative_mean(x)

    n = (1:length(x)).';
    mse = cumsum(x) ./ n;

end

function labels = build_error_legend(series, idx_ref)

    labels = {};

    for k = 1:numel(series)

        if k ~= idx_ref
```

```
            labels{end+1} = series(k).name; %#ok<AGROW>
        end
    end

end

function ang = wrapToPi_local(ang)

    ang = mod(ang + pi, 2*pi) - pi;

end

function plot_correction_windows(ax, range_no_corr, show_labels, label_y_value)

    if nargin < 4
        label_y_value = [];
    end

    if isempty(range_no_corr) || size(range_no_corr,2) ~= 2
        return;
    end

    axes(ax); %#ok<LAXES>

    yl = ylim(ax);
    y_span = yl(2) - yl(1);

    if y_span <= 0
        y_span = 1;
    end

    %% Calcula posição vertical do texto
    if ~isempty(label_y_value)
        label_y = label_y_value;
    else
        line_handles = findobj(ax, 'Type', 'line');

        y_all = [];

        for ii = 1:numel(line_handles)
            try
                y_i = line_handles(ii).YData;
                y_i = y_i(:);
                y_i = y_i(isfinite(y_i));

                if ~isempty(y_i)
                    y_all = [y_all; y_i]; %#ok<AGROW>
                end
            catch
            end
        end

        if ~isempty(y_all)
            label_y = 0.5 * (min(y_all) + max(y_all));
        else
            label_y = 0.5 * (yl(1) + yl(2));
        end
    end

    %% Plota janelas sem correção
    for kk = 1:size(range_no_corr,1)

        ti = range_no_corr(kk,1);
        tf = range_no_corr(kk,2);

        if ti == 0 && tf == 0
            continue;
        end

        if tf <= ti
            continue;
        end

        hp = patch(ax, ...
            [ti tf tf ti], ...
            [yl(1) yl(1) yl(2) yl(2)], ...
            [0.85 0.85 0.85], ...
            'FaceAlpha', 0.22, ...
            'EdgeColor', 'none', ...
            'HandleVisibility', 'off');

        try
            uistack(hp, 'bottom');
        catch
        end
```

```
        xline(ax, ti, 'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
        xline(ax, tf, 'k-',  'LineWidth', 1.0, 'HandleVisibility', 'off');

        if show_labels
            text(ax, ...
                (ti + tf)/2, label_y, ...
                sprintf('sem correção %d', kk), ...
                'HorizontalAlignment', 'center', ...
                'VerticalAlignment', 'middle', ...
                'FontSize', 8, ...
                'Color', [0.15 0.15 0.15], ...
                'BackgroundColor', [1 1 1], ...
                'Margin', 1, ...
                'HandleVisibility', 'off');
        end
    end

    ylim(ax, yl);

end
```

4.53. plots/plot_compare_all5.m

Arquivo: plots/plot_compare_all5.m

Extensão: .m

Conteúdo:

```
%% plot_compare_all5.m
% Comparação de velocidades:
%   V_N, V_E, V_D
%   Velocidade em Relacao ao Solo 3D - NED
%   MSE acumulado de cada velocidade
%   Erro comparativo de V_3D vs XPlane

fprintf('\n========== COMPARAÇÃO ALL5: VELOCIDADES E MSE ==========\n');

%% ==================== ESCOLHA DOS MODELOS ====================

use_xplane_ref  = true;

use_dbn         = false;
use_dbn_em      = false;

use_ekf_di      = true;
use_ekf_di_em   = false;

use_ekf_indi    = true;
use_ekf_indi_em = false;

%% ==================== CORES ====================

colors.xplane      = [0.000 0.500 0.000];  % verde escuro

colors.dbn         = [1.000 0.000 1.000];  % magenta
colors.dbn_em      = [0.500 0.500 0.500];  % cinza

colors.ekf_di      = [0.000 0.000 1.000];  % azul
colors.ekf_di_em   = [0.500 0.000 0.500];  % roxo

colors.ekf_indi    = [1.000 0.000 0.000];  % vermelho
colors.ekf_indi_em = [0.500 0.250 0.000];  % marrom

colors.xplane_vt   = [0.000 0.000 0.000];  % preto para VT XPlane

lineWidth = 1.4;

%% ==================== INTERVALOS SEM CORREÇÃO ====================

try
    range_time_without_correction_local = evalin('base','range_time_without_correction');
catch
    range_time_without_correction_local = [0 0];
end

if isempty(range_time_without_correction_local) || size(range_time_without_correction_local,2) ~= 2
    range_time_without_correction_local = [0 0];
end

%% ==================== VERIFICAR OUT ====================

if ~evalin('base','exist(''out'',''var'')')
    error('Base workspace variable ''out'' not found. Load simulation data before running this script.');
end

out_local = evalin('base','out');

%% ==================== CARREGAR SÉRIES ====================

series = struct( ...
    'name', {}, ...
    'key', {}, ...
    't', {}, ...
    'vN', {}, ...
    'vE', {}, ...
    'vD', {}, ...
    'V3D', {}, ...
    'VT_xplane', {}, ...
    'color', {} ...
);

%% XPlane referência
if use_xplane_ref
    try
        [data, t] = get_out_signal(out_local, {'XplaneSimulationData'});
```

```
        s = new_series('XPlane', 'xplane', colors.xplane);

        s.t = t;

        % XPlane:
        % 14: vN
        % 15: vE
        % 16: vD
        s.vN = data(:,14);
        s.vE = data(:,15);
        s.vD = data(:,16);

        s.V3D = sqrt(s.vN.^2 + s.vE.^2 + s.vD.^2);

        % VT do XPlane:
        % 1: true airspeed
        s.VT_xplane = data(:,1);

        series(end+1) = s;
    catch ME
        warning('XPlane out.XplaneSimulationData não encontrado. Erro: %s', ME.message);
    end
end

%% DBN sem erro
if use_dbn
    try
        [data, t] = get_out_signal(out_local, {'DBN_Data','DBN_data','BDN_data'});

        s = read_dbn_format(data, t, 'DBN', 'dbn', colors.dbn);

        series(end+1) = s;
    catch ME
        warning('DBN não encontrado. Erro: %s', ME.message);
    end
end

%% DBN com erro
if use_dbn_em
    try
        [data, t] = get_out_signal(out_local, {'DBN_data_with_error','DBN_Data_with_error','BDN_data_with_error'});

        s = read_dbn_format(data, t, 'DBN EM', 'dbn_em', colors.dbn_em);

        series(end+1) = s;
    catch ME
        warning('DBN com erro não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto sem erro
if use_ekf_di
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_data','EKF_DI'});

        s = read_ekf_format(data, t, 'EKF DI', 'ekf_di', colors.ekf_di);

        series(end+1) = s;
    catch ME
        warning('EKF_DI não encontrado. Erro: %s', ME.message);
    end
end

%% EKF direto com erro
if use_ekf_di_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_DI_EM_data','EKF_DI_EM'});

        s = read_ekf_format(data, t, 'EKF DI EM', 'ekf_di_em', colors.ekf_di_em);

        series(end+1) = s;
    catch ME
        warning('EKF_DI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% EKF indireto sem erro
if use_ekf_indi
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_data','EKF_INDI'});

        s = read_ekf_format(data, t, 'EKF INDI', 'ekf_indi', colors.ekf_indi);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI não encontrado. Erro: %s', ME.message);
```

```
    end
end

%% EKF indireto com erro
if use_ekf_indi_em
    try
        [data, t] = get_out_signal(out_local, {'EKF_INDI_EM_data','EKF_INDI_EM'});

        s = read_ekf_format(data, t, 'EKF INDI EM', 'ekf_indi_em', colors.ekf_indi_em);

        series(end+1) = s;
    catch ME
        warning('EKF_INDI_EM não encontrado. Erro: %s', ME.message);
    end
end

%% Verificação

if isempty(series)
    error('Nenhuma série carregada.');
end

idx_ref = find(strcmp({series.key}, 'xplane'), 1);

if isempty(idx_ref)
    error('XPlane precisa estar carregado como referência.');
end

ref = series(idx_ref);

%% ==================== FIGURA ====================

figure('Name','Comparação ALL5 - Velocidades e MSE', ...
       'Position',[80 40 1650 1050]);

%% 1 - Velocidade Norte

subplot(5,2,1)
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).vN, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('V_N [m/s]')
title('Velocidade Norte - V_N')
legend({series.name}, 'Location','best')
hold off

%% 2 - MSE V_N

subplot(5,2,2)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.vN, series(k).t, series(k).vN);

    e2 = (ref_i - y_i).^2;
    mse_vN = cumulative_mean(e2);

    plot(tc, mse_vN, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, true);

grid on
xlabel('Tempo [s]')
ylabel('MSE V_N [(m/s)^2]')
title('MSE V_N Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 3 - Velocidade Leste

subplot(5,2,3)
```

```
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).vE, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('V_E [m/s]')
title('Velocidade Leste - V_E')
legend({series.name}, 'Location','best')
hold off

%% 4 - MSE V_E

subplot(5,2,4)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.vE, series(k).t, series(k).vE);

    e2 = (ref_i - y_i).^2;
    mse_vE = cumulative_mean(e2);

    plot(tc, mse_vE, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('MSE V_E [(m/s)^2]')
title('MSE V_E Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 5 - Velocidade Down

subplot(5,2,5)
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).vD, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('V_D [m/s]')
title('Velocidade Down - V_D')
legend({series.name}, 'Location','best')
hold off

%% 6 - MSE V_D

subplot(5,2,6)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.vD, series(k).t, series(k).vD);

    e2 = (ref_i - y_i).^2;
    mse_vD = cumulative_mean(e2);

    plot(tc, mse_vD, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);
```

```
grid on
xlabel('Tempo [s]')
ylabel('MSE V_D [(m/s)^2]')
title('MSE V_D Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 7 - Velocidade em Relação ao Solo 3D - NED

subplot(5,2,7)
hold on

for k = 1:numel(series)
    plot(series(k).t, series(k).V3D, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

if ~isempty(ref.VT_xplane)
    plot(ref.t, ref.VT_xplane, '--', ...
        'Color', colors.xplane_vt, ...
        'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('V_{3D} [m/s]')
title('Velocidade em Relação ao Solo 3D - NED')

legend_labels = {series.name};
legend_labels{end+1} = 'VT XPlane - true airspeed';

legend(legend_labels, 'Location','best')
hold off

%% 8 - MSE V_3D

subplot(5,2,8)
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.V3D, series(k).t, series(k).V3D);

    e2 = (ref_i - y_i).^2;
    mse_v3d = cumulative_mean(e2);

    plot(tc, mse_v3d, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

plot_correction_windows(gca, range_time_without_correction_local, false);

grid on
xlabel('Tempo [s]')
ylabel('MSE V_{3D} [(m/s)^2]')
title('MSE V_{3D} Acumulado vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

%% 9/10 - Erro comparativo V_3D

subplot(5,2,[9 10])
hold on

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [tc, ref_i, y_i] = align_by_time(ref.t, ref.V3D, series(k).t, series(k).V3D);

    erro_v3d = ref_i - y_i;

    plot(tc, erro_v3d, ...
        'Color', series(k).color, 'LineWidth', lineWidth);
end

yline(0, 'k--', 'XPlane ref');
plot_correction_windows(gca, range_time_without_correction_local, false);
```

```
grid on
xlabel('Tempo [s]')
ylabel('Erro V_{3D} [m/s]')
title('Erro de Velocidade em Relação ao Solo 3D - NED vs XPlane')
legend(build_error_legend(series, idx_ref), 'Location','best')
hold off

annotation('textbox', [0 0.955 1 0.035], ...
    'String', 'Velocidades NED, V_{3D} e MSE - XPlane x Estimadores', ...
    'HorizontalAlignment', 'center', ...
    'VerticalAlignment', 'middle', ...
    'FontWeight', 'bold', ...
    'FontSize', 14, ...
    'EdgeColor', 'none');

%% ==================== ESTATÍSTICAS ====================

fprintf('\n--- Métricas finais de velocidade vs XPlane ---\n');

for k = 1:numel(series)

    if k == idx_ref
        continue;
    end

    [~, vN_ref, vN_k]   = align_by_time(ref.t, ref.vN,  series(k).t, series(k).vN);
    [~, vE_ref, vE_k]   = align_by_time(ref.t, ref.vE,  series(k).t, series(k).vE);
    [~, vD_ref, vD_k]   = align_by_time(ref.t, ref.vD,  series(k).t, series(k).vD);
    [~, V3D_ref, V3D_k] = align_by_time(ref.t, ref.V3D, series(k).t, series(k).V3D);

    rmse_vN  = sqrt(mean((vN_ref - vN_k).^2));
    rmse_vE  = sqrt(mean((vE_ref - vE_k).^2));
    rmse_vD  = sqrt(mean((vD_ref - vD_k).^2));
    rmse_V3D = sqrt(mean((V3D_ref - V3D_k).^2));

    fprintf('%-12s | RMSE V_N=%.3f m/s | RMSE V_E=%.3f m/s | RMSE V_D=%.3f m/s | RMSE V3D=%.3f m/s\n', ...
        series(k).name, rmse_vN, rmse_vE, rmse_vD, rmse_V3D);
end

fprintf('\n========== FIM DO PLOT_COMPARE_ALL5 ==========\n');

%% ========================================================================
% FUNÇÕES LOCAIS
%% ========================================================================

function s = new_series(name, key, color)

    s.name = name;
    s.key = key;
    s.t = [];
    s.vN = [];
    s.vE = [];
    s.vD = [];
    s.V3D = [];
    s.VT_xplane = [];
    s.color = color;

end

function s = read_dbn_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % DBN:
    % 4:6 = [vN vE vD]
    s.vN = data(:,4);
    s.vE = data(:,5);
    s.vD = data(:,6);

    s.V3D = sqrt(s.vN.^2 + s.vE.^2 + s.vD.^2);

end

function s = read_ekf_format(data, t, name, key, color)

    s = new_series(name, key, color);

    s.t = t;

    % EKF:
    % 7:9 = [vN vE vD]
    s.vN = data(:,7);
    s.vE = data(:,8);
    s.vD = data(:,9);
```

```
    s.V3D = sqrt(s.vN.^2 + s.vE.^2 + s.vD.^2);

end

function [data, t] = get_out_signal(out_local, names)

    for i = 1:numel(names)

        name_i = names{i};

        try
            sig = out_local.(name_i);
        catch
            try
                sig = out_local.get(name_i);
            catch
                continue;
            end
        end

        try
            if isstruct(sig) && isfield(sig,'signals') && isfield(sig,'time')
                data = sig.signals.values;
                t = sig.time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'timeseries')
                data = sig.Data;
                t = sig.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

            if isa(sig,'Simulink.SimulationData.Signal')
                data = sig.Values.Data;
                t = sig.Values.Time;

                data = squeeze(data);

                if size(data,1) ~= numel(t) && size(data,2) == numel(t)
                    data = data.';
                end

                return;
            end

        catch
        end
    end

    error('Nenhum dos sinais solicitados foi encontrado em out.');

end

function [tc, yref_i, y_i] = align_by_time(tref, yref, t, y)

    tref = tref(:);
    t = t(:);
    yref = yref(:);
    y = y(:);

    t0 = max(tref(1), t(1));
    tf = min(tref(end), t(end));

    N = min([length(tref), length(t), 5000]);

    tc = linspace(t0, tf, N).';

    yref_i = interp1(tref, yref, tc, 'linear');
    y_i    = interp1(t, y, tc, 'linear');

end
```

```
function mse = cumulative_mean(x)

    n = (1:length(x)).';
    mse = cumsum(x) ./ n;

end

function labels = build_error_legend(series, idx_ref)

    labels = {};

    for k = 1:numel(series)

        if k ~= idx_ref
            labels{end+1} = series(k).name; %#ok<AGROW>
        end
    end

end

function plot_correction_windows(ax, range_no_corr, show_labels, label_y_value)

    if nargin < 4
        label_y_value = [];
    end

    if isempty(range_no_corr) || size(range_no_corr,2) ~= 2
        return;
    end

    axes(ax); %#ok<LAXES>

    yl = ylim(ax);
    y_span = yl(2) - yl(1);

    if y_span <= 0
        y_span = 1;
    end

    %% Calcula posição vertical do texto
    if ~isempty(label_y_value)
        label_y = label_y_value;
    else
        line_handles = findobj(ax, 'Type', 'line');

        y_all = [];

        for ii = 1:numel(line_handles)
            try
                y_i = line_handles(ii).YData;
                y_i = y_i(:);
                y_i = y_i(isfinite(y_i));

                if ~isempty(y_i)
                    y_all = [y_all; y_i]; %#ok<AGROW>
                end
            catch
            end
        end

        if ~isempty(y_all)
            label_y = 0.5 * (min(y_all) + max(y_all));
        else
            label_y = 0.5 * (yl(1) + yl(2));
        end
    end

    %% Plota janelas sem correção
    for kk = 1:size(range_no_corr,1)

        ti = range_no_corr(kk,1);
        tf = range_no_corr(kk,2);

        if ti == 0 && tf == 0
            continue;
        end

        if tf <= ti
            continue;
        end

        hp = patch(ax, ...
            [ti tf tf ti], ...
            [yl(1) yl(1) yl(2) yl(2)], ...
            [0.85 0.85 0.85], ...
            'FaceAlpha', 0.22, ...
```

```
            'EdgeColor', 'none', ...
            'HandleVisibility', 'off');

        try
            uistack(hp, 'bottom');
        catch
        end

        xline(ax, ti, 'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
        xline(ax, tf, 'k-',  'LineWidth', 1.0, 'HandleVisibility', 'off');

        if show_labels
            text(ax, ...
                (ti + tf)/2, label_y, ...
                sprintf('sem correção %d', kk), ...
                'HorizontalAlignment', 'center', ...
                'VerticalAlignment', 'middle', ...
                'FontSize', 8, ...
                'Color', [0.15 0.15 0.15], ...
                'BackgroundColor', [1 1 1], ...
                'Margin', 1, ...
                'HandleVisibility', 'off');
        end
    end

    ylim(ax, yl);

end
```

4.54. README.md

Arquivo: README.md

Extensão: .md

Conteúdo:

````
# PIPER-1-6

Repositório de desenvolvimento do ambiente de simulação, guiagem, navegação e integração com X-Plane para a
aeronave **Piper J-3 Cub em escala 1/6**.

O projeto reúne modelos matemáticos não lineares, modelos Simulink, scripts MATLAB, algoritmos de guiagem por
waypoints, filtros de navegação, modelos de sensores e rotinas de integração com o simulador X-Plane.

---

## Visão geral

O repositório está organizado em módulos independentes, mas integrados pelo script principal de inicialização. A
estrutura atual contém aproximadamente:

- **103 arquivos listados**; (para atualizar números: [Geração automática de relatório](#geração-automática-de-
relatório)).
- **77 arquivos `.m`**;
- **9 arquivos `.md`**;
- modelos Simulink `.slx`;
- arquivos de parâmetros `.mat`;
- documentos técnicos `.pdf`;
- integração com X-Plane via XPlaneConnect.

O fluxo típico de uso é:

1. configurar o ambiente MATLAB;
2. executar o script de inicialização;

    inicializar_UI.m no root. Esse arquivo permite configurar a telemetria e intervalo sem
    correção da medida.

    plots podem ser configurados no aquirvo plots/init_plots e também um a um no inicio, definindo
    quais resultados plotar, cores, intervalos, etc.
3. abrir ou executar o modelo Simulink desejado;

    Nota sobre o ponto 3: Para simulações SIL com X-Plane,
rodar com o modelo NL_guidance.slx fechado.
Com o modelo aberto, a interface gráfica do Simulink pode atrasar a
execução real e degradar a trajetória.
4. configurar waypoints, sensores ou filtros;
5. executar a simulação;
6. visualizar resultados por meio dos scripts de plotagem;
7. gerar documentação automática do repositório, quando necessário.

---

## Estrutura do repositório

```text
PIPER-1-6/
├── docs/
├── guiagem/
├── modelos/
│   └── Não Linear/
├── navigation/
│   ├── DBN/
│   ├── FK/
│   └── sensors/
├── plots/
├── xplane/
│   ├── Xplane_Interface/
│   └── XPlaneConnect-master/
├── .gitignore
├── inicializar_UI.m
├── gerar_relatorio_repositorio.py
├── relatorio_repositorio.pdf
└── README.md
```

---

## Módulos principais

### `docs/`

Contém documentos técnicos e referências utilizadas no desenvolvimento, como dissertações, materiais de aula e
documentos compartilhados do trabalho.
````

Exemplos de arquivos identificados:

- `aula7_c.pdf`
- `Dissertacao_Mestrado_Sato (1).pdf`
- `dissertação_Marcelo.pdf`
- `Trabalho3_compartilhado.pdf`

`guiagem/`

Módulo responsável pela guiagem por waypoints, controle de missão e interface visual de simulação.

Arquivos principais:

Arquivo	Função
`NL_guidance.slx`	Modelo Simulink de guiagem LOS, piloto automático e planta não linear.
`gui_waypoints.m`	Interface gráfica para criação e edição de waypoints.
`inicializar.m`	Inicialização de parâmetros, controladores, waypoints e integração.
`scr_waypoints.m`	Definição textual de missão por waypoints.
`scr_aux_wp.m`	Rotina auxiliar para preparar e executar simulações por waypoint.
`analise_sim.m`	Análise pós-simulação.
`find_trim.m`	Busca alternativa de ponto de equilíbrio.
`trim_cost_fn.m`	Função custo usada em trimagem.
`plots_guidance.mlx`	Live Script para validação e plotagem da guiagem.

A guiagem utiliza lógica **Line-of-Sight (LOS)**, recebendo posição atual, matriz de waypoints, raio de aceitação e estado inicial. As principais saídas são:

- `psi_ref`: proa desejada;
- `h_ref`: altitude desejada;
- `v_ref`: velocidade desejada;
- `wp_idx_monitor`: índice do waypoint ativo;
- `dist_monitor`: distância até o waypoint alvo.

A troca de waypoint ocorre quando a distância até o ponto alvo é menor ou igual a `R_accept`.

`modelos/`

Contém os modelos matemáticos da aeronave. O subdiretório mais importante é `modelos/Não Linear/`, que reúne o modelo 6-DOF usado pelos modelos Simulink.

`modelos/Não Linear/`

Arquivos principais:

Arquivo	Função
`sfunction_piper.m`	S-Function Level-1 para integração do modelo 6-DOF no Simulink.
`dyn_rigidbody.m`	Equações de movimento e derivadas dos 12 estados.
`obs_rigidbody.m`	Conversão dos estados em saídas observáveis.
`aerodynamics.m`	Cálculo de forças e momentos aerodinâmicos.
`aerodynamics2.m`	Versão alternativa do modelo aerodinâmico.
`propulsion.m`	Modelo de propulsão.
`equilibrium.m`	Definição do ponto de equilíbrio `Xe` e entrada de equilíbrio `Ue`.
`lin.m`	Linearização numérica em torno do equilíbrio.
`decoupling.m`	Separação longitudinal e látero-direcional.
`ISA.m`	Modelo de atmosfera padrão.
`modelo.slx`	Modelo Simulink da planta não linear.
`gerar_log.m`	Extração de variáveis e geração de gráficos/logs.
`plot_long.m`	Visualização de variáveis longitudinais.

A S-Function `sfunction_piper.m` possui:

- **12 estados contínuos**: velocidades, taxas angulares, ângulos de Euler e posição NED;
- **4 entradas**: throttle, profundor, aileron e leme;
- **21 saídas**: variáveis aerodinâmicas, estados, posição, velocidades no corpo e derivadas.

O ponto de equilíbrio definido no projeto corresponde a voo reto e nivelado com:

```matlab
VT = 15;        % m/s
Theta = -0.121684;   % rad
Xe = [15; 0; -1.8343; 0; 0; 0; 0; -0.1217; 0; 0; 0; -100]
Ue = [0.4914; 0.0155; 0; 0]
```

`navigation/`

Módulo dedicado à navegação inercial, estimação de estados, sensores e filtros.

Estrutura principal:

```text
navigation/
├── DBN/
├── FK/
├── sensors/
├── ins_init_block.m
└── README.md
```

`navigation/DBN/`

Contém o algoritmo de navegação baseado em DBN e sua inicialização:

- `DBN_solver.m`
- `init_DBN.m`

O DBN propaga atitude, velocidade e posição usando leituras de aceleração e giroscópio, com inicialização baseada no estado inicial do X-Plane.

`navigation/FK/`

Contém filtros de Kalman estendidos diretos e indiretos:

- `EKF_DI_solver.m`
- `EKF_DI_EM_solver.m`
- `EKF_INDI_solver.m`
- `EKF_INDI_EM_solver.m`
- `init_EKF_DI.m`
- `init_EKF_INDI.m`

Os filtros estimam posição, velocidade, atitude, aceleração em NED e estados internos associados a bias, escala e erros de medição.

`navigation/sensors/`

Modelos de sensores e inicialização:

- IMU `ICM20689`;
- magnetômetro `IST8310`;
- GPS/GNSS `NEOM8`;
- `ins_init_sensors.m`;
- `ins_telemetry.m`.

Os sensores incluem modelos de erro, ruído, bias, escala, desalinhamento, quantização, saturação e efeitos de temperatura.

`plots/`

Contém scripts para visualização e comparação de resultados de simulação.

Arquivos típicos:

- `init_plots.m`
- `plot3d_voo.m`
- `plot3d_voo_DBN.m`
- `plot3d_voo_xplane.m`
- `plot_compare_all.m`
- `plot_compare_all2.m`
- `plot_compare_all3.m`
- `plot_compare_all4.m`
- `plot_compare_all5.m`

Esses scripts permitem gerar trajetórias 3D, gráficos de altitude, ângulos de Euler, velocidades, erros de estimadores e métricas comparativas entre X-Plane, DBN e EKFs.

`xplane/`

Módulo de integração com o simulador X-Plane.

Estrutura principal:

```text
xplane/
├── Xplane_Interface/
└── XPlaneConnect-master/
```

`xplane/Xplane_Interface/`

Contém os scripts próprios de interface entre o projeto e o X-Plane:

- `ins_init_xplane.m`
- `ins_initial_state_xplane.m`
- `ins_read_xplane.m`
- `ins_send_xplane.m`
- `ins_delta_posi_calculation.m`
- `ins_wyps_insertion.m`

Também há uma pasta de testes com scripts e modelo Simulink para validação da comunicação.

`xplane/XPlaneConnect-master/`

Contém a biblioteca XPlaneConnect, incluindo funções MATLAB para comunicação UDP com o X-Plane, como:

- `openUDP.m`
- `closeUDP.m`
- `getDREFs.m`
- `getPOSI.m`
- `sendCTRL.m`
- `sendDREF.m`
- `sendPOSI.m`
- `pauseSim.m`
- `sendWYPT.m`

Dependências

Requisitos principais:

- MATLAB R2024a ou superior;
- Simulink;
- Optimization Toolbox, se forem utilizadas rotinas de trimagem ou otimização;
- X-Plane, para simulações integradas;
- XPlaneConnect, já incluído no repositório em `xplane/XPlaneConnect-master/`;
- Python 3, para geração automática de relatório;
- biblioteca Python `reportlab`, para geração de PDF.

Para instalar a dependência Python do relatório:

```bash
pip install reportlab
```

Se estiver usando ambiente virtual:

```bash
python -m pip install reportlab
```

Como executar a simulação principal

1. Abrir o MATLAB na raiz do repositório

Certifique-se de que a pasta atual do MATLAB seja a raiz do projeto `PIPER-1-6`.

2. Inicializar o ambiente

Execute:

```matlab
inicializar_UI
```

Esse script limpa o ambiente, adiciona caminhos essenciais, configura telemetria, executa `inicializar` e abre a interface de waypoints.

Alternativamente, para inicialização manual do módulo de guiagem:

```matlab
inicializar
```

3. Usar a interface de waypoints

Execute:

```matlab
gui_waypoints
```

Na interface, é possível:

- clicar no mapa para adicionar waypoints;
- editar coordenadas, altitude e velocidade;

- ajustar o raio de aceitação `R_accept`;
- definir tempo de simulação;
- executar automaticamente a simulação;
- visualizar os resultados ao final.

4. Executar missão por script

Também é possível configurar a missão diretamente em:

```matlab
guiagem/scr_waypoints.m
```

Depois, execute:

```matlab
scr_waypoints
```

Convenções importantes

- O sistema de coordenadas utilizado é principalmente **NED**:
  - Norte;
  - Leste;
  - Down.
- A altitude positiva para cima é representada como `-xD`.
- O waypoint inicial é geralmente fixo na origem.
- O estado de equilíbrio da aeronave utiliza altitude de referência de aproximadamente 100 m.
- A S-Function `sfunction_piper.m` é compartilhada entre os modelos Simulink.
- As variáveis principais são carregadas no workspace base do MATLAB.

Geração automática de relatório

O repositório inclui o script:

```text
gerar_relatorio_repositorio.py
```

Esse script gera automaticamente um relatório PDF do projeto a partir da raiz do repositório.

O que o script faz

O script:

1. percorre a pasta raiz e todas as subpastas;
2. registra a estrutura de pastas e arquivos;
3. lista todos os arquivos encontrados por caminho relativo;
4. gera um resumo por extensão;
5. abre e copia o conteúdo apenas de arquivos `.m` e `.md`;
6. gera um único PDF final chamado:

```text
relatorio_repositorio.pdf
```

Arquivos como `.pdf`, `.slx`, `.mat`, `.mlx`, `.jar`, `.txt` e outros são apenas listados. O conteúdo desses arquivos não é lido nem copiado para o relatório.

Como executar

Na raiz do repositório, execute:

```bash
python gerar_relatorio_repositorio.py
```

Ou, se estiver usando ambiente virtual:

```bash
.
venv\Scripts\activate
python gerar_relatorio_repositorio.py
```

No PowerShell, a ativação pode ser:

```powershell
.\venv\Scripts\Activate.ps1
python gerar_relatorio_repositorio.py
```

Saída esperada

Após a execução, será criado:

```text
relatorio_repositorio.pdf
```

O PDF contém:

1. estrutura de pastas e arquivos;
2. lista ordenada de arquivos;
3. resumo por extensão;
4. conteúdo completo dos arquivos `.m` e `.md`.

Relatórios e documentação

Além dos arquivos `README.md` presentes nos módulos, o projeto pode ser documentado automaticamente usando o relatório gerado pelo script Python. Recomenda-se gerar um novo relatório sempre que houver alterações significativas em:

- scripts MATLAB;
- modelos Simulink;
- documentação Markdown;
- estrutura de pastas;
- arquivos de entrada, parâmetros ou resultados relevantes.

Observações de manutenção

- Manter os READMEs dos módulos sincronizados com alterações nos scripts principais.
- Atualizar este README sempre que um novo módulo for adicionado.
- Evitar versionar arquivos temporários do MATLAB, caches e outputs pesados de simulação.
- Se o relatório PDF for grande, avaliar se ele deve ser mantido no repositório ou gerado sob demanda.
- Conferir periodicamente os caminhos configurados em `inicializar.m` e `inicializar_UI.m`.

Arquivos de entrada e resultados identificados

O relatório atual identifica, entre outros, os seguintes tipos de arquivo:

Extensão	Função típica no projeto
`.m`	Scripts e funções MATLAB.
`.md`	Documentação dos módulos.
`.slx`	Modelos Simulink.
`.mat`	Parâmetros, trim e dados de modelo.
`.mlx`	Live Scripts MATLAB.
`.pdf`	Referências e documentos técnicos.
`.jar`	Biblioteca Java do XPlaneConnect.
`.txt`	Arquivos auxiliares e gravações.

Referência rápida

Inicialização com interface

```matlab
inicializar_UI
```

Interface de waypoints

```matlab
gui_waypoints
```

Missão por script

```matlab
scr_waypoints
```

Abrir modelo de guiagem

```matlab
open('guiagem/NL_guidance.slx')
```

Gerar relatório do repositório

```bash
pip install reportlab
python gerar_relatorio_repositorio.py
```

Este README foi atualizado com base no relatório automático `relatorio_repositorio.pdf`, incluindo a documentação do script `gerar_relatorio_repositorio.py` e do fluxo de geração do relatório do repositório.

Esse gerador pode ser utilizado como input para LLMs, contribuindo para dar contexto ao desenvolvimento e também para
atualizar informações (por exemplo a contagem de arquivos no inicio) ou ter visão geral. É possível, no
main do script, selecionar quais pastas do repositório incluir ou não no relatório.

4.55. xplane/Xplane_Interface/interface/ins_delta_posi_calculation.m

Arquivo: xplane/Xplane_Interface/interface/ins_delta_posi_calculation.m

Extensão: .m

Conteúdo:

[vazio]

4.56. xplane/Xplane_Interface/interface/ins_init_xplane.m

Arquivo: xplane/Xplane_Interface/interface/ins_init_xplane.m

Extensão: .m

Conteúdo:

```
%% ========== Limpar conexao anterior ==========
fprintf('\n----------------------ins_init_xplane:----------------------')
fprintf('\n------------------------ X-plane -------------------------')
global GlobalSocket;

import XPlaneConnect.*; % Importa a biblioteca para o escopo atual

if ~isempty(GlobalSocket)
    try
        closeUDP(GlobalSocket);
        fprintf('\nins_init_xplane: Conexao anterior fechada.\n');
    catch ME
        fprintf(['\nins_init_xplane: Erro ao fechar a conexão: ' ...
            '\nError: %s\n'], ME.message);
    end
    GlobalSocket = []; % Limpa após fechar
end

% AGORA CRIAMOS OU REINICIAMOS A CONEXÃO
fprintf('\nins_init_xplane: Iniciando/Testando conexão UDP...');

try
    % 1. Inicializa o soquete antes de testar
    % (Ajuste o IP/Porta se necessário)
    % Se a função openUDP() não receber argumentos, ela usa os padrões
    % (127.0.0.1, 49009)
    GlobalSocket = openUDP();

    % 2. Tenta ler a altitude AGL para validar o teste
    drefs = {'sim/flightmodel/position/y_agl'};
    result = double(getDREFs(drefs, GlobalSocket));

    ALG_ins_init_xplane_test_connection = result(1);
    fprintf('\nins_init_xplane: Readed AGL: %.2f m\n', ...
        ALG_ins_init_xplane_test_connection);
    fprintf('\nins_init_xplane: Conexão bem sucedida, continuando...\n');
    clear ALG_ins_init_xplane_test_connection

catch ME
    % Limpa o soquete mal sucedido para não travar a próxima execução
    GlobalSocket = [];

    fprintf(['\nins_init_xplane: Erro na leitura de AGL, ' ...
        'falha no teste de conexão.\nError: %s\n' ...
        '\nins_init_xplane: Encerrando simulação.\n'], ME.message);
    error('ins_init_xplane:SimulacaoEncerrada', ...
        'ins_init_xplane: Simulação encerrada por falta de conexão.');
end

%% ========== Parametros do X-Plane ==========
Ts_xplane = 0.05;  % Sample time: 20 Hz (adjust if necessary)
assignin('base', 'Ts_xplane', Ts_xplane);
fprintf('\n------------------------------------------------------------\n')
```

4.57. xplane/Xplane_Interface/interface/ins_initial_state_xplane.m

Arquivo: xplane/Xplane_Interface/interface/ins_initial_state_xplane.m

Extensão: .m

Conteúdo:

```
function ins_initial_state_xplane()
%Teleporta a aeronave para a condicao inicial de voo
    global GlobalSocket;
    import XPlaneConnect.*;

    if isempty(GlobalSocket)
        try
            GlobalSocket = openUDP('127.0.0.1', 49009);
        catch ME
            disp(['posicionar_xplane: falha ao conectar - ' ME.message]);
            return;
        end
    % else
    %     disp('X-Plane Communication: ins_initial_state_xplane: UDP conection alread exists.')
    end
    WPs = evalin('base','WPs'); %getting initial state (position, velocity)
    try
        drefs_lla = {'sim/flightmodel/position/latitude', ...
                    'sim/flightmodel/position/longitude', ...
                    'sim/flightmodel/position/elevation', ...
                    'sim/flightmodel/position/y_agl'};
        lla = double(getDREFs(drefs_lla, GlobalSocket)); %ll: lat - long - MSL
        pauseSim(1, GlobalSocket);
        pause(0.2);% anterior pause(0.5);
        psi0 = 0;  % Norte
        % Set the initial position and orientation of the aircraft
        % lla(1) - latitude
        % lla(2) - longitude
        % lla(3) - elevation-Mean Sea Level - MSL
        % lla(4) - above ground level - AGL
        elev_msl = lla(3); % Mean Sea Level elevation
        y_agl = lla(4); %
        ground_msl = elev_msl - y_agl;
        target_msl = ground_msl + WPs(1,3); % final height = WPs(1,3) above ground
        sendPOSI([lla(1), lla(2), target_msl, 0, -0.121684, psi0, 0], 0, GlobalSocket);
        pause(0.2);% anterior pause(0.5);
        abs_V0 = WPs(1,4);
        fprintf("Velocidade absoluta inicial: %.2f [m/s] \n",abs_V0);
        hdg_rad = psi0 * pi/180;
        sendDREF('sim/flightmodel/position/local_vx',  abs_V0*sin(hdg_rad), GlobalSocket);
        sendDREF('sim/flightmodel/position/local_vy',  0,                   GlobalSocket);
        sendDREF('sim/flightmodel/position/local_vz', -abs_V0*cos(hdg_rad), GlobalSocket);
        pause(0.2);
        sendCTRL([0, 0, 0, 0.49, -998, -998], 0, GlobalSocket);
        pause(0.2);
        pauseSim(0, GlobalSocket);

        % Limpar persistent vars (xN0/xE0) para reiniciar refs de posicao
        clear ins_read_xplane;

        fprintf('posicionar_xplane: aeronave em %.1f m, VT=%.1f m/s, hdg=%.1f deg.\n',WPs(1,3), WPs(1,4), psi0);
        %% Inicializa DBN após posicionamento do X-Plane
        phi0_rad   = 0;
        theta0_rad = 0;
        psi0_rad   = hdg_rad;

        pos0_ned = [ ...
            WPs(1,1);
            WPs(1,2);
            -WPs(1,3) ...
            ];

        vel0_ned = [ ...
            abs_V0*cos(psi0_rad);
            abs_V0*sin(psi0_rad);
            0 ...
            ];

        DBN_data_struct = struct();
        DBN_data_struct.euler0 = [phi0_rad; theta0_rad; psi0_rad];
        DBN_data_struct.pos0_ned = pos0_ned;
        DBN_data_struct.vel0_ned = vel0_ned;

        init_DBN(DBN_data_struct); % DBN
        init_EKF_DI(DBN_data_struct); % EKF DIRETO
        init_EKF_INDI(DBN_data_struct); % EKF INDIRETO
        % ------------------- FIM DBN --------------------------
    catch ME
```

```
        disp(['posicionar_xplane: erro - ' ME.message]);
    end
end
```

4.58. xplane/Xplane_Interface/interface/ins_read_xplane.m

Arquivo: xplane/Xplane_Interface/interface/ins_read_xplane.m

Extensão: .m

Conteúdo:

```
function xplane_sensors = ins_read_xplane(~)
%READ_XPLANE Le o estado da aeronave no X-Plane via XPlaneConnect.
%
% Output:
%   sensors(1)  = VT    - velocidade aerodinamica (m/s)
%   sensors(2)  = theta - arfagem (rad)
%   sensors(3)  = q     - taxa de arfagem (rad/s)
%   sensors(4)  = h     - altitude AGL (m)
%   sensors(5)  = phi   - rolamento (rad)
%   sensors(6)  = p     - taxa de rolamento (rad/s)
%   sensors(7)  = psi   - proa/heading atual (rad)
%   sensors(8)  = r     - taxa de guinada (rad/s)
%   sensors(9)  = xN    - posicao Norte relativa ao inicio (m)
%   sensors(10) = xE    - posicao Leste relativa ao inicio (m)
%   sensors(11) = abx   - aceleracao no eixo x do corpo (m/s^2)
%   sensors(12) = aby   - aceleracao no eixo y do corpo (m/s^2)
%   sensors(13) = abz   - aceleracao no eixo z do corpo (m/s^2)
%   sensors(14) = vN    - velocidade Norte (m/s)
%   sensors(15) = vE    - velocidade Leste (m/s)
%   sensors(16) = vD    - velocidade Down (m/s)
%   sensors(17) = magnetic_variation - variacao magnetica local (rad)
%   sensors(18) = t_xplane - tempo de execução do X-Plane (s)-Total time the sim has been up

    global GlobalSocket;
    import XPlaneConnect.*;

    persistent xN0 xE0 initialized;

    xplane_sensors = zeros(1, 18);

    %% CONEXAO COM XPLANE
    if isempty(GlobalSocket)
        try
            GlobalSocket = openUDP('127.0.0.1', 49009);
            disp('ins_read_xplane: Conexao X-Plane aberta.');
        catch ME
            disp(['ins_read_xplane: Falha ao conectar - ' ME.message]);
            return;
        end
    end

    if ~isa(GlobalSocket, 'gov.nasa.xpc.XPlaneConnect')
        return;
    end

    %% Ler DataRefs
    try
        drefs = {
            'sim/flightmodel/position/true_airspeed',       % 1: VT (m/s)
            'sim/flightmodel/position/theta',               % 2: pitch (deg)
            'sim/flightmodel/position/Qrad',                % 3: pitch rate (rad/s)
            'sim/flightmodel/position/y_agl',               % 4: altitude AGL (m)
            'sim/flightmodel/position/phi',                 % 5: roll (deg)
            'sim/flightmodel/position/Prad',                % 6: roll rate (rad/s)
            'sim/flightmodel/position/psi',                 % 7: heading (deg)
            'sim/flightmodel/position/Rrad',                % 8: yaw rate (rad/s)
            'sim/flightmodel/position/local_x',             % 9: local_x = East
            'sim/flightmodel/position/local_z',             % 10: local_z = South
            'sim/flightmodel/position/local_ax',            % 11: acceleration x
            'sim/flightmodel/position/local_ay',            % 12: acceleration y
            'sim/flightmodel/position/local_az',            % 13: acceleration z
            'sim/flightmodel/position/local_vx',            % 14: local_vx = East
            'sim/flightmodel/position/local_vy',            % 15: local_vy = Up
            'sim/flightmodel/position/local_vz',            % 16: local_vz = South
            'sim/flightmodel/position/magnetic_variation',  % 17: magnetic variation (deg)
            'sim/time/total_running_time_sec',              % 18: Total time the sim has been up
        };

        result = double(getDREFs(drefs, GlobalSocket));

        d2r = pi / 180;

        %% Sinais originais - indices 1 a 16 preservados
        VT    = result(1);
        theta = result(2) * d2r;
        q     = result(3);
        h     = result(4);
        phi   = result(5) * d2r;
```

```
p     = result(6);
psi   = result(7) * d2r;
psi   = wrapToPi_local(psi);
r     = result(8);

%% Posicao OpenGL EUS -> NED
xE_abs = result(9);
xN_abs = -result(10);

%% Aceleracoes locais EUS -> NED
aE_local = result(11);
aU_local = result(12);
aS_local = result(13);

aN = -aS_local;
aE =  aE_local;
aD = -aU_local;

%% Euler para rotacao NED -> BODY
roll = phi;
pit  = theta;
yaw  = psi;

R_roll = [ ...
    1,          0,           0; ...
    0,  cos(roll),   sin(roll); ...
    0, -sin(roll),   cos(roll)];

R_pitch = [ ...
     cos(pit), 0, -sin(pit); ...
            0, 1,         0; ...
     sin(pit), 0,  cos(pit)];

R_yaw = [ ...
     cos(yaw),  sin(yaw), 0; ...
    -sin(yaw),  cos(yaw), 0; ...
            0,         0, 1];

R_n2b = R_roll * R_pitch * R_yaw;

acc_b = R_n2b * [aN; aE; aD];

abx = acc_b(1);
aby = acc_b(2);
abz = acc_b(3);

%% Velocidades EUS -> NED
vE_local = result(14);
vU_local = result(15);
vS_local = result(16);

vN = -vS_local;
vE =  vE_local;
vD = -vU_local;

%% Posicao relativa
if isempty(initialized)
    xN0 = xN_abs;
    xE0 = xE_abs;
    initialized = true;

    disp(['ins_read_xplane: Posicao inicial capturada (N=' ...
        num2str(xN0, '%.1f') ', E=' num2str(xE0, '%.1f') ')']);
end

xN = xN_abs - xN0;
xE = xE_abs - xE0;

%% Nova variavel - indice 17
magnetic_variation = result(17) * d2r;

%% Tempo da simulação
t_xplane = result(18);
%% Vetor de saida
xplane_sensors = [ ...
    VT, theta, q, h, phi, p, psi, r, ...
    xN, xE, abx, aby, abz, vN, vE, vD, ...
    magnetic_variation, t_xplane];

% fprintf(['VT: %.3f, theta: %.3f, q: %.3f, h: %.3f, phi: %.3f, p: %.3f, psi: %.3f, r: %.3f, ' ...
%          'xN: %.3f, xE: %.3f, abx: %.3f, aby: %.3f, abz: %.3f, vN: %.3f, vE: %.3f, vD: %.3f, ' ...
%          'mavar: %.3f\n'], ...
%          VT, theta, q, h, phi, p, psi, r, ...
%          xN, xE, abx, aby, abz, vN, vE, vD, ...
%          magnetic_variation);
```

```
    catch ME
        disp(['ins_read_xplane: Erro na leitura: ' ME.message]);
    end
end

function ang = wrapToPi_local(ang)
    ang = mod(ang + pi, 2*pi) - pi;
end
```

4.59. xplane/Xplane_Interface/interface/ins_send_xplane.m

Arquivo: xplane/Xplane_Interface/interface/ins_send_xplane.m

Extensão: .m

Conteúdo:

```
function status = ins_send_xplane(u)
%SEND_XPLANE Envia comandos de controle do autopiloto para o X-Plane.
%
%   status = send_xplane(u)
%
%   Input u (4x1):
%     u(1) = delta_e  - elevator (rad, [-0.4363, +0.4363])
%     u(2) = delta_a  - aileron  (rad, [-0.4363, +0.4363])
%     u(3) = delta_r  - rudder   (rad, [-0.4363, +0.4363])
%     u(4) = delta_T  - throttle ([0, 1])
%
%   Converte de radianos para normalizado [-1, 1] antes de enviar.
%   Usa variavel global GlobalSocket (compartilhada com read_xplane).

    global GlobalSocket;
    import XPlaneConnect.*;

    status = 0;

    % --- Abrir conexao se necessario ---
    if isempty(GlobalSocket)
        try
            GlobalSocket = openUDP('127.0.0.1', 49009);
            disp('send_xplane: Conexao X-Plane aberta.');
        catch
            return;
        end
    end

    % --- Verificar socket valido ---
    if ~isa(GlobalSocket, 'gov.nasa.xpc.XPlaneConnect')
        return;
    end

    % --- Converter e enviar ---
    try
        max_deflection = 0.4363;  % 25 deg em rad

        elevator = u(1) / max_deflection;
        aileron  = u(2) / max_deflection;
        rudder   = u(3) / max_deflection;
        throttle = u(4);

        % Clamp para faixa valida
        elevator = max(-1, min(1, elevator));
        aileron  = max(-1, min(1, aileron));
        rudder   = max(-1, min(1, rudder));
        throttle = max(0, min(1, throttle));

        % XPC: [elevator, aileron, rudder, throttle, gear (0 up, 1 down), flaps]
        % -998 = "nao alterar"
        ctrl_data = [elevator, aileron, rudder, throttle, 0, -998];

        sendCTRL(ctrl_data, 0, GlobalSocket);
        status = 1;
    catch ME
        disp(['send_xplane: Erro no envio - ' ME.message]);
    end
end
```

4.60. xplane/Xplane_Interface/interface/ins_wyps_insertion.m

Arquivo: xplane/Xplane_Interface/interface/ins_wyps_insertion.m

Extensão: .m

Conteúdo:

[vazio]

4.61. xplane/Xplane_Interface/README.md

Arquivo: xplane/Xplane_Interface/README.md

Extensão: .md

Conteúdo:

```
# Interface for sending commands and receiving data from X-Plane.
```

4.62. xplane/Xplane_Interface/tests/ins_initial_state_xplane_test.m

Arquivo: xplane/Xplane_Interface/tests/ins_initial_state_xplane_test.m

Extensão: .m

Conteúdo:

```
function ins_initial_state_xplane_test()
%Teleporta a aeronave para a condicao inicial de voo
    global GlobalSocket;
    import XPlaneConnect.*;

    if isempty(GlobalSocket)
        try
            GlobalSocket = openUDP('127.0.0.1', 49009);
        catch ME
            disp(['posicionar_xplane: falha ao conectar - ' ME.message]);
            return;
        end
    else
        disp('X-Plane Communication: ins_initial_state_xplane: UDP conection alread exists.')
    end
    WPs = evalin('base','WPs'); %getting initial state (position, velocity)
    try
        drefs_lla = {'sim/flightmodel/position/latitude', ...
                    'sim/flightmodel/position/longitude', ...
                    'sim/flightmodel/position/elevation', ...
                    'sim/flightmodel/position/y_agl'};
        lla = double(getDREFs(drefs_lla, GlobalSocket)); %ll: lat - long - MSL
        pauseSim(1, GlobalSocket);
        pause(0.5);
        psi0 = 0;  % Norte
        % Set the initial position and orientation of the aircraft
        % lla(1) - latitude
        % lla(2) - longitude
        % lla(3) - elevation-Mean Sea Level - MSL
        % lla(4) - above ground level - AGL
        h0 = 500; %WPs(1,3);
        v0 = 30; %WPs(1,4);
        %
        elev_msl = lla(3); % Mean Sea Level elevation
        y_agl = lla(4); %
        ground_msl = elev_msl - y_agl;
        target_msl = ground_msl + h0; % final height = WPs(1,3) above ground
        sendPOSI([lla(1), lla(2), target_msl, 0, 0, psi0, 0], 0, GlobalSocket);
        pause(0.5);
        abs_V0 = v0;
        hdg_rad = psi0 * pi/180;
        sendDREF('sim/flightmodel/position/local_vx',  abs_V0*sin(hdg_rad), GlobalSocket);
        sendDREF('sim/flightmodel/position/local_vy',  0,                   GlobalSocket);
        sendDREF('sim/flightmodel/position/local_vz', -abs_V0*cos(hdg_rad), GlobalSocket);
        pause(0.2);
        sendCTRL([0, 0, 0, 0.49, -998, -998], 0, GlobalSocket);
        pause(0.2);
        pauseSim(0, GlobalSocket);

        % Limpar persistent vars (xN0/xE0) para reiniciar refs de posicao
        clear ins_read_xplane;

        fprintf('posicionar_xplane: aeronave em %.1f m, VT=%.1f m/s, hdg=%.1f deg.\n',WPs(1,3), WPs(1,4), psi0);
    catch ME
        disp(['posicionar_xplane: erro - ' ME.message]);
    end
end
```

4.63. xplane/Xplane_Interface/tests/README.md

Arquivo: xplane/Xplane_Interface/tests/README.md

Extensão: .md

Conteúdo:

```
## How to use
First run the initiate_xplane_tests.m in the root directory
then run tests_inicializar_xplane in the current folder
## Utility
This Simulink Model can be used to perform several communication tests with X-Plane
```

4.64. xplane/Xplane_Interface/tests/tests_inicializar_xplane.m

Arquivo: xplane/Xplane_Interface/tests/tests_inicializar_xplane.m

Extensão: .m

Conteúdo:

```
%% inicializar_xplane.m - Inicializacao para integracao com X-Plane
% Carrega todos os ganhos PID e parametros via inicializar.m,
% depois configura paths e variaveis especificas do X-Plane.
%
% Uso:
%   1) >> inicializar_xplane
%   2) >> criar_modelo_xplane   (apenas na primeira vez, gera o .slx)
%   3) >> open('Xplane/xplane_autopilot.slx')
%   4) Iniciar X-Plane com Piper J-3 Cub
%   5) Simular (Ctrl+T)
%
% %% ========== Carregar base (ganhos, Xe, Ue, matrizes, etc.) ==========
% % Salvar dir atual via setenv (sobrevive ao 'clear' do inicializar.m)
% setenv('XPLANE_INIT_OLDDIR', pwd);
% rootDir_ = fileparts(fileparts(mfilename('fullpath')));
% cd(rootDir_);
% inicializar;   % Faz clear/clc - apaga todo o workspace
% % Recuperar apos o clear
% cd(getenv('XPLANE_INIT_OLDDIR'));
% setenv('XPLANE_INIT_OLDDIR', '');
% rootDir = fileparts(fileparts(which('inicializar_xplane')));

% %% ========== Paths do XPlaneConnect ==========
% addpath(fullfile(rootDir, 'Xplane', 'XPlaneConnect-master', 'MATLAB'));
% addpath(fullfile(rootDir, 'Xplane'));

%% ========== Limpar conexao anterior ==========
global GlobalSocket;
if ~isempty(GlobalSocket)
    try
        import XPlaneConnect.*;
        closeUDP(GlobalSocket);
        disp('inicializar_xplane: Conexao anterior fechada.');
    catch
    end
end
GlobalSocket = [];

%% ========== Parametros do X-Plane ==========
Ts_xplane = 0.05;  % Sample time: 20 Hz (ajustar se necessario)

%% ========== Referencias manuais (modo sem guiagem) ==========
h_ref   = 500; %-Xe(12);   % altitude de equilibrio (100 m)
VT_ref  = 30; %Xe(1);     % velocidade de equilibrio (15 m/s)
psi_ref = 0;         % proa inicial (rad)

%% ========== Pronto ==========
disp(' ');
disp('=== Workspace carregado para X-Plane ===');
disp(['  Sample time: ' num2str(Ts_xplane) ' s (' num2str(1/Ts_xplane) ' Hz)']);
disp(['  h_ref:  ' num2str(h_ref) ' m']);
disp(['  VT_ref: ' num2str(VT_ref) ' m/s']);
disp(['  psi_ref: ' num2str(rad2deg(psi_ref)) ' deg']);
disp(' ');
disp('  Proximo passo: abrir xplane_tests.slx e simular');
```

4.65. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/closeUDP.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/closeUDP.m

Extensão: .m

Conteúdo:

```
function closeUDP(socket)
% closeUDP Closes the specified connection and releases resources associated with it.
%
% Inputs
%     socket: An XPC client to close
%
% Use
% 1. import XPlaneConnect.*;
% 2. socket = openUDP();
% 3. closeUDP(socket);
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

%% Close the socket
assert(isequal(class(socket), 'gov.nasa.xpc.XPlaneConnect'),...
    '[closeUDP] ERROR: socket was not an XPC client.');
socket.close;

%% Track open clients
global clients;
for i = 1:length(clients)
    if socket == clients(i)
        clients = [clients(1:i-1) clients(i+1:length(clients))];
    end
end

end
```

4.66. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getCTRL.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getCTRL.m

Extensão: .m

Conteúdo:

```
function ctrl = getCTRL(ac, socket)
% getCTRL Gets control surface information for the specified aircraft
%
% Inputs
%     ac: The aircraft number to get control surface data for.
% Outputs
%     posi: An array of values matching the the format used by sendCTRL


import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[getCTRL] ERROR: Multiple clients open. You must specify which client
to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
ac = int32(ac);

%% Send command
ctrl = socket.getCTRL(ac);

end
```

4.67. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getDREFs.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getDREFs.m

Extensão: .m

Conteúdo:

```
function result = getDREFs( drefs, socket )
%requestDREF request the value of a specific DataRef from X-Plane over UDP
%
%Inputs
%     drefs: Cell Array of DataRefs to be requested
%     socket (optional): The client to use when sending the command.
%Outputs
% result: cell array of resulting data.
%
%Use
% 1. import XPlaneConnect.*;
%   2. socket = opendUDP();
%   3. drefs = {'sim/cockpit2/controls/yoke_heading_ratio','sim/cockpit2/controls/yoke_roll_ratio'};
% 4. result = requestDREF(drefs, socket);
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%        christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%        jason.w.watkins@nasa.gov

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[getDREFs] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Send command
result = socket.getDREFs(drefs);

end
```

4.68. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getPOSI.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/getPOSI.m

Extensão: .m

Conteúdo:

```
function posi = getPOSI(ac, socket)
% getPOSI Gets position information for the specified aircraft
%
% Inputs
%     ac: The aircraft number to get position data for.
% Outputs
%     posi: An array of values matching the the format used by sendPOSI


import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[getPOSI] ERROR: Multiple clients open. You must specify which client
to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
ac = int32(ac);

%% Send command
posi = socket.getPOSI(ac);

end
```

4.69. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/openUDP.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/openUDP.m

Extensão: .m

Conteúdo:

```
function [ socket ] = openUDP(varargin)
%openUDP  Initializes a new instance of the XPC client and returns it.
%
%Inputs
%   xpHost: The network host on which X-Plane is running.
%   xpPort: The port on which the X-Plane Connect plugin is listening.
%   port: The local port to use when sending and receiving data from XPC.
%   timeout (optional): Optional parameter for time to UDP timeout (in ms)
%Outputs
%   socket: An instance of the XPC client.
%
% Use
% 1. import XPlaneConnect.*;
% 2. socket = openUDP(); % Open a socket using the default settings
% or
% 2. Socket = openUDP('127.0.0.1', 49008, 49005); % Open a socket to connect to X-Plane on the local machine and
receive data on port 49005
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

%% Handle input
p = inputParser;
addOptional(p,'xpHost','127.0.0.1',@ischar);
addOptional(p,'xpPort',49009,@isnumeric);
addOptional(p,'port',0,@isnumeric);
addOptional(p,'timeout',100,@isnumeric);
parse(p,varargin{:});

%% Create client
if ~exist('gov.nasa.xpc.XPlaneConnect', 'class')
    [folder, ~, ~] = fileparts(which('XPlaneConnect.openUDP'));
    javaaddpath(fullfile(folder, 'XPlaneConnect.jar'));
end
socket = gov.nasa.xpc.XPlaneConnect(p.Results.xpHost, p.Results.xpPort, p.Results.port, p.Results.timeout);

%% Track open clients
global clients;
clients = [clients, socket];

end
```

4.70. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/pauseSim.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/pauseSim.m

Extensão: .m

Conteúdo:

```
function pauseSim( pause, socket )
%pauseSim Pauses or unpauses X-Plane.
%
%Inputs
%     pause: int 0= unpause all, 1= pause all, 2= switch all, 100:119= pause a/c 0:19, 200:219= unpause a/c 0:19
%     socket (optional): The client to use when sending the command.
%
%Use
% 1. import XPlaneConnect.*;
% 2. status = pauseSim(1);
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[pauseSim] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
pause = int32(pause);

%% Send command
socket.pauseSim(pause);

end
```

4.71. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/readDATA.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/readDATA.m

Extensão: .m

Conteúdo:

```
function [ result ] = readDATA( socket )
% readDATA Reads UDP Socket and interprets data
%
% Inputs
%     socket (optional): The client to read from.
% Outputs
% If data is X-Plane data format:
%     data: Matlab X-Plane DATA Structure
%         .h: Header array containing header numbers corresponding to values in the X-Plane UDP Data screen.
%         .d: 2-D Data array. Each row contains eight values corresponding to the header value (.h(1) corresponds
to .d(1,:))
%         .raw: raw UDP data array received by readUDP
%
% If data is matlab structure:
%     data: Matlab Structure. raw UDP data saved to data.raw
%
% If data is any other format:
%     data: Matlab Structure containing one field
%         .raw & .d: raw UDP data array received by readUDP
%
% Use
% 1. import XPlaneConnect.*;
% 2. socket = openUDP(49005);
% 3. data = readDATA(socket);
% 4. status = closeUDP(socket);
% or
% 1. import XPlaneConnect.*;
% 2. data = readDATA(49005);
%
% NOTE: sending in a port number instead of an opened socket clears the UDP buffer. This only works if the data is
being sent at a fast rate. Sending in a UDP Socket reads the first packet in the buffer.
%
% Contributors
%   Christopher Teubert (SGT, Inc.) <christopher.a.teubert@nasa.gov>
%   Jason Watkins <jason.w.watkins@nasa.gov>

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal((length(clients) < 2),1), '[readDATA] ERROR: Multiple clients open. You must specify which
client to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Get data
result.raw = socket.readData();

%% Format data
rows = (length(result.raw) - 5) / 9;
result.h = zeroes(rows);
for i=1:rows
    j = 6 + (i - 1) * 9;
    result.h(i) = result.rows(j);
    result.d = [result.d; result.rows((j+1):(j+9))];
end
```

4.72. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/selectDATA.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/selectDATA.m

Extensão: .m

Conteúdo:

```
function selectDATA( rows, socket )
% selectDATA Choose specific X-Plane parameters to be send over UDP
%
% Inputs
%     rows: An array of the values that to be sent. Corresponds to the numbers on the XPlane Output screen
%     socket (optional): The client to use when sending the command.
%
% Use
% 1. import XPlaneConnect.*;
% 2. values = [1, 2, 3, 27, 40];
% 3. status = selectDATA( values, '127.0.0.1', 49005 ); % send to localhose port 49005
%
% Contributors
%   Christopher Teubert (SGT, Inc.) <christopher.a.teubert@nasa.gov>
%   Jason Watkins <jason.w.watkins@nasa.gov>

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal((length(clients) < 2),1), '[selectDATA] ERROR: Multiple clients open. You must specify which
client to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
rows = int32(rows);

%% Send command
socket.selectDATA(rows);
```

4.73. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendCTRL.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendCTRL.m

Extensão: .m

Conteúdo:

```
function sendCTRL( values, ac, socket )
% sendCTRL Sends command to X-Plane setting control surfaces on the specified aircraft.
%
% Inputs
%     values: control array where the elements are as follows:
%           1. Latitudinal Stick [-1,1]
%           2. Longitudinal Stick [-1,1]
%           3. Pedal [-1, 1]
%           4. Throttle [-1, 1]
%           5. Gear (0=up, 1=down)
%           6. Flaps [0, 1]
%           7. Speed Brakes [-0.5, 1.5]
%     ac (optional): The aircraft to set. 0 for the player aircraft.
%     socket (optional): The client to use when sending the command.
%
% Outputs
%     status: If there was an error. Status<0 means there was an error.
%
% Use
% 1. import XPlaneConnect.*;
% 2. socket = openUDP();
% 3. status = sendCTRL([0, 0, 0, 0.8, 0, 1], 0, socket); % Set throttle and flaps on the player aircraft.
%
% Note: send the value -998 to not overwrite that parameter. That is, if
% -998 is sent, the parameter will stay at the current X-Plane value.
%
% Contributors
%   Christopher Teubert (SGT, Inc.) <christopher.a.teubert@nasa.gov>
%   Jason Watkins <jason.w.watkins@nasa.gov>

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[sendCTRL] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
values = single(values);
if ~exist('ac', 'var')
    ac = 0;
end
ac = logical(ac);

%% Send command
socket.sendCTRL(values, ac);

end
```

4.74. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDATA.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDATA.m

Extensão: .m

Conteúdo:

```
function sendDATA(data, socket)
% sendDATA Send X-Plane formatted DATA over UDP
%
% Inputs
%     data: X-Plane formatted data. Is a matlab structure with the following fields:
%         .h: Header array containing header numbers corresponding to values in the X-Plane UDP Data screen.
%         .d: 2-D Data array. Each row contains eight values corresponding to the header value (.h(1) corresponds
to .d(1,:))
%     socket (optional): The client to use when sending the command.
%
% Use
% 1. import XPlaneConnect.*;
% 2. data = struct('h',27,'d',[1,-998,-998,-998,-998,-998,-998,-998]);
% 3. %Send the data array to port 49005 on the computer at IP address 172.0.100.54.
% 4. status = sendDATA(data, '172.0.100.54', 49005);
%
% Note: send the value -998 to not overwrite that parameter. That is, if -998 is sent, the parameter will stay at
the current X-Plane value
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%
% To Do
%
% BEGIN CODE

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[sendDATA] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
javaData = [];
for i = 1:length(data.h);
   javaData = [javaData; data.h(i) data.d(i, 1:8)];
end

%% Send command
socket.sendDATA(javaData);
```

4.75. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREF.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREF.m

Extensão: .m

Conteúdo:

```
function sendDREF( dref, value, socket )
% sendDREFs Sends a command to X-Plane that sets the given DREF. This
% function is now an alias to sendDREFs. It is kept only for backwards
% compatibility.
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

import XPlaneConnect.*

if ~exist('socket', 'var')
    sendDREFs(dref, value)
else
    sendDREFs(dref, value, socket)
end
```

4.76. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREFs.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendDREFs.m

Extensão: .m

Conteúdo:

```
function sendDREFs( drefs, values, socket )
% sendDREFs Sends a command to X-Plane that sets the given DREF(s).
%
% Inputs
%     dref: The name or names of the X-Plane dataref(s) to set.
%     Value: An array or an multidimensional array of floating point values
%            whose structure depends on the dref specified.
%     socket (optional): The client to use when sending the command.
%
% Use
%   1. import XPlaneConnect.*
%   2. dataRef  = 'sim/aircraft/parts/acf_gear_deploy'; // Landing Gear
%   3. Value    = 0;
%   4. status = setDREF(dataRef, Value);
%
% Contributors
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[setDREF] ERROR: Multiple clients open. You must specify which client
to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
values = single(values);

%%Send command
socket.sendDREFs(drefs, values);
```

4.77. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendPOSI.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendPOSI.m

Extensão: .m

Conteúdo:

```
function sendPOSI( posi, ac, socket )
% sendPOSI Sets the position of the specified aircraft.
%
% Inputs
%     posi: Position array where the elements are as follows:
%           1. Latitiude (deg)
%           2. Longitude (deg)
%           3. Altitude (m above MSL)
%           4. Roll (deg)
%           5. Pitch (deg)
%           6. True Heading (deg)
%           7. Gear (0=up, 1=down)
%     acft (optional): The aircraft to set. 0 for the player aircraft.
%     socket (optional): The client to use when sending the command.
%
% Use
%   1. import XPlaneConnect.*;
%   2. sendPOSI([37.5242422, -122.06899, 2500, 0, 0, 0, 1], 1);
%
% Note: send the value -998 to not overwrite that parameter. That is, if
% -998 is sent, the parameter will stay at the current X-Plane value.
%
% Contributors
%   [CT] Christopher Teubert (SGT, Inc.)
%       christopher.a.teubert@nasa.gov
%   [JW] Jason Watkins
%       jason.w.watkins@nasa.gov

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[sendPOSI] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
posi = double(posi);
if ~exist('ac', 'var')
    ac = 0;
end
ac = logical(ac);

%% Send command
socket.sendPOSI(posi, ac);

end
```

4.78. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendTEXT.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendTEXT.m

Extensão: .m

Conteúdo:

```
function sendTEXT( msg, x, y, socket )
% sendTEXT Sends a string to be displayed in X-Plane.
%
% Inputs
%     msg: The string to be displayed
%     x (optional): The distance from the left edge of the screen to display the message.
%     y (optional): The distance from the bottom edge of the screen to display the message.
%     IP Address (optional): IP Address of the machine that will receive the data as a character array. Default is
'127.0.0.1' (local machine)
%     port (optional): Port on the receiving machine where the data will be sent. Default is 49009 (XPlaneConnect).
 In general use 49009 to send to the plugin and 49005 to send to the x-plane udp
%
% Outputs
%     status: 0 if successful, otherwise a negative value.
%
% Use
% 1. import XPlaneConnect.*;
% 2. % Set a message to be displayed near the top middle of the screen.
% 3. status = sendTEXT('Some text', 512, 600);
%
% Contributors
%   Jason Watkins (jason.w.watkins@nasa.gov)

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[sendTEXT] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
if ~exist('x', 'var')
    x = -1;
end
if ~exist('y', 'var')
    y = -1;
end
x = int32(x);
y = int32(y);

%% Send command
socket.sendTEXT(msg, x, y);
end
```

4.79. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendVIEW.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendVIEW.m

Extensão: .m

Conteúdo:

```
function sendVIEW(view, socket)
% sendVIEW Sets the camera
%
%Inputs
%     view: The view to use.
%     socket (optional): The client to use when sending the command.
%
%Use
% 1. import XPlaneConnect.*;
% 2. sendView(Forwards);
%
% Contributors
%   [JW] Jason Watkins <jason.w.watkins@nasa.gov>

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[pauseSim] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input


%% Send command
socket.sendVIEW(view)

end
```

4.80. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendWYPT.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/sendWYPT.m

Extensão: .m

Conteúdo:

```
function sendWYPT( op, points, socket )
% sendWYPT Adds, removes, or clears a set of waypoints to be rendered in
% the simulator.
%
% Inputs
%     op: The operation to perform. 1=add, 2=remove, 3=clear.
%     points: An array of values representing points. Each triplet in the
%             array will be interpreted as a (Lat, Lon, Alt) point.
%     socket (optional): The client to use when sending the command.
%
% Use
% 1. import XPlaneConnect.*;
% 2. %Set a message to be displayed near the top middle of the screen.
% 3. status = sendTEXT('Some text', 512, 600);
%
% Contributors
%   Jason Watkins (jason.w.watkins@nasa.gov)
%
% To Do
%
% BEGIN CODE

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[sendWYPT] ERROR: Multiple clients open. You must specify which client
 to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
len = uint32(length(points));
assert(op > 0 && op < 4);
wyptOp = gov.nasa.xpc.WaypointOp.Add;
if isequal(op, 2)
    wyptOp = gov.nasa.xpc.WaypointOp.Del;
elseif isequal(op, 3)
    wyptOp = gov.nasa.xpc.WaypointOp.Clr;
end
assert(mod(len, 3) == 0);

%% Send command
socket.sendWYPT(wyptOp, points);

end
```

4.81. xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/setConn.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/+XPlaneConnect/setConn.m

Extensão: .m

Conteúdo:

```
function setConn(port, socket)
% setConn Send a command to set up the port where you will receive data on
% this computer.
%
% Inputs
%     port: Port that data will be sent to in the future for this connection.
%     socket (optional): The client to use when sending the command.
%
% Use
%   1. import XPlaneConnect.*
%   2. status = setConn(49011);
%
% Contributors
%   Christopher Teubert (SGT, Inc.) <christopher.a.teubert@nasa.gov>
%   Jason Watkins <jason.w.watkins@nasa.gov>

import XPlaneConnect.*

%% Get client
global clients;
if ~exist('socket', 'var')
    assert(isequal(length(clients) < 2, 1), '[setCONN] ERROR: Multiple clients open. You must specify which client
to use.');
    if isempty(clients)
     socket = openUDP();
    else
     socket = clients(1);
    end
end

%% Validate input
port = int32(port);

%% Send command
socket.setCONN(port);

end
```

4.82. xplane/XPlaneConnect-master/MATLAB/Example/Example.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/Example/Example.m

Extensão: .m

Conteúdo:

```
%% X-Plane Connect MATLAB Example Script
% This script demonstrates how to read and write data to the XPC plugin.
% Before running this script, ensure that the XPC plugin is installed and
% X-Plane is running.
%% Import XPC
addpath('../')
import XPlaneConnect.*
%% Setup
% Create variables and open connection to X-Plane

disp('xplaneconnect Example Script-');
disp('Setting up Simulation');
Socket = openUDP('127.0.0.1', 49009);

%% Ensure connected
getDREFs('sim/test/test_float', Socket);

%% Set position of the player aircraft
disp('Setting position');
 pauseSim(1, Socket);
%       Lat     Lon         Alt   Pitch Roll Heading Gear
POSI = [37.524, -122.06899, 2500, 0,    0,   0,      1];
sendPOSI(POSI, 0, Socket); % Set own aircraft position

%       Lat     Lon           Alt   Pitch Roll Heading Gear
POSI = [37.52465, -122.06899, 2500, 0,    20,  0,      1];
sendPOSI(POSI, 1, Socket); % Place another aircraft just north of us
%% Set rates
disp('Setting rates');
%                 Alpha Velocity PQR
data = struct('h',[18,   3,       16],...
              'd',[0,-999,0,-999,-999,-999,-999,-999;... % Alpha data
                   130,130,130,130,-999,-999,-999,-999;...  % Velocity data
                   0,0,0,-999,-999,-999,-999,-999]);       % PQR data
sendDATA(data, Socket);
 %% Set CTRL
 %                     Throttle
 CTRL = [0,0,0,0.8,0,0];
 sendCTRL(CTRL, 0, Socket);
 pause(5);
 pauseSim(0, Socket);
 pause(5) % Run sim for 5 seconds
 %% Pause sim
 disp('Pausing simulation');
 pauseSim(1, Socket);
 pause(5);
 disp('Unpausing simulation');
 pauseSim(0, Socket);
 pause(10) % Run sim for 10 seconds
 %% Use DREF to raise landing gear
 disp('Raising gear');
 gearDREF = 'sim/cockpit/switches/gear_handle_status';
 sendDREF(gearDREF, 0, Socket);
 pause(10) % Run sim for 10 seconds
 %% Confirm gear and paus status by reading DREFs
 disp('Checking gear');
 pauseDREF = 'sim/operation/override/override_planepath';
 result = getDREFs({gearDREF, pauseDREF}, Socket);
 if result{1} == 0
     disp('Gear stowed');
 else
     disp('Error stowing gear');
 end
 %% Exit
 closeUDP(Socket);
 disp('--End of example program--');
```

4.83. xplane/XPlaneConnect-master/MATLAB/MonitorExample/MonitorExample.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/MonitorExample/MonitorExample.m

Extensão: .m

Conteúdo:

```
%% X-Plane Connect MATLAB Example Script
% This script demonstrates how to read and write data to the XPC plugin.
% Before running this script, ensure that the XPC plugin is installed and
% X-Plane is running.
%% Import XPC
clear all
addpath('../')
import XPlaneConnect.*

Socket = openUDP();
while 1
    posi = getPOSI(0, Socket);
    ctrl = getCTRL(0, Socket);

    fprintf('Loc: (%4f, %4f, %4f) Aileron:%2f Elevator:%2f Rudder:%2f\n', ...
    posi(1), posi(2), posi(3), ctrl(2), ctrl(1), ctrl(3));
    pause(0.1);
end
closeUDP(Socket);
```

4.84. xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Playback.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Playback.m

Extensão: .m

Conteúdo:

```
%% X-Plane Connect MATLAB Playback Example Script
% This script demonstrates how to playback recorded data from X-Plane.
% (See Record.m)
% Before running this script, ensure that the XPC plugin is installed and
% X-Plane is running.
%% Import XPC
addpath('../')
import XPlaneConnect.*

%% Setup
% Create variables and open connection to X-Plane
path = 'MyRecording.txt'; % File containing stored data
interval = 0.1; % Time between snapshots in seconds
duration = 10;
Socket = openUDP(); % Open connection to X-Plane
fd = fopen(path, 'r'); % Open file

disp('X-Plane Connect Playback Example Script');
fprintf('Playing back ''%s'' in %fs increments.\n', path, interval);

%% Start Playback
count = floor(duration / interval);
pauseSim(1);
for i = 1:count
    line = fgetl(fd);
    posi = sscanf(line, '%f, %f, %f, %f, %f, %f, %f\n');
    sendPOSI(posi);
    pause(interval);
end

%% Close connection and file
pauseSim(0);
closeUDP(Socket);
fclose(fd);

disp('Playback complete.');
```

4.85. xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Record.m

Arquivo: xplane/XPlaneConnect-master/MATLAB/PlaybackExample/Record.m

Extensão: .m

Conteúdo:

```
%% X-Plane Connect MATLAB Recording Example Script
% This script demonstrates how to record data from X-Plane that can later
% be played back. (See Playback.m)
% Before running this script, ensure that the XPC plugin is installed and
% X-Plane is running.
%% Import XPC
addpath('../')
import XPlaneConnect.*

%% Setup
% Create variables and open connection to X-Plane
path = 'MyRecording.txt'; % File to save the data in
interval = 0.1; % Time between snapshots in seconds
duration = 10; % Time to record for in seconds
Socket = openUDP(); % Open connection to X-Plane
fd = fopen(path, 'w'); % Open file

disp('X-Plane Connect Recording Example Script');
fprintf('Recording to ''%s'' for %f seconds in %fs increments.\n', path, duration, interval);

%% Start Recording
count = floor(duration / interval);
for i = 1:count
    posi = getPOSI(0, Socket);
    fprintf(fd, '%f, %f, %f, %f, %f, %f, %f\n', ...
        posi(1), posi(2), posi(3), posi(4), posi(5), posi(6), posi(7));
    pause(interval);
end

%% Close connection and file
closeUDP(Socket);
fclose(fd);

disp('Recording complete.');
```

4.86. xplane/XPlaneConnect-master/README.md

Arquivo: xplane/XPlaneConnect-master/README.md

Extensão: .md

Conteúdo:

```
# X-Plane Connect
The X-Plane Connect (XPC) Toolbox is an open source research tool used to
interact with the commercial flight simulator software X-Plane. XPC allows users
to control aircraft and receive state information from aircraft simulated in
X-Plane using functions written in C, C++, Java, MATLAB, or Python in real time over the
network. This research tool has been used to visualize flight paths, test control
algorithms, simulate an active airspace, or generate out-the-window visuals for
in-house flight simulation software. Possible applications include active control
of an XPlane simulation, flight visualization, recording states during a flight,
or interacting with a mission over UDP.

### Architecture
XPC includes an X-Plane plugin (xpcPlugin) and clients written in several
languages that interact with the plugin.

#### Quick Start
To get started using X-Plane Connect, do the following.

1. Purchase and install X-Plane 9, 10 or 11.
2. Download the `XPlaneConnect.zip` file from the latest release on the
[releases](https://github.com/nasa/XPlaneConnect/releases) page.
3. Copy the contents of the .zip archive to the plugin directory (`[X-Plane Directory]/Resources/plugins/`)
4. Write some code using one of the clients to manipulate X-Plane data.

Each client is located in a top-level directory of the repository named for the
client's language. The client directories generally include a `src` folder
containing the client source code, and an `Examples` folder containing sample
code demonstrating how to use the client.

#### Additional Information
For detailed information about XPC and how to use the XPC clients, refer to the
[XPC Wiki](https://github.com/nasa/XPlaneConnect/wiki).

#### Capabilities
The XPC Toolbox allows the user to manipulate the internal state of X-Plane by
reading and setting DataRefs, a complete list of which can be found on the
[X-Plane SDK wiki](http://www.xsquawkbox.net/xpsdk/docs/DataRefs.html).

In addition, several convenience functions are provided, which allow the user to
efficiently execute common commands. These functions include the ability to set
the position and control surfaces of both player and multiplayer aircraft. In
addition, the pause function allows users to easily pause and un-pause X-Plane's
physics simulation engine.

### Compatibility
XPC has been tested with the following software versions:
* Windows: Vista, 7, & 8
* Mac OSX: 10.8-10.14
* Linux: Tested on Red Hat Enterprise Linux Workstation release 6.6
* X-Plane: 9, 10 & 11

### Contributing
All contributions are welcome! If you are having problems with the plugin, please
open an issue on GitHub or email [Chris Teubert](mailto:christopher.a.teubert@nasa.gov).
If you would like to contribute directly, please feel free to open a pull request
against the "develop" branch. Pull requests will be evaluated and integrated into
the next official release.


### Notices
Copyright ©2013-2018 United States Government as represented by the Administrator
of the National Aeronautics and Space Administration.  All Rights Reserved.

No Warranty: THE SUBJECT SOFTWARE IS PROVIDED "AS IS" WITHOUT ANY WARRANTY OF ANY
KIND, EITHER EXPRESSED, IMPLIED, OR STATUTORY, INCLUDING, BUT NOT LIMITED TO, ANY
WARRANTY THAT THE SUBJECT SOFTWARE WILL CONFORM TO SPECIFICATIONS, ANY IMPLIED
WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR FREEDOM FROM
INFRINGEMENT, ANY WARRANTY THAT THE SUBJECT SOFTWARE WILL BE ERROR FREE, OR ANY
WARRANTY THAT DOCUMENTATION, IF PROVIDED, WILL CONFORM TO THE SUBJECT SOFTWARE.
THIS AGREEMENT DOES NOT, IN ANY MANNER, CONSTITUTE AN ENDORSEMENT BY GOVERNMENT
AGENCY OR ANY PRIOR RECIPIENT OF ANY RESULTS, RESULTING DESIGNS, HARDWARE,
SOFTWARE PRODUCTS OR ANY OTHER APPLICATIONS RESULTING FROM USE OF THE SUBJECT
SOFTWARE.  FURTHER, GOVERNMENT AGENCY DISCLAIMS ALL WARRANTIES AND LIABILITIES
REGARDING THIRD-PARTY SOFTWARE, IF PRESENT IN THE ORIGINAL SOFTWARE, AND
DISTRIBUTES IT "AS IS."

Waiver and Indemnity:  RECIPIENT AGREES TO WAIVE ANY AND ALL CLAIMS AGAINST THE
UNITED STATES GOVERNMENT, ITS CONTRACTORS AND SUBCONTRACTORS, AS WELL AS ANY
```

PRIOR RECIPIENT. IF RECIPIENT'S USE OF THE SUBJECT SOFTWARE RESULTS IN ANY LIABILITIES, DEMANDS, DAMAGES, EXPENSES OR LOSSES ARISING FROM SUCH USE, INCLUDING ANY DAMAGES FROM PRODUCTS BASED ON, OR RESULTING FROM, RECIPIENT'S USE OF THE SUBJECT SOFTWARE, RECIPIENT SHALL INDEMNIFY AND HOLD HARMLESS THE UNITED STATES GOVERNMENT, ITS CONTRACTORS AND SUBCONTRACTORS, AS WELL AS ANY PRIOR RECIPIENT, TO THE EXTENT PERMITTED BY LAW. RECIPIENT'S SOLE REMEDY FOR ANY SUCH MATTER SHALL BE THE IMMEDIATE, UNILATERAL TERMINATION OF THIS AGREEMENT.

X-Plane API
Copyright (c) 2008, Sandy Barbour and Ben Supnik All rights reserved.
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
* Neither the names of the authors nor that of X-Plane or Laminar Research may be used to endorse or promote products derived from this software without specific prior written permission from the authors or Laminar Research, respectively.

X-Plane API SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.